TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



ĐẶNG HỒ THIÊN PHÚC - 52300145

TRẦN PHẠM ANH QUÂN - 52300150

NGHIÊN CỨU VÀ ỨNG DỤNG MÔ HÌNH AGENTIC RAG TRONG HỆ THỐNG CÁ NHÂN HÓA MUA SẮM TRÊN NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ

DỰ ÁN CÔNG NGHỆ THÔNG TIN

KHOA HỌC MÁY TÍNH

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



ĐẶNG HỒ THIÊN PHÚC - 52300145

TRẦN PHẠM ANH QUÂN – 52300150

NGHIÊN CỨU VÀ ỨNG DỤNG MÔ HÌNH AGENTIC RAG TRONG HỆ THỐNG CÁ NHÂN HÓA MUA SẮM TRÊN NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ

DỰ ÁN CÔNG NGHỆ THÔNG TIN

KHOA HỌC MÁY TÍNH

Người hướng dẫn

TS. Trần Trung Tín

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn đến TS. Trần Trung Tín đã tận tình giúp đỡ và tạo điều kiện cho chúng em trong quá trình học tập. Thầy là người đã trực tiếp giảng dạy, hướng dẫn tâm huyết để chúng em hoàn thành tài liệu báo cáo của Dự án Công nghệ Thông tin.

Chúng em đã vận dụng những kiến thức đã học được trên lớp và dành thời gian tìm hiểu thêm nhiều kiến thức kiên quan đến môn học để có thêm thông tin để hoàn thành bài tiểu luận này. Tuy nhiên, vốn kiến thức của chúng em còn hạn hẹp và chưa có nhiều kinh nghiệm trong thực tiễn nên khó tránh được những sai sót và hạn chế trong phần nội dung. Rất mong thầy cho em thêm những góp ý, nhận xét để tài liệu báo cáo của chúng em được hoàn thiện và trực quan hơn.

Chúng em xin chân thành cảm ơn thầy.

Trân trọng!

TP. Hồ Chí Minh, ngày 20 tháng 10 năm 2026

Tác giả 1 *Tác giả 2*

Đặng Hồ Thiên Phúc **Trần Phạm Anh Quân**

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của TS. Trần Trung Tín. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 20 tháng 10 năm 2026

Tác giả 1 *Tác giả 2*

Đặng Hồ Thiên Phúc **Trần Phạm Anh Quân**

NGHIÊN CỨU VÀ ỨNG DỤNG MÔ HÌNH AGENTIC RAG TRONG HỆ THỐNG CÁ NHÂN HÓA MUA SẮM TRÊN NỀN TẢNG THƯƠNG MẠI ĐIỆN TỬ

TÓM TẮT

Sự phát triển của thương mại điện tử cùng với sự gia tăng về số lượng và tính đa dạng của sản phẩm đặt ra yêu cầu nâng cao khả năng cá nhân hóa và hỗ trợ ra quyết định trong quá trình mua sắm. Các hệ thống gợi ý truyền thống chủ yếu tập trung vào việc dự đoán sở thích hoặc xếp hạng sản phẩm, do đó còn gặp hạn chế khi xử lý các yêu cầu có nhiều tiêu chí, các ràng buộc thay đổi theo ngữ cảnh và nhu cầu tương tác qua nhiều bước. Trước thực tế đó, đề tài "Nghiên cứu và Ứng dụng Mô hình Agentic RAG trong Hệ thống Cá nhân hóa Mua Sắm trên nền tảng Thương mại Điện tử" hướng đến nghiên cứu và xây dựng một hệ thống gợi ý sản phẩm có định hướng mục tiêu, kết hợp khả năng truy xuất thông tin với cơ chế điều phối tác vụ dựa trên mô hình ngôn ngữ lớn.

Hệ thống được định hướng xây dựng theo kiến trúc Agentic RAG, trong đó mô hình ngôn ngữ lớn đảm nhiệm việc diễn giải yêu cầu bằng ngôn ngữ tự nhiên, xác định các tiêu chí cần đáp ứng và điều phối quá trình xử lý. Các phương pháp gợi ý sản phẩm được sử dụng để tạo hoặc xếp hạng danh sách ứng viên, cơ chế truy xuất thông tin khai thác dữ liệu sản phẩm và các nguồn tri thức liên quan, trong khi các công cụ chuyên biệt hỗ trợ truy vấn, xử lý dữ liệu và kiểm tra kết quả. Nghiên cứu sử dụng bộ dữ liệu Amazon Reviews 2023, lựa chọn các miền dữ liệu phù hợp để phục vụ việc xây dựng, kiểm thử và đánh giá hệ thống.

Phương pháp đánh giá định lượng được xây dựng một cách toàn diện nhằm xem xét hiệu quả của toàn bộ hệ thống từ chất lượng gợi ý, hành vi thực thi cho đến hiệu quả vận hành thực tế. Đối với khả năng truy xuất ứng viên và chất lượng gợi ý, hệ thống sử dụng các chỉ số như Candidate Recall@K để đo tỷ lệ sản phẩm mục tiêu xuất hiện trong tập ứng viên ban đầu, cùng các chỉ số Recall@K, Precision@K,

NDCG@K và Hit Rate@K để đánh giá mức độ chính xác cũng như chất lượng xếp hạng của danh sách gợi ý cuối cùng, kết hợp phân tích chuyên sâu trên các nhóm người dùng cold, sparse và warm. Về mặt hành vi thực thi và khả năng hoàn thành tác vụ của Agent, các tiêu chí Task Success, Tool Selection, Tool Success, Plan Validity, Execution Trajectory, Constraint Satisfaction và Clarification Behavior được áp dụng để theo dõi quá trình chuyển đổi yêu cầu thành kế hoạch, việc chọn lựa công cụ và mức độ đáp ứng các ràng buộc, qua đó so sánh sự khác biệt giữa các cấu hình Agent (có hoặc không có Memory/Reflection). Cuối cùng, hiệu quả vận hành và chất lượng câu trả lời được đo lường thông qua thời gian xử lý (Execution Latency), số bước thực thi, số lần gọi công cụ, lượng token/chi phí LLM, tính có căn cứ của câu trả lời (Grounding/Invented IDs) và các trạng thái dự phòng (Fallback/Answer Status), giúp đảm bảo câu trả lời vừa đạt mức độ tin cậy vừa tối ưu về mặt chi phí vận hành.

RESEARCH AND APPLICATION OF AGENTIC RAG MODEL IN PERSONALIZED E-COMMERCE SHOPPING SYSTEMS

ABSTRACT

The development of e-commerce, together with the increasing number and diversity of products, has created a need to enhance personalization and decision-making support throughout the shopping process. Traditional recommender systems primarily focus on predicting preferences or ranking products, and therefore face limitations when handling requests involving multiple criteria, context-dependent constraints, and multi-step interactions. In response to these challenges, the study entitled "Research and Application of Agentic RAG Models in Personalized Shopping Systems on E-commerce Platforms" aims to investigate and develop a goal-oriented product recommendation system that combines information retrieval capabilities with task orchestration mechanisms based on large language models.

The system is designed around an Agentic RAG architecture, in which the large language model interprets natural language requests, identifies the criteria to be satisfied, and orchestrates the processing workflow. Product recommendation methods are used to generate or rank candidate lists, information retrieval mechanisms leverage product data and related knowledge sources, while specialized tools support querying, data processing, and result verification. The study uses the Amazon Reviews 2023 dataset, selecting suitable domains to support system development, testing, and evaluation.

A comprehensive quantitative evaluation method is established to assess the effectiveness of the entire system, ranging from recommendation quality and execution behavior to actual operational efficiency. For candidate retrieval capabilities and recommendation quality, the system employs metrics such as Candidate Recall@K to measure the proportion of target products appearing in the initial candidate set, along with Recall@K, Precision@K, NDCG@K, and Hit Rate@K to evaluate the accuracy and ranking quality of the final recommendation

list, combined with in-depth analysis across cold, sparse, and warm user segments. In terms of execution behavior and the Agent's task completion capabilities, criteria including Task Success, Tool Selection, Tool Success, Plan Validity, Execution Trajectory, Constraint Satisfaction, and Clarification Behavior are applied to track the transformation of requests into plans, tool selection, and constraint satisfaction, thereby comparing differences among Agent configurations (with or without Memory/Reflection). Finally, operational efficiency and response quality are measured through processing time (Execution Latency), number of execution steps, tool call frequency, LLM token usage/costs, answer groundedness (Grounding/Invented IDs), and fallback states (Fallback/Answer Status), ensuring that responses are both reliable and cost-effective in operation.

MỤC LỤC

DANH MỤC HÌNH VẼ xiv

DANH MỤC BẢNG BIỂU xv

DANH MỤC CÁC CHỮ VIẾT TẮT..... xvii

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI.....1

1.1 Bối cảnh và động lực nghiên cứu.....1

1.2 Mục tiêu nghiên cứu.....2

1.2.1 Mục tiêu tổng quát 2

1.2.2 Mục tiêu cụ thể.....2

1.3 Phạm vi và giới hạn nghiên cứu.....3

1.3.1 Phạm vi nghiên cứu.....3

1.3.2 Dữ liệu sử dụng.....4

1.3.3 Giới hạn nghiên cứu 4

1.4 Phương pháp nghiên cứu tổng quát.....5

1.4.1 Nghiên cứu tài liệu.....5

1.4.2 Thiết kế và phát triển hệ thống 6

1.4.3 Thực nghiệm và đánh giá định lượng 7

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....9

2.1 Tổng quan về hệ thống gợi ý.....9

2.1.1 Khái niệm và vai trò.....9

2.1.2 Mô hình hóa bài toán gợi ý.....9

2.1.3 Hệ thống gợi ý truyền thống 9

2.1.4 Hệ thống gợi ý hiện đại.....10

2.1.2 So sánh hệ thống gợi ý truyền thống và hiện đại *17*

2.2 Bài toán gợi ý Top-K và Quy trình xử lý 18

2.2.1 Khái niệm bài toán Top-K *18*

2.2.2 Quy trình của một hệ thống gợi ý *18*

2.3 Các thách thức chính trong hệ thống gợi ý 19

2.3.1 Dữ liệu thưa (Data Sparsity) *19*

2.3.2 Vấn đề khởi động lạnh (Cold-start) *19*

2.3.3 Các thách thức trong hệ thống gợi ý dựa trên LLM (Agentic RS) *20*

2.4 Các phương pháp gợi ý chi tiết 20

2.4.1 Popularity-based Recommendation *20*

2.4.2 Collaborative Filtering (CF) *21*

2.4.3 ItemKNN *21*

2.4.4 Bayesian Personalized Ranking (BPR-MF) *22*

2.5 Truy xuất thông tin và Kho tri thức sản phẩm (Information Retrieval & Knowledge Base) 23

2.5.1 Thuật toán tìm kiếm BM25 ..... *23*

2.5.2 Kho tri thức sản phẩm (Product Knowledge Base) *24*

2.6 Mô hình ngôn ngữ lớn, Tác tử thông minh và RAG 25

2.6.1 Mô hình ngôn ngữ lớn (LLM) và Structured Output *25*

2.6.2 LLM Agent và các cơ chế điều phối hành động *26*

2.6.3 Grounded Response Generation & Giảm thiểu ảo giác *28*

2.7 Phương pháp đánh giá hệ thống 28

2.7.1 Các chỉ số đánh giá gợi ý *28*

2.7.2 Đánh giá Agent và Hiệu năng vận hành *29*

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT VÀ THIẾT KẾ HỆ THỐNG..... 31

3.1 Kiến trúc tổng thể 31

3.1.1 Nguyên tắc thiết kế kiến trúc *31*

3.1.2 Các tầng chức năng *32*

3.1.3 Quy tắc phụ thuộc giữa các tầng *34*

3.1.4 Hai luồng xử lý chính *34*

3.2 Dữ liệu và kho tri thức sản phẩm 35

3.2.1 Pipeline xử lý dữ liệu tương tác *35*

3.2.2 Định danh và ánh xạ *38*

3.2.3 ItemCorpus *38*

3.2.4 Phân biệt kho sản phẩm và dữ liệu tương tác *39*

3.3 Mô hình gợi ý và bộ chấm điểm ứng viên 40

3.3.1 Các mô hình gợi ý *40*

3.3.2 Bộ chấm điểm ứng viên *40*

3.4 Hệ thống công cụ 41

3.4.1 Hợp đồng chung của công cụ *41*

3.4.2 Phân nhóm và tổng quan các công cụ *42*

3.4.3 SemanticSearchTool *43*

3.4.4 ItemCFTool *44*

3.4.5 SQLTool *44*

3.4.6 RecoModelTool *45*

3.4.7 QueryTool *46*

3.4.8 MemoryTool *46*

3.4.9 Kết nối công cụ và phân loại lỗi *46*

3.5 Tầng tác tử 48

3.5.1 Lớp truy cập mô hình ngôn ngữ *48*

3.5.2 Bộ công cụ theo từng loại tác tử *48*

3.5.3 Type1Agent: pipeline cố định *48*

3.5.4 PlanExecutor: lập kế hoạch bằng LLM *49*

3.5.5 Reflection và cơ chế thử lại *51*

3.5.6 Memory và cá nhân hóa *52*

3.5.7 Phạm vi hội thoại nhiều lượt [Phần này nhóm chưa hoàn thiện] *52*

3.6 Tầng inference và tạo câu trả lời 53

3.6.1 Vai trò và đầu vào *53*

3.6.2 Phân loại trạng thái *53*

3.6.3 Hai chế độ sinh câu trả lời *54*

3.6.4 Grounding và fallback *54*

3.7 Cấu hình, kiểm thử và quy ước kỹ thuật 55

3.7.1 Cấu hình. *55*

3.7.2 Quy ước cấu trúc mã. *55*

3.7.3 Kiểm thử. *55*

3.8 Hệ thống đánh giá trong kiến trúc 56

3.8.1 Bốn bộ đánh giá *56*

3.8.2 Tách ba nhóm chất lượng *57*

3.8.3 Truy vấn ORACLE và truy vấn thực tế *58*

3.9 Ví dụ xử lý một yêu cầu 58
3.10 Quyết định thiết kế và giới hạn kiến trúc 59
3.10.1 Các quyết định thiết kế chính 59
3.10.2 Giới hạn kiến trúc hiện tại 61
CHƯƠNG 4. THỰC NGHIỆM VÀ THIẾT LẬP ĐÁNH GIÁ 63
4.1 Mục tiêu và nguyên tắc thực nghiệm 63
4.1.1 Mục tiêu đánh giá 63
4.1.2 Nguyên tắc thiết kế thực nghiệm 63
4.2 Dữ liệu và môi trường thực nghiệm 64
4.2.1 Tập dữ liệu và lý do chọn 64
4.2.2 Tiền xử lý và phân chia dữ liệu 65
4.2.3 Kho thông tin sản phẩm 66
4.2.4 Môi trường và cấu hình chạy 66
4.3 Thiết lập đánh giá mô hình gợi ý 67
4.4 Các mô hình cơ sở 67
4.4.1 Giao thức huấn luyện và đánh giá 67
4.4.2 Đánh giá độ bao phủ ứng viên 68
4.5 Thiết lập đánh giá tác tử 68
4.5.1 Bốn tác tử được so sánh 68
4.5.2 Xây dựng bộ yêu cầu đánh giá 69
4.5.3 Tiêu chí thành công và chấm điểm 70
4.5.4 Ghi nhận quỹ đạo và chỉ số tác tử 71
4.5.5 Đánh giá tầng inference 71

4.6 Thiết kế so sánh và ablation 72
4.6.1 So sánh với mô hình cơ sở 72
4.6.2 Ablation theo thành phần 72
4.6.3 Đánh giá nhiều lượt hội thoại 73
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH 74
5.1 Kết quả đánh giá hệ thống gợi ý 74
5.1.1 Kết quả của các mô hình baseline 74
5.1.2 Phân tích Recall@K, Precision@K và NDCG@K 75
5.1.3 Phân tích khả năng đưa sản phẩm mục tiêu vào tập ứng viên 76
5.1.4 Phân tích các trường hợp gợi ý thất bại 77
5.2 Kết quả đánh giá tác tử 77
5.2.1 Kết quả thực hiện các kịch bản đánh giá 77
5.2.2 Phân tích quỹ đạo thực thi (Execution Trajectory) 79
5.2.3 Phân tích khả năng lựa chọn và sử dụng công cụ 79
5.2.4 Phân tích các trường hợp thực thi thất bại 80
5.3 Kết quả thực nghiệm so sánh và Ablation 80
5.3.1 So sánh với baseline 80
5.3.2 Phân tích đóng góp của từng thành phần 80
5.3.3 Phân tích ảnh hưởng của Memory và Reflection 81
5.4 Đánh giá hiệu năng và chi phí thực thi 81
5.5 Thảo luận kết quả 82
5.5.1 Những kết quả đạt được 82
5.5.2 Những vấn đề chưa giải quyết được 82

5.5.3 Các yếu tố ảnh hưởng đến kết quả *83*

CHƯƠNG 6. CẢI THIỆN VÀ HƯỚNG PHÁT TRIỂN **84**

6.1 Cải thiện cần thực hiện đến cuối kỳ 84

6.1.1 Các vấn đề cần giải quyết *84*

6.1.2 Cải thiện chất lượng và độ bao phủ của tập ứng viên *85*

6.1.3 Hoàn thiện đánh giá mô hình gợi ý và tác tử *86*

6.1.4 Củng cố độ tin cậy của công cụ và tác tử *86*

6.1.5 Hoàn thiện tầng inference và giao diện demo *87*

6.1.6 Đo thời gian và chi phí *87*

6.1.7 Đánh giá nhiều lượt hội thoại ở phạm vi nhỏ *88*

6.1.8 Kế hoạch thực hiện đến hạn nộp *88*

6.2 Hướng phát triển sau đồ án 89

KẾT LUẬN **92**

TÀI LIỆU THAM KHẢO **94**

DANH MỤC HÌNH VẼ

Hình 2.1. Sơ đồ biểu thị trục mức độ tự chủ từ L0 đến L6 12

Hình 2.2. Agent-assisted Recommendation: agent hỗ trợ các khâu quanh mô hình gợi ý trung tâm 13

Hình 2.3. Agent-as-Recommender: (a) một agent, (b) nhiều agent 15

Hình 2.4. Agent-as-User-Simulator: agent đóng vai người dùng tương tác với hệ thống gợi ý 16

Hình 3.1. Kiến trúc tổng thể của hệ thống 32

Hình 3.2. Pipeline xử lý dữ liệu ba giai đoạn 36

Hình 3.3. Quy trình lập kế hoạch, kiểm tra và thực thi của PlanExecutor 50

DANH MỤC BẢNG BIỂU

Bảng 2.1. So sánh ba hướng hệ thống gợi ý dựa trên Agent.....17

Bảng 3.2. Nhiệm vụ, cách thực hiện và lý do của từng tầng 34

Bảng 3.3. Ba giai đoạn của pipeline dữ liệu 37

Bảng 3.4. Tổng quan các công cụ 43

Bảng 3.5. Phân loại các tình huống lỗi khi thực thi 48

Bảng 3.6. Các trạng thái kết quả của tầng inference.....54

Bảng 3.7. Bốn bộ đánh giá trong hệ thống.....57

Bảng 3.8. Ba nhóm chất lượng được theo dõi độc lập.....58

Bảng 3.9. Các quyết định thiết kế chính và đánh đổi.....61

Bảng 4.2. Phân nhóm người dùng theo độ dài lịch sử 65

Bảng 4.3. Các giai đoạn tiền xử lý dữ liệu.....65

Bảng 4.4 Các mô hình gợi ý cơ sở.....67

Bảng 4.6. Các cấu hình tác tử trong đánh giá 69

Bảng 4.7. Các loại yêu cầu đánh giá 70

Bảng 4.8. Nhóm chỉ số đánh giá tác tử 71

Bảng 4.9. Các thí nghiệm ablation.....73

Bảng 5.1. Các lượt chạy được dùng trong chương này.....74

Bảng 5.2. Kết quả Popularity theo lượt chạy và phạm vi 75

Bảng 5.3. Candidate Recall theo phân đoạn, nguồn ứng viên và ngân sách 77

Bảng 5.4. Một số kết quả đánh giá tác tử theo loại yêu cầu 78

Bảng 6.1. Các vấn đề hiện tại và cải thiện tương ứng.....85

Bảng 6.2. Kế hoạch theo tuần đến hạn nộp.....89

Bảng 6.3. Tổng hợp các hướng phát triển.....91

DANH MỤC CÁC CHỮ VIẾT TẮT

Thuật ngữ viết tắt	Thuật ngữ đầy đủ	Thuật ngữ tiếng Việt
@K	At K	Tại K / Trong K vị trí đầu
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
BM25	Best Matching 25	Thuật toán xếp hạng truy xuất thông tin xác suất BM25
BPR	Bayesian Personalized Ranking	Xếp hạng cá nhân hóa Bayes
CF	Collaborative Filtering	Lọc cộng tác
CPU	Central Processing Unit	Bộ xử lý trung tâm
DCG	Discounted Cumulative Gain	Độ lợi tích lũy giảm dần
EDA	Exploratory Data Analysis	Phân tích dữ liệu khám phá
FTS	Full-Text Search	Tìm kiếm toàn văn
GPU	Graphics Processing Unit	Bộ xử lý đồ họa

ID	Identifier	Mã định danh
IDF	Inverse Document Frequency	Tần suất văn bản nghịch đảo
IR	Information Retrieval	Truy xuất thông tin
JSON	JavaScript Object Notation	Định dạng ký họa đối tượng JavaScript
JSONL	JSON Lines	Định dạng dữ liệu JSON theo từng dòng
KNN / ItemKNN	K-Nearest Neighbors / Item-based K-Nearest Neighbors	K láng giềng gần nhất dựa trên sản phẩm
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MAS	Multi-Agent System	Hệ thống đa tác tử
MF	Matrix Factorization	Phân rã ma trận
MRR	Mean Reciprocal Rank	Thứ hạng nghịch đảo trung bình
NDCG	Normalized Discounted Cumulative Gain	Độ lợi tích lũy giảm dần chuẩn hóa
RAG	Retrieval-Augmented Generation	Mô hình sinh tăng cường bằng truy xuất thông tin

ReAct	Reasoning and Acting	Cơ chế kết hợp suy luận và hành động
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
YAML	YAML Ain't Markup Language	Định dạng cấu hình dữ liệu dạng văn bản

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh và động lực nghiên cứu

Sự phát triển của thương mại điện tử làm gia tăng số lượng sản phẩm và lượng thông tin mà người tiêu dùng cần xem xét trước khi đưa ra quyết định mua sắm. Trong bối cảnh đó, hệ thống gợi ý (Recommender System) đóng vai trò hỗ trợ người dùng khám phá những sản phẩm phù hợp với sở thích, nhu cầu và lịch sử tương tác. Các phương pháp như gợi ý dựa trên mức độ phổ biến (Popularity-based), lọc cộng tác (Collaborative Filtering) và phân rã ma trận (Matrix Factorization) cung cấp những cách tiếp cận khác nhau để dự đoán mức độ quan tâm của người dùng và xếp hạng sản phẩm.

Tuy nhiên, bài toán gợi ý trong thực tế không chỉ dừng lại ở việc xác định danh sách sản phẩm có điểm phù hợp cao. Người dùng có thể đưa ra yêu cầu kết hợp nhiều tiêu chí, chẳng hạn như sở thích cá nhân, ngân sách, danh mục sản phẩm, mức đánh giá hoặc những điều kiện cụ thể khác. Các tiêu chí này có thể cần được làm rõ, đối chiếu với dữ liệu sản phẩm và xử lý qua nhiều bước trước khi đưa ra kết quả cuối cùng. Vì vậy, bên cạnh chất lượng của mô hình gợi ý, khả năng điều phối quá trình xử lý yêu cầu cũng là một khía cạnh cần được nghiên cứu.

Mô hình ngôn ngữ lớn (Large Language Model – LLM) cung cấp khả năng diễn giải yêu cầu bằng ngôn ngữ tự nhiên, tạo đầu ra có cấu trúc và hỗ trợ lựa chọn hành động. Khi kết hợp với cơ chế truy xuất thông tin (Retrieval-Augmented Generation – RAG), LLM có thể sử dụng dữ liệu được truy xuất làm căn cứ cho quá trình tạo câu trả lời. Trên cơ sở đó, Agentic RAG tổ chức việc xử lý thành nhiều bước, trong đó tác tử (Agent) có thể lập kế hoạch, lựa chọn công cụ, tiếp nhận kết quả và điều chỉnh quá trình thực thi theo mục tiêu.

Trong bài toán gợi ý sản phẩm, cách tiếp cận này cho phép nghiên cứu sự phối hợp giữa mô hình gợi ý, kho tri thức sản phẩm, các công cụ xử lý dữ liệu và cơ chế

điều phối dựa trên LLM. Thay vì giao toàn bộ nhiệm vụ lựa chọn sản phẩm cho LLM, hệ thống có thể sử dụng các mô hình gợi ý để cung cấp tín hiệu xếp hạng, các công cụ để truy xuất và kiểm tra dữ liệu, còn Agent đảm nhiệm việc tổ chức các thao tác cần thiết. Kết quả sau đó được chuyển đến tầng sinh câu trả lời để diễn đạt thông tin dựa trên những sản phẩm đã được lựa chọn.

Từ những vấn đề trên, đề tài "Nghiên cứu và Ứng dụng Mô hình Agentic RAG trong Hệ thống Cá nhân hóa Mua Sắm trên nền tảng Thương mại Điện tử" tập trung nghiên cứu và xây dựng một nguyên mẫu hệ thống gợi ý sản phẩm có định hướng mục tiêu. Nghiên cứu đồng thời xem xét chất lượng gợi ý, khả năng truy xuất ứng viên, hành vi thực thi của Agent và hiệu quả vận hành. Qua đó, đề tài hướng đến việc xác định những lợi ích, giới hạn và điều kiện cần thiết khi kết hợp hệ thống gợi ý truyền thống với cơ chế điều phối tác vụ dựa trên LLM.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và phát triển một nguyên mẫu hệ thống gợi ý sản phẩm theo hướng Agentic RAG, kết hợp các mô hình gợi ý, cơ chế truy xuất thông tin sản phẩm, hệ thống công cụ và tác tử điều phối dựa trên mô hình ngôn ngữ lớn. Trên cơ sở đó, xây dựng quy trình thực nghiệm nhằm đánh giá chất lượng gợi ý, khả năng thực hiện tác vụ và hiệu quả vận hành, từ đó phân tích tính phù hợp cũng như những hạn chế của cách tiếp cận được nghiên cứu.

1.2.2 Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, đề tài tập trung vào các mục tiêu cụ thể sau:

- Nghiên cứu và hệ thống hóa cơ sở lý thuyết: Khảo sát các phương pháp gợi ý sản phẩm, truy xuất thông tin, mô hình ngôn ngữ lớn và kiến trúc Agent; xác định vai trò của từng nhóm kỹ thuật trong bài toán cá nhân hóa mua sắm.

- Xây dựng pipeline dữ liệu và kho tri thức sản phẩm: Tiền xử lý dữ liệu Amazon Reviews 2023, tổ chức dữ liệu tương tác người dùng – sản phẩm và thông tin mô tả sản phẩm, đồng thời xây dựng các tập dữ liệu phục vụ huấn luyện và đánh giá.
- Phát triển các thành phần gợi ý và truy xuất: Triển khai các mô hình cơ sở, cơ chế truy xuất ứng viên, công cụ truy vấn dữ liệu và công cụ xếp hạng nhằm cung cấp thông tin cho quá trình xử lý yêu cầu.
- Thiết kế cơ chế điều phối của Agent: Tổ chức quá trình tiếp nhận yêu cầu, lập kế hoạch, gọi công cụ, xử lý kết quả và quản lý trạng thái thực thi; nghiên cứu ảnh hưởng của Memory và Reflection thông qua các cấu hình có thể so sánh.
- Xây dựng quy trình đánh giá nhiều tầng: Đánh giá độ bao phủ của tập ứng viên, chất lượng danh sách gợi ý, khả năng hoàn thành tác vụ và hành vi sử dụng công cụ; đồng thời ghi nhận thời gian thực thi, số bước xử lý và chi phí sử dụng LLM khi có dữ liệu phù hợp.
- Phân tích kết quả và xác định hướng cải thiện: So sánh các cấu hình thực nghiệm trong điều kiện được kiểm soát, phân tích các trường hợp thất bại và xác định những thành phần cần hoàn thiện để nâng cao chất lượng, độ tin cậy và hiệu quả vận hành của hệ thống.

1.3 Phạm vi và giới hạn nghiên cứu

1.3.1 Phạm vi nghiên cứu

- Về mặt dữ liệu: Hệ thống giới hạn phạm vi thử nghiệm trên bộ dữ liệu Amazon Reviews 2023, trong đó thu hẹp trọng tâm vào các miền dữ liệu đặc thù (như Video Games, Toys and Games, Electronics).
- Về kỹ thuật và mô hình: Đề tài không tập trung vào việc tinh chỉnh lại (fine-tuning) toàn bộ trọng số của một mô hình ngôn ngữ lớn (LLM) khổng lồ. Thay vào đó, nghiên cứu giới hạn ở việc thiết kế kiến trúc phân rã nhiệm vụ và phối hợp các tác tử (Agent). Các phương pháp gợi ý cơ sở được sử dụng trong phạm

vi thử nghiệm bao gồm Random, Popularity, ItemKNN và BPR-MF, kết hợp cùng thuật toán truy xuất từ khóa BM25.

- Về mặt kiến trúc hệ thống: Phạm vi thiết kế tập trung vào việc điều phối của Agent trong kiến trúc Agentic RAG (bao gồm khả năng gọi công cụ, sử dụng bộ nhớ Memory và cơ chế tự phản tư Reflection) để xử lý các truy vấn phức tạp hoặc người dùng có lịch sử tương tác đa dạng. Tầng sinh câu trả lời có nhiệm vụ diễn đạt kết quả đã được lựa chọn, không thay thế chức năng truy xuất hoặc xếp hạng sản phẩm.
- Về mặt đánh giá: Nghiên cứu chỉ đánh giá hệ thống ở mức độ nguyên mẫu (prototype) trong môi trường thực nghiệm với các chỉ số đo lường như: chất lượng xếp hạng (Recall@K, NDCG@K, Precision@K), độ bao phủ của tập ứng viên, và khả năng hoàn thành tác vụ của Agent. Hệ thống chưa tích hợp đầy đủ các chức năng thương mại điện tử thực tế như giao dịch, quản lý tồn kho trực tiếp hay đo lường chuyển đổi mua hàng.

1.3.2 Dữ liệu sử dụng

Đề tài sử dụng bộ dữ liệu đánh giá sản phẩm Amazon Reviews 2023, bao gồm thông tin về tương tác của người dùng với sản phẩm và dữ liệu mô tả sản phẩm. Dữ liệu tương tác phản ánh hành vi đánh giá của người dùng, trong khi dữ liệu mô tả cung cấp thông tin hỗ trợ nhận diện và tìm hiểu đặc điểm của từng sản phẩm.

Nguồn dữ liệu này được sử dụng để xây dựng tập dữ liệu phục vụ huấn luyện và đánh giá các mô hình gợi ý, đồng thời cung cấp thông tin đầu vào cho hệ thống đề xuất sản phẩm dựa trên tác tử ngôn ngữ lớn (LLM Agent). Trước khi đưa vào thực nghiệm, dữ liệu được xử lý và tổ chức lại nhằm đáp ứng yêu cầu của hệ thống và quy trình đánh giá.

1.3.3 Giới hạn nghiên cứu

Nghiên cứu được thực hiện trên một nguyên mẫu hệ thống với phạm vi dữ liệu, mô hình và tài nguyên tính toán xác định. Các giới hạn chính, nguyên nhân và hướng xử lý tương ứng được trình bày như sau:

- Tính đại diện của dữ liệu: Tính đại diện của dữ liệu: Nghiên cứu chỉ sử dụng ba domain thuộc Amazon Reviews 2023. Vì vậy, kết quả chỉ phản ánh phạm vi dữ liệu đã đánh giá, chưa thể khái quát cho mọi loại sản phẩm, nền tảng hoặc nhóm người dùng.
- Khả năng khái quát hóa kết quả thực nghiệm: Kết quả có thể chịu ảnh hưởng bởi dữ liệu, cấu hình và số lần chạy. Do đó, nghiên cứu phân biệt kết quả quan sát với kết luận nhân quả, chỉ so sánh các cấu hình có điều kiện tương thích và xem các thí nghiệm chưa đủ kiểm soát là bằng chứng chẩn đoán.
- Khả năng triển khai thương mại: Hệ thống hiện là nguyên mẫu nghiên cứu, chưa tích hợp đầy đủ các chức năng như giao dịch, tồn kho trực tiếp và đo lường chuyển đổi mua hàng. Vì vậy, việc đánh giá chỉ giới hạn ở chất lượng gợi ý, hành vi thực thi và hiệu quả vận hành trong môi trường thực nghiệm.

1.4 Phương pháp nghiên cứu tổng quát

Đề tài kết hợp nghiên cứu tài liệu, thiết kế và phát triển hệ thống, cùng với thực nghiệm định lượng và phân tích kết quả. Quy trình nghiên cứu được tổ chức theo ba hướng: hệ thống hóa cơ sở lý thuyết, hiện thực hóa kiến trúc và kiểm chứng các thành phần thông qua dữ liệu cùng các kịch bản thực nghiệm.

1.4.1 Nghiên cứu tài liệu

1.1.1 Nghiên cứu tài liệu

Nghiên cứu tài liệu của đề tài dựa trên bài báo khảo sát "Autonomous Information Seeking: A Roadmap for Agentic Recommender Systems" (Lin et al., 2026). Đóng góp chính của bài báo là một hệ thống phân loại (taxonomy) các hệ thống gợi ý dựa trên Agent, nên đề tài dùng taxonomy này làm cơ sở để định vị hướng tiếp cận, sau đó tham khảo các mô hình và framework cụ thể được tổng hợp trong bài. Công việc gồm ba phần:

- Nghiên cứu phân loại theo mức độ tự chủ (Level of Autonomy): xem xét thang mức tự chủ từ L0 đến L6, trong đó đa phần các mô hình và framework hiện có nằm trong khoảng L2 đến L5: từ truy xuất có căn cứ (L2 – Retrieval-

Augmented RecSys), gọi công cụ (L3 – Tool-Driven), lập kế hoạch chủ động (L4 – Single-Agent Planner) đến điều phối đa agent (L5 – Multi-Agent Orchestration). Từ đó xác định mức tự chủ phù hợp cho hệ thống của đề tài.

- Nghiên cứu phân loại theo mô hình cốt lõi: trong ba hướng của taxonomy, đề tài tập trung vào Agent-assisted Recommendation (agent hỗ trợ hệ thống gợi ý truyền thống trong việc truy xuất và làm phong phú thông tin) và Agent-as-Recommender (agent trực tiếp điều phối, ra quyết định và đưa ra gợi ý), vì hai hướng này có giao thoa lớn với bài toán và chiếm phần lớn các nghiên cứu được khảo sát. Hướng Agent-as-User-Simulator không nằm trong phạm vi vì chi phí mô phỏng bằng LLM lớn.
- Tham khảo các mô hình và framework trong taxonomy: từ danh sách các công trình mà bài báo phân loại, đề tài chọn đọc những công trình gần với bài toán, gồm các framework gọi công cụ và lập kế hoạch trong gợi ý, các framework kết hợp truy xuất với tác tử, các framework dùng bộ nhớ và tự phản hồi, và các framework đa agent. Từ đó, đề tài rút ra các cơ chế cấu thành một hệ thống Agentic RAG, gồm lập kế hoạch (Planning), sử dụng công cụ (Tool Calling), quản lý bộ nhớ (Memory) và tự phản tư (Reflection), làm căn cứ cho thiết kế ở Chương 3.

1.4.2 Thiết kế và phát triển hệ thống

Thay vì thiết kế hệ thống theo một luồng xử lý dữ liệu tuyến tính truyền thống, phương pháp phát triển được định hướng chặt chẽ theo framework Agentic RAG, biến quá trình gợi ý thành một chuỗi ra quyết định đa bước do Agent điều phối. Hệ thống được xây dựng và tích hợp theo các thành phần chức năng của framework:

- Xây dựng kho tri thức và công cụ (Tool Integration & Product Corpus): Tiền xử lý tập dữ liệu Amazon Reviews 2023 để xây dựng kho tri thức (Corpus) hỗ trợ truy vấn. Thiết kế hệ thống công cụ (Tools) bao gồm truy xuất ngữ nghĩa (Semantic Search), lọc cộng tác (ItemCF), xếp hạng ứng viên (RecoModel) và

tra cứu chi tiết (Query), đóng vai trò như các giao diện để Agent tương tác với dữ liệu.

- Thiết kế framework điều phối (Planning & Execution): Áp dụng framework điều phối bằng LLM, nơi Agent tiếp nhận yêu cầu ngôn ngữ tự nhiên, thực hiện phân rã nhiệm vụ thành một kế hoạch (Planning) dạng JSON và tự động gọi các công cụ (Tool Calling) để thu thập thông tin. Quá trình này tuân theo vòng lặp: quan sát - lập kế hoạch - hành động - kiểm chứng.
- Tích hợp Memory và Reflection vào framework: Triển khai cơ chế Memory để Agent có khả năng duy trì ngữ cảnh và hồ sơ người dùng qua các bước xử lý hoặc qua nhiều lượt hội thoại. Bổ sung cơ chế Reflection (tự đánh giá lại) để hệ thống tự động kiểm tra chéo các ràng buộc, phát hiện thiếu sót từ bước trước và điều chỉnh kế hoạch, qua đó hạn chế hiện tượng ảo giác (hallucination).
- Xây dựng tầng diễn đạt (Inference/Grounded Generation): Tầng Inference được thiết kế để xử lý đầu ra từ Agent, biến các sản phẩm đã được chọn thành câu trả lời tự nhiên cho người dùng dựa trên bằng chứng cụ thể đã truy xuất được.

1.4.3 Thực nghiệm và đánh giá định lượng

Phương pháp đánh giá định lượng được xây dựng một cách toàn diện nhằm xem xét hiệu quả của toàn bộ hệ thống từ chất lượng gợi ý, hành vi thực thi cho đến hiệu quả vận hành thực tế. Đối với khả năng truy xuất ứng viên và chất lượng gợi ý, hệ thống sử dụng các chỉ số như Candidate Recall@K để đo tỷ lệ sản phẩm mục tiêu xuất hiện trong tập ứng viên ban đầu, cùng các chỉ số Recall@K, Precision@K, NDCG@K và Hit Rate@K để đánh giá mức độ chính xác cũng như chất lượng xếp hạng của danh sách gợi ý cuối cùng, kết hợp phân tích chuyên sâu trên các nhóm người dùng cold, sparse và warm. Về mặt hành vi thực thi và khả năng hoàn thành tác vụ của Agent, các tiêu chí Task Success, Tool Selection, Tool Success, Plan Validity, Execution Trajectory, Constraint Satisfaction và Clarification Behavior

được áp dụng để theo dõi quá trình chuyển đổi yêu cầu thành kế hoạch, việc chọn lựa công cụ và mức độ đáp ứng các ràng buộc, qua đó so sánh sự khác biệt giữa các cấu hình Agent (có hoặc không có Memory/Reflection). Cuối cùng, hiệu quả vận hành và chất lượng câu trả lời được đo lường thông qua thời gian xử lý (Execution Latency), số bước thực thi, số lần gọi công cụ, lượng token/chi phí LLM, tính có căn cứ của câu trả lời (Grounding/Invented IDs) và các trạng thái dự phòng (Fallback/Answer Status), giúp đảm bảo câu trả lời vừa đạt mức độ tin cậy vừa tối ưu về mặt chi phí vận hành.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về hệ thống gợi ý

2.1.1 Khái niệm và vai trò

Hệ thống gợi ý là các hệ thống phần mềm được thiết kế nhằm hỗ trợ người dùng lựa chọn những đối tượng phù hợp với sở thích, nhu cầu hoặc mục tiêu của họ từ một tập hợp lớn các lựa chọn. Các đối tượng này có thể là sản phẩm, phim ảnh, âm nhạc, tin tức hoặc dịch vụ.

Trong thương mại điện tử, hệ thống gợi ý giúp người dùng khám phá sản phẩm, giảm thời gian tìm kiếm và hỗ trợ quá trình ra quyết định mua sắm. Một hệ thống gợi ý thường khai thác thông tin từ nhiều nguồn, bao gồm lịch sử tương tác của người dùng, thuộc tính sản phẩm, thông tin ngữ cảnh và các mẫu hành vi được học từ dữ liệu. Tùy theo phương pháp được sử dụng, hệ thống có thể dự đoán mức độ quan tâm của người dùng đối với từng sản phẩm hoặc trực tiếp tạo ra danh sách các sản phẩm được xếp hạng theo mức độ phù hợp.

2.1.2 Mô hình hóa bài toán gợi ý

- Một hệ thống gợi ý tổng quát được mô tả thông qua hàm dự đoán điểm phù hợp:

$$\hat{r}_{ui} = f(u, i; \theta) \quad (1)$$

  Trong đó:

  - $u \in U$: Người dùng u thuộc tập người dùng U.
  - $i \in I$: Sản phẩm i thuộc tập sản phẩm I.
  - $\hat{r}_{ui}$: Điểm phù hợp dự đoán giữa người dùng u và sản phẩm i.
  - θ: Tập tham số của mô hình.
- Điểm phù hợp $\hat{r}_{ui}$ được sử dụng để xếp hạng và lựa chọn các sản phẩm tiềm năng nhất cho người dùng.

2.1.3 Hệ thống gợi ý truyền thống

Hệ thống gợi ý truyền thống chủ yếu dựa trên dữ liệu tương tác tĩnh giữa người dùng và sản phẩm (lịch sử mua hàng, lượt nhấp, lượt xem hoặc điểm đánh giá). Các phương pháp tiêu biểu gồm: Content-based Filtering, Collaborative Filtering, Matrix Factorization và các mô hình xếp hạng như BPR.

- Đặc điểm chính:
  - Học từ dữ liệu lịch sử tương tác đã được thu thập trước đó.
  - Tối ưu hóa một mục tiêu cố định (ví dụ: dự đoán điểm đánh giá hoặc xếp hạng Top-K).
  - Quy trình xử lý tuyến tính: truy xuất ứng viên - xếp hạng - trả về kết quả.
  - Khó thích ứng tức thì khi nhu cầu người dùng thay đổi trong phiên tương tác.

2.1.4 Hệ thống gợi ý hiện đại

Hệ thống gợi ý hiện đại mở rộng từ việc xếp hạng tĩnh sang các hệ thống có khả năng hiểu ngữ cảnh, tương tác nhiều lượt và thích ứng linh hoạt với mục tiêu của người dùng.

- Các hướng tiếp cận tiêu biểu bao gồm:
  - Deep learning-based recommendation
  - Sequential recommendation
  - Graph-based recommendation
  - Retrieval-augmented recommendation (RAG)
  - Agentic recommendation (Wu et al., 2022; Lin et al., 2026).
- Đặc điểm nổi bật:
  - Kết hợp đa nguồn tín hiệu: hành vi, nội dung sản phẩm, ngữ cảnh, kiến thức đồ thị và văn bản tự nhiên.
  - Sử dụng mô hình học sâu hoặc mô hình ngôn ngữ lớn (LLM) để học biểu diễn người dùng và sản phẩm.
  - Hỗ trợ tương tác nhiều lượt, giúp làm rõ nhu cầu và điều chỉnh gợi ý theo phản hồi thời gian thực.

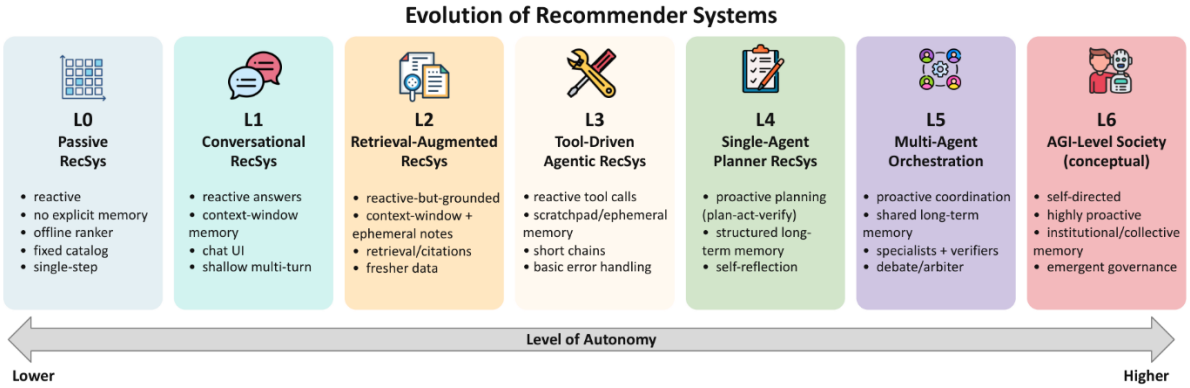
- Tích hợp truy xuất thông tin (IR) và công cụ ngoài để kiểm tra thuộc tính, giá cả, ràng buộc mua sắm.
- Cung cấp khả năng giải thích dựa trên bằng chứng thực tế thay vì chỉ dựa vào điểm số số học.

2.1.1 Phân loại hệ thống gợi ý dựa trên Agent

- Theo taxonomy của Lin et al. (2026), hệ thống gợi ý dựa trên Agent được phân thành ba hướng chính, phân biệt theo vai trò của agent trong quá trình gợi ý:
  - Agent là thành phần hỗ trợ cho một mô hình gợi ý có sẵn.
  - Agent chính là bộ gợi ý.
  - Agent đóng vai người dùng để mô phỏng.
- Song song với trục này, taxonomy còn xếp các framework theo mức độ tự chủ (Level of Autonomy), gồm bảy mức từ L0 đến L6:
  - L0 – Passive RecSys: bộ xếp hạng offline phản ứng trên một danh mục cố định, không có bộ nhớ tường minh và chỉ xử lý trong một bước.
  - L1 – Conversational RecSys: trả lời theo yêu cầu qua giao diện hội thoại, dùng cửa sổ ngữ cảnh làm bộ nhớ, tương tác nhiều lượt ở mức nông.
  - L2 – Retrieval-Augmented RecSys: vẫn phản ứng theo từng yêu cầu nhưng đầu ra được truy xuất (RAG) làm căn cứ, kèm ghi chú tạm thời và dữ liệu cập nhật hơn.
  - L3 – Tool-Driven Agentic RecSys: agent gọi công cụ theo chuỗi ngắn, dùng bộ nhớ nháp (scratchpad) và xử lý lỗi cơ bản.
  - L4 – Single-Agent Planner RecSys: một agent chủ động lập kế hoạch, thực thi và kiểm tra, có bộ nhớ dài hạn có cấu trúc và tự phản hồi (self-reflection).
  - L5 – Multi-Agent Orchestration: nhiều agent chuyên hóa phối hợp với nhau qua bộ nhớ dài hạn dùng chung, bộ kiểm tra, tranh luận hoặc trọng tài.

- L6 – AGI-Level Society: một tập thể agent tự định hướng, có bộ nhớ mang tính thể chế và cơ chế quản trị tự phát sinh; đây chỉ là mức khái niệm.

Trong đó L0, L1 là các hệ thống chưa mang tính agent, còn L6 mới dừng ở mặt lý thuyết, nên đa phần các mô hình và framework hiện có nằm trong khoảng L2 đến L5. Hai trục phân loại này không loại trừ nhau, nên một framework có thể vừa hỗ trợ mô hình gợi ý, vừa tự đưa ra gợi ý.



Hình 2.1. Sơ đồ biểu thị trục mức độ tự chủ từ L0 đến L6

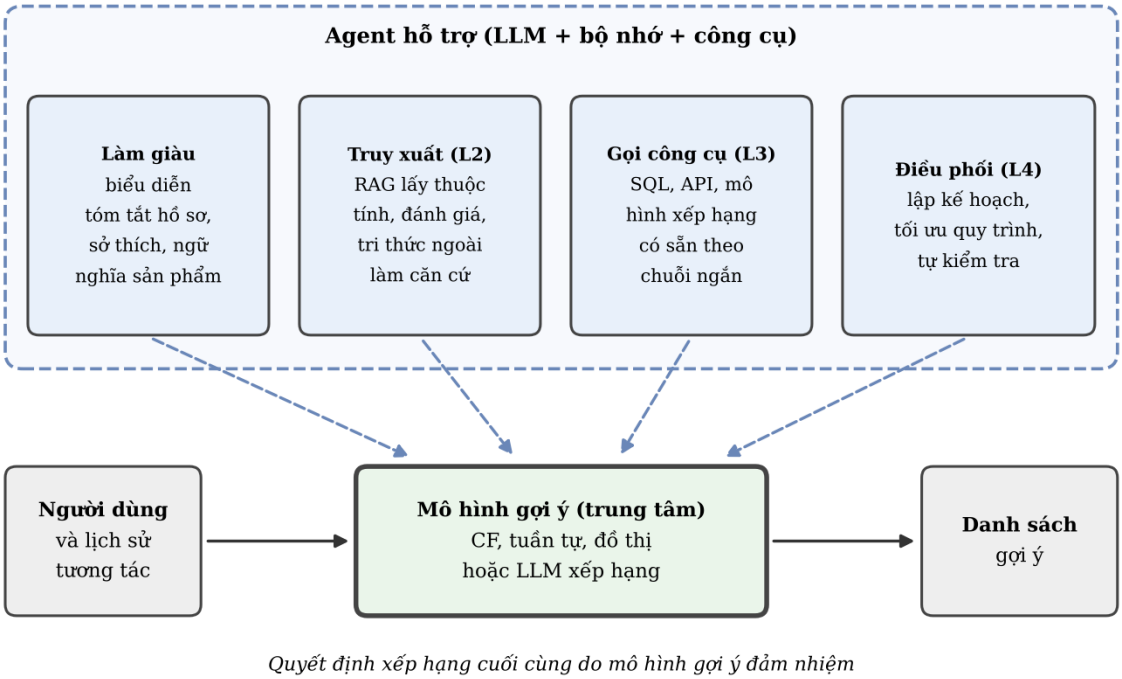
2.1.1.1 Agent-assisted Recommendation (agent hỗ trợ gợi ý).

– Mô hình gợi ý vẫn là thành phần trung tâm: nó nhận dữ liệu người dùng và tạo ra danh sách xếp hạng cuối cùng. Agent không thay thế mô hình này mà cải thiện các khâu xung quanh nó, như minh họa ở Hình 2.1. Tùy mức tự chủ, agent có thể hỗ trợ theo bốn cách:

- Làm giàu biểu diễn: dùng LLM tóm tắt lịch sử tương tác thành hồ sơ người dùng và mô tả ngữ nghĩa của sản phẩm, rồi lưu vào bộ nhớ để mô hình gợi ý sử dụng.
- Truy xuất bổ sung (L2): dùng RAG lấy thuộc tính sản phẩm, đánh giá hoặc tri thức ngoài để LLM trả lời có căn cứ, giảm ảo giác.
- Gọi công cụ (L3): agent gọi API, câu lệnh SQL hoặc một mô hình xếp hạng truyền thống theo chuỗi ngắn, rồi dùng kết quả cho bước tiếp theo.

- Điều phối quy trình (L4): agent lập kế hoạch cho cả quy trình và tự kiểm tra kết quả trung gian.

– Ưu điểm của hướng này là chất lượng xếp hạng vẫn dựa trên mô hình gợi ý đã được kiểm chứng nên dễ kiểm soát. Hạn chế là agent chỉ cải thiện từng khâu, quyết định cuối cùng vẫn nằm ngoài tầm điều khiển của agent.

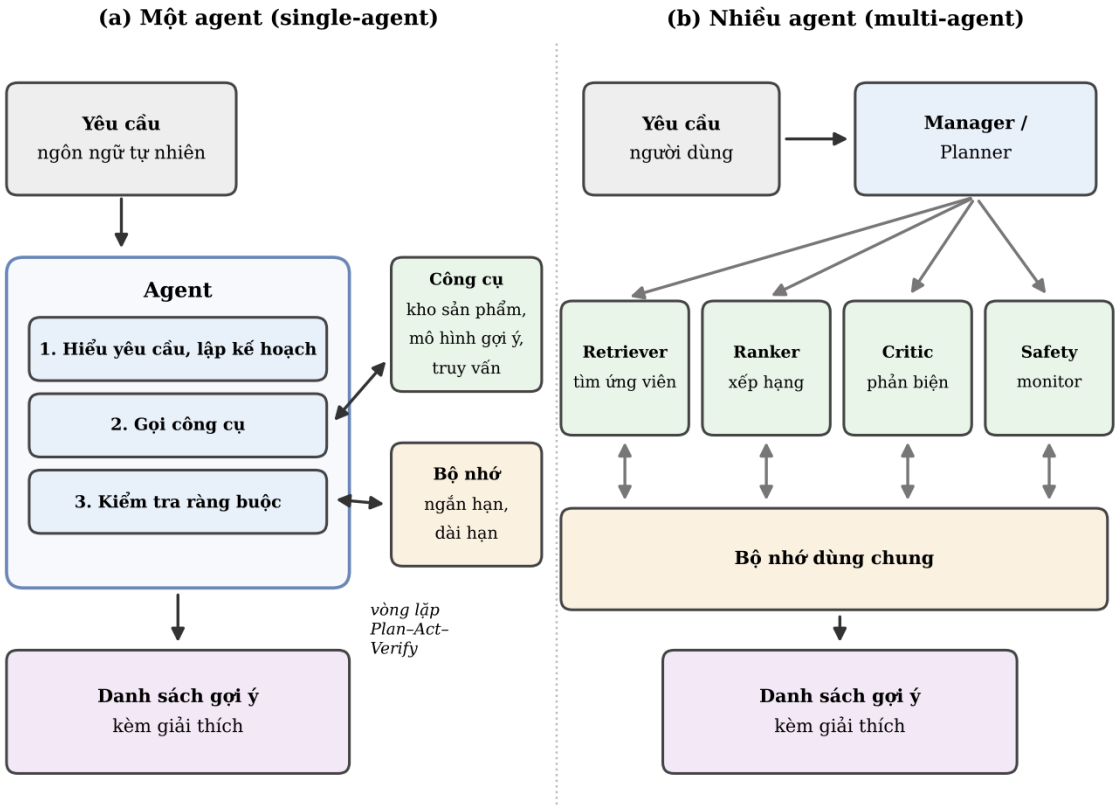


Hình 2.2. Agent-assisted Recommendation: agent hỗ trợ các khâu quanh mô hình gợi ý trung tâm

2.1.1.2 Agent-as-Recommender (agent là bộ gợi ý).

- Agent trực tiếp thực hiện hoặc điều phối toàn bộ quá trình gợi ý: hiểu yêu cầu của người dùng, lập kế hoạch, gọi công cụ để truy xuất thông tin, kiểm tra ràng buộc (ngân sách, danh mục, độ phù hợp), rồi tạo danh sách gợi ý kèm giải thích. Các mô hình gợi ý truyền thống không còn là trung tâm mà trở thành một công cụ trong số các công cụ agent có thể gọi. Hình 2.2 minh họa hai dạng của hướng này:

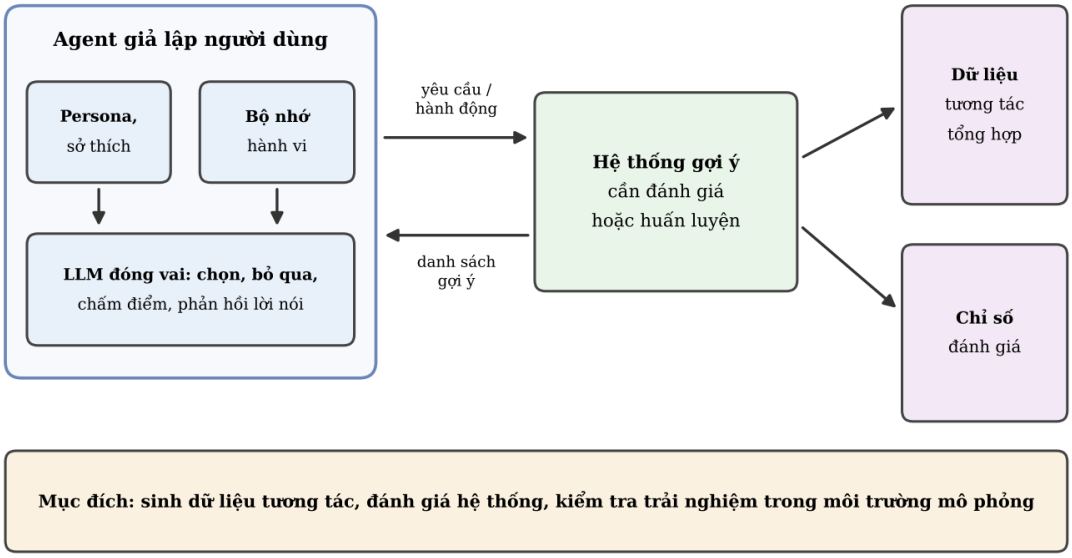
  - Single-agent (Hình 2.2a): một agent duy nhất điều khiển toàn bộ vòng lặp Plan–Act–Verify, có bộ nhớ để duy trì ngữ cảnh và có cơ chế tự phản hồi (reflection) để phát hiện và sửa lỗi trước khi trả kết quả.
  - Multi-agent (Hình 2.2b): nhiều agent chuyên hóa cùng xử lý một yêu cầu, mỗi agent giữ một vai trò, chẳng hạn Manager/Planner điều phối, Retriever tìm ứng viên, Ranker xếp hạng, Critic phản biện và Safety Monitor giám sát an toàn. Các agent trao đổi qua các giao thức như manager–worker, tranh luận – trọng tài (debate–judge) hoặc thương lượng, và dùng chung một bộ nhớ dài hạn.
- Hướng này xử lý được yêu cầu bằng ngôn ngữ tự nhiên, nhiều ràng buộc và tương tác nhiều lượt, đồng thời giải thích được lựa chọn dựa trên bằng chứng truy xuất. Đổi lại, mỗi yêu cầu cần nhiều lượt gọi LLM nên chi phí và độ trễ cao, và kết quả có thể chứa thông tin bịa nếu không có bước kiểm tra. Mức độ phức tạp cũng tăng khi chuyển từ một agent sang nhiều agent do phải điều phối giao tiếp giữa chúng.



Hình 2.3. Agent-as-Recommender: (a) một agent, (b) nhiều agent

2.1.1.3 Agent-as-User-Simulator (agent mô phỏng người dùng).

- Agent không tạo ra gợi ý mà đóng vai người dùng. Mỗi agent được gán persona, sở thích và bộ nhớ hành vi; khi nhận danh sách gợi ý từ một hệ thống, agent dùng LLM để quyết định chọn hay bỏ qua, chấm điểm hoặc phản hồi bằng lời, như Hình 2.3.
- Hướng này phục vụ ba mục đích:
  - Sinh dữ liệu tương tác tổng hợp để huấn luyện.
  - Đánh giá hệ thống (đặc biệt với gợi ý hội thoại nhiều lượt, nơi khó thuê người thật với số lượng lớn).
  - Kiểm tra trải nghiệm người dùng trong môi trường mô phỏng (mô phỏng cả người dùng lẫn sản phẩm như các agent học cộng tác).
- Rủi ro chính là hành vi giả lập có thể không giống hành vi người thật, và mỗi lượt mô phỏng tiêu tốn nhiều token.



Hình 2.4. Agent-as-User-Simulator: agent đóng vai người dùng tương tác với hệ thống gợi ý

Tiêu chí	Agent-assisted	Agent-as-Recommender	Agent-as-User-Simulator
Vai trò của agent	Hỗ trợ một hoặc vài khâu của quy trình gợi ý	Thực hiện hoặc điều phối toàn bộ quá trình gợi ý	Đóng vai người dùng, phản hồi trước gợi ý
Vai trò của mô hình gợi ý	Trung tâm, tạo danh sách cuối cùng	Là một công cụ để agent gọi	Là đối tượng được đánh giá hoặc huấn luyện
Đầu ra chính	Danh sách xếp hạng của mô hình, có hồ sơ và thông tin bổ sung	Danh sách gợi ý kèm giải thích do agent tạo	Dữ liệu tương tác tổng hợp, chỉ số đánh giá

Tiêu chí	Agent-assisted	Agent-as-Recommender	Agent-as-User-Simulator
Mức tự chủ điển hình	L2 đến L4	L4 (single-agent), L5 (multi-agent)	Không phân theo mức gợi ý; tập trung vào độ chân thực của hành vi
Điểm mạnh	Dễ kiểm soát, giữ được chất lượng xếp hạng đã có	Xử lý yêu cầu tự do, ràng buộc, giải thích	Đánh giá không cần người thật, mở rộng được
Hạn chế	Agent không quyết định kết quả cuối	Chi phí LLM, độ trễ, rủi ro bịa thông tin	Hành vi giả lập có thể lệch người thật, tốn token

Bảng 2.1. So sánh ba hướng hệ thống gợi ý dựa trên Agent

2.1.1.4 Hướng phát triển của đề tài

Theo taxonomy này, hệ thống của đề tài thuộc hướng Agent-as-Recommender ở dạng single-agent (mức L4): một LLM lập kế hoạch, thực thi các bước gọi công cụ, tự kiểm tra và sửa kế hoạch (reflection), rồi tạo gợi ý.

Do hệ thống gọi các mô hình gợi ý truyền thống như công cụ nên đề tài cũng có phần giao thoa với Agent-assisted (L3). Đề tài không đi theo hướng Agent-as-User-Simulator; việc đánh giá dựa trên bộ yêu cầu đánh giá được mô tả ở Chương 4.

2.1.2 So sánh hệ thống gợi ý truyền thống và hiện đại

- Dữ liệu đầu vào chính: Hệ thống truyền thống dựa vào lịch sử tương tác người dùng – sản phẩm, trong khi hệ thống hiện đại tận dụng hành vi, nội dung, ngữ cảnh, kiến thức sản phẩm, văn bản tự nhiên và dữ liệu đa nguồn.
- Phương pháp tiêu biểu: Hệ thống truyền thống bao gồm Content-based, Collaborative Filtering, MF, BPR, ItemKNN; hệ thống hiện đại sử dụng Deep

learning, sequential/graph recommendation, LLM-based, RAG, Agentic Recommendation.

- Cách xử lý yêu cầu: Truyền thống dùng pipeline cố định (Retrieval - Ranking), hiện đại hỗ trợ tương tác nhiều lượt, truy xuất bổ sung và điều chỉnh kế hoạch linh hoạt theo ngữ cảnh.
- Khả năng giải thích: Truyền thống dựa trên độ tương đồng hoặc đặc trưng sản phẩm, hiện đại dựa trên bằng chứng truy xuất, thuộc tính sản phẩm và tiêu chí cụ thể của người dùng.

2.2 Bài toán gợi ý Top-K và Quy trình xử lý

2.2.1 Khái niệm bài toán Top-K

Bài toán gợi ý Top-K hướng đến việc lựa chọn K sản phẩm phù hợp nhất cho người dùng u từ tập ứng viênI_u. Mục tiêu cốt lõi là đưa các sản phẩm liên quan lên các vị trí cao trong danh sách kết quả thay vì ước lượng chính xác giá trị rating tuyệt đối.

$$L_u = *Top - K_{i \in I_u} f(u, i; \theta)$$

Tập ứng viên I_u được xác định bằng cách loại bỏ các sản phẩm hết hàng hoặc không khả dụng, sản phẩm vi phạm ràng buộc bắt buộc (như vượt ngân sách, sai danh mục) và sản phẩm người dùng đã tương tác trước đó (tùy chiến lược).

2.2.2 Quy trình của một hệ thống gợi ý

Trong hệ thống quy mô lớn, quy trình gợi ý thường được tổ chức thành 2 giai đoạn:

- Giai đoạn Candidate Generation / Retrieval (Sinh ứng viên): Sinh một tập ứng viên nhỏ C_u từ toàn bộ danh mục I. Giai đoạn này tập trung vào tốc độ và độ bao phủ (Recall), giúp lọc bỏ hàng triệu sản phẩm không liên quan để tạo ra một danh sách ngắn gọn nhưng đảm bảo các sản phẩm mục tiêu tiềm năng không bị bỏ sót. Hiệu quả của giai đoạn này được đo lường bằng Candidate Recall.

– Giai đoạn Ranking (Xếp hạng): Đánh giá chi tiết các ứng viên trong C_u và sắp xếp thành danh sách Top-K (L_u). Giai đoạn này sử dụng các mô hình học máy phức tạp với nhiều đặc trưng (features) chi tiết hơn để tính điểm phù hợp chính xác cho từng ứng viên, ưu tiên đưa những sản phẩm tốt nhất lên vị trí cao nhất. Chất lượng xếp hạng được đo lường bằng Precision@K, Recall@K, NDCG@K.

2.3 Các thách thức chính trong hệ thống gợi ý

2.3.1 Dữ liệu thưa (Data Sparsity)

Hiện tượng số lượng tương tác quan sát được nhỏ hơn rất nhiều so với tổng số cặp người dùng – sản phẩm có thể có, tức $|\Omega| \ll |U| \times |I|$, trong đó Ω là tập các cặp (u, i) đã có tương tác, U là tập người dùng và I là tập sản phẩm.

Công thức tính độ thưa:

$$\text{Sparsity}(R) = 1 - \frac{|\Omega|}{|U| \times |I|}$$

Hậu quả: Khó tính toán độ tương đồng tin cậy, vector tiềm ẩn không ổn định trong Matrix Factorization, và sản phẩm ít phổ biến dễ bị bỏ quên. Giải pháp giảm thiểu bao gồm khai thác thuộc tính sản phẩm, sử dụng Knowledge Graph, ứng dụng LLM và RAG để bổ sung ngữ cảnh.

2.3.2 Vấn đề khởi động lạnh (Cold-start)

Xảy ra khi hệ thống thiếu thông tin ban đầu đối với đối tượng mới xuất hiện:

– User cold-start:
  - Là hiện tượng xảy ra đối với người dùng mới chưa có lịch sử tương tác
  - Giải pháp: Khai thác khảo sát sở thích, thông tin thiết bị, hoặc áp dụng thuật toán Bandit.

– Item cold-start:
  - Là hiện tượng xảy ra đối với sản phẩm mới chưa có lượt tương tác.
  - Giải pháp: Khai thác mô tả, thuộc tính kỹ thuật, hình ảnh, embedding nội dung, Knowledge Graph và trích xuất đặc trưng bằng LLM.

- System cold-start:
    - Là hiện tượng xảy ra khi hệ thống mới triển khai.
    - Giải pháp: Dựa vào dữ liệu danh mục, xu hướng thị trường và cơ chế thu thập phản hồi nhanh.

2.3.3 Các thách thức trong hệ thống gợi ý dựa trên LLM (Agentic RS)

Bên cạnh các thách thức truyền thống, hệ thống gợi ý ứng dụng LLM đối mặt với những vấn đề đặc thù mới:

- Ảo giác (Hallucination): LLM có thể tự bịa ra thông số sản phẩm, tạo ra các mã định danh (Invented IDs) không tồn tại, hoặc khẳng định sai lệch rằng một sản phẩm đáp ứng hoàn toàn ngân sách của người dùng dù thực tế không phải vậy.
- Độ trễ và rào cản chi phí: Việc thực thi quy trình lập kế hoạch (Planning) và gọi nhiều công cụ liên tiếp đòi hỏi LLM phải tạo ra lượng token lớn qua nhiều vòng lặp, làm tăng đáng kể độ trễ phản hồi (End-to-end Latency) và chi phí vận hành API.
- Lỗi phân rã nhiệm vụ và phối hợp công cụ: Tác tử có thể vướng vào các lỗi như tạo ra kế hoạch JSON không hợp lệ, truyền sai tham số schema cho công cụ, hoặc không thể phục hồi linh hoạt khi một công cụ trả về kết quả rỗng.

2.4 Các phương pháp gợi ý chi tiết

2.4.1 Popularity-based Recommendation

Popularity-based là phương pháp gợi ý dựa trên mức độ phổ biến của sản phẩm trong tập dữ liệu. Thay vì học mô hình sở thích riêng cho từng người dùng, phương pháp này tính một điểm phổ biến cho mỗi sản phẩm và ưu tiên những sản phẩm có điểm cao.

Gọi c_i là số lượt tương tác liên quan đến sản phẩm i. Điểm phổ biến cơ bản có thể được xác định bằng:

$$s_i = c_i$$

Tùy theo dữ liệu và mục tiêu nghiên cứu, c_i có thể đại diện cho số lượt mua, lượt nhấp, lượt đánh giá hoặc tổng số tương tác. Ưu điểm của phương pháp này là cách triển khai đơn giản, chi phí tính toán thấp và không yêu cầu lịch sử cá nhân hóa đầy đủ cho từng người dùng. Tuy nhiên, phương pháp không trực tiếp mô hình hóa sự khác biệt về sở thích giữa những người dùng và có xu hướng ưu tiên những sản phẩm đã phổ biến, khiến các sản phẩm ít tương tác khó được đề xuất.

2.4.2 Collaborative Filtering (CF)

Collaborative Filtering (CF) là nhóm phương pháp gợi ý dựa trên các mẫu tương tác giữa người dùng và sản phẩm. Ý tưởng chính là những người dùng có hành vi tương tự có thể có sở thích tương đồng; tương tự, các sản phẩm thường được cùng một nhóm người dùng quan tâm có thể có quan hệ với nhau. Đối với phương pháp dựa trên độ tương đồng, điểm gợi ý có thể được tính từ các sản phẩm mà người dùng đã tương tác:

$$s_{u,i} = \sum_{j \in H_u} w_{u,j} \, \mathrm{sim}(i,j)$$

Trong đó, $H_u$là tập sản phẩm trong lịch sử của người dùng u, $\mathrm{sim}(i,j)$ biểu diễn độ tương đồng giữa hai sản phẩm, còn $w_{u,j}$ là trọng số của sản phẩm j trong lịch sử.

2.4.3 ItemKNN

Item-based Collaborative Filtering tập trung vào việc xác định mức độ tương đồng giữa các sản phẩm dựa trên hành vi tương tác của người dùng. Một cách tính độ tương đồng phổ biến là Cosine Similarity:

$$\mathrm{sim}(i,j) = \frac{x_i^{\top} x_j}{|x_i|2|x_j|2}$$

Giá trị tương đồng lớn cho thấy hai vector có hướng gần nhau, tức là các sản phẩm có mẫu tương tác tương tự. ItemKNN là một biến thể dựa trên K láng giềng

gần nhất (K-Nearest Neighbors), lựa chọn tối đa K sản phẩm có độ tương đồng cao nhất. Điểm tổng hợp khái quát được tính như sau:

$$su, i = \sum j \in H_u \cap N_K(i)\, \mathrm{sim}(i, j) w_{u,j}$$

Trong đó, $N_K(i)$ là tập láng giềng của sản phẩm i. Sự tương đồng trong hành vi không đồng nghĩa với sự tương đồng về thuộc tính hoặc tính tương thích thực tế, vì vậy kết quả ItemKNN nên được xem là tín hiệu gợi ý thay vì bằng chứng đầy đủ.

2.4.4 Bayesian Personalized Ranking (BPR-MF)

- Bayesian Personalized Ranking (BPR) là phương pháp học xếp hạng được thiết kế cho bài toán gợi ý với dữ liệu phản hồi ngầm định. BPR học cách xếp hạng một sản phẩm người dùng đã tương tác cao hơn một sản phẩm chưa được quan sát trong điều kiện huấn luyện.
- BPR xây dựng hàm mục tiêu dựa trên giả định $y_{ui} > y_{uj}$, tối ưu xác suất để điểm của sản phẩm i lớn hơn điểm của sản phẩm j. Hàm mục tiêu BPR có thể viết dưới dạng:

  $$\mathcal{L} * BPR = -\sum * (u, i, j) \in D_S \ln \sigma \left(y_{ui} - y_{uj}\right) + \lambda |\Theta| 2^2$$

  Trong đó:
  - $\sigma(x) = \frac{1}{1+e^{-x}}$ là hàm sigmoid
  - Θ là tập tham số mô hình
  - λ là hệ số điều chuẩn.
- Matrix Factorization (MF) biểu diễn người dùng và sản phẩm trong không gian vector tiềm ẩn, khi đó điểm phù hợp có thể được xác định bằng tích vô hướng:

  $$yui = p_u^{\top} q_i$$
- Mô hình học các vector này để ưu tiên những cặp người dùng–sản phẩm có tương tác so với các sản phẩm được lấy mẫu âm. BPR-MF có ưu điểm là tối ưu trực tiếp mục tiêu xếp hạng, song mô hình không tự động giải quyết được

việc thiếu dữ liệu hoặc các ràng buộc ngữ nghĩa chưa được biểu diễn trong lịch sử tương tác.

2.5 Truy xuất thông tin và Kho tri thức sản phẩm (Information Retrieval & Knowledge Base)

- Truy xuất thông tin (Information Retrieval – IR) là lĩnh vực nghiên cứu các phương pháp tìm kiếm và xếp hạng tài liệu hoặc đối tượng dựa trên mức độ liên quan với truy vấn của người dùng. Quá trình truy xuất bằng hàm có thể mô tả:

$$s(q, d) = \text{Rel}(q, d)$$

  Trong đó:
  - q là truy vấn
  - d là tài liệu hoặc bản ghi cần đánh giá
  - $s(q, d)$ là điểm liên quan giữa chúng.
- Các phương pháp truy xuất phổ biến bao gồm truy xuất từ khóa, truy xuất ngữ nghĩa, và truy xuất kết hợp.

2.5.1 Thuật toán tìm kiếm BM25

- BM25 là một thuật toán xếp hạng tài liệu dựa trên mô hình truy xuất thông tin xác suất, đánh giá mức độ liên quan giữa truy vấn và tài liệu dựa trên sự xuất hiện của các thuật ngữ, độ hiếm của thuật ngữ và độ dài tài liệu.

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}_t \cdot \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

  Trong đó
  - q là truy vấn
  - d là tài liệu
  - t là thuật ngữ
  - $f_{t,d}$ là tần suất xuất hiện

- $|d|$ là độ dài tài liệu
- avgdl là độ dài trung bình
- k_1 là điều chỉnh bão hòa tần suất
- b là chuẩn hóa theo độ dài.

– Một dạng tính IDF thường dùng là:

$$\mathrm{IDF}_t = \ln\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$$

– BM25 không đòi hỏi huấn luyện mô hình embedding chuyên biệt và phù hợp với những truy vấn có thuật ngữ phân biệt rõ ràng. Tuy nhiên, BM25 chủ yếu dựa trên tín hiệu từ vựng, có thể không nhận diện đầy đủ mức độ tương đồng ngữ nghĩa khi truy vấn và mô tả sản phẩm dùng từ khác nhau.

2.5.2 Kho tri thức sản phẩm (Product Knowledge Base)

– Kho tri thức sản phẩm là tập hợp dữ liệu về các sản phẩm được tổ chức có cấu trúc nhằm hỗ trợ tìm kiếm, truy vấn và cung cấp thông tin cho hệ thống gợi ý. Mỗi sản phẩm được biểu diễn bằng một bản ghi gồm mã định danh và tập các thuộc tính, bao gồm thuộc tính có cấu trúc (giá, thương hiệu, danh mục, điểm đánh giá trung bình) và thuộc tính văn bản (tên, đặc điểm nổi bật, mô tả).

– Việc tổ chức kho tri thức cần đáp ứng các yêu cầu cơ bản sau:

- Tính đầy đủ: bản ghi chứa các thuộc tính cần thiết để đánh giá các ràng buộc có thể xuất hiện trong yêu cầu của người dùng.
- Tính nhất quán: cùng một thuộc tính được biểu diễn theo cùng kiểu dữ liệu và đơn vị trên toàn bộ kho.
- Khả năng truy xuất: hỗ trợ lọc theo điều kiện trên thuộc tính có cấu trúc và tìm kiếm theo nội dung trên thuộc tính văn bản, thường thông qua chỉ mục.
- Khả năng cập nhật: thêm, sửa hoặc xóa bản ghi mà không phải xây dựng lại toàn bộ kho.

  - Khả năng truy nguyên: mỗi thông tin được sử dụng đều liên kết được về bản ghi gốc thông qua mã định danh sản phẩm.
- Về mặt lưu trữ, dữ liệu có thể được tổ chức dưới dạng bảng quan hệ, tài liệu bán cấu trúc hoặc các đơn vị văn bản được lập chỉ mục. Bảng quan hệ phù hợp với việc lọc theo điều kiện chính xác trên thuộc tính có cấu trúc, còn chỉ mục văn bản phù hợp với việc xếp hạng theo mức độ liên quan.
- Trong hệ thống gợi ý có sử dụng LLM, quá trình lựa chọn thông tin từ kho tri thức gồm hai cơ chế bổ sung cho nhau:
  - Truy vấn theo điều kiện cấu trúc: chọn ra tập bản ghi thỏa mãn chính xác các điều kiện xác định, như giá không vượt quá một ngưỡng hoặc thuộc một thương hiệu.
  - Truy xuất theo mức độ liên quan: tính điểm giữa yêu cầu và nội dung văn bản của từng bản ghi, rồi xếp hạng theo điểm số.
- Hai cơ chế này giải quyết hai loại điều kiện khác nhau: truy vấn cấu trúc đảm bảo các ràng buộc bắt buộc, còn truy xuất theo liên quan đo mức độ phù hợp về nội dung. Một bản ghi có điểm liên quan cao chưa chắc thỏa mãn mọi ràng buộc bắt buộc (ví dụ một sản phẩm có mô tả rất khớp nhưng vượt ngân sách), do đó hệ thống cần kết hợp cả hai, đồng thời kết hợp dữ liệu truy xuất với kết quả của mô hình gợi ý. Bên cạnh đó, hệ thống cần giữ được liên kết giữa sản phẩm được chọn và thông tin căn cứ, để mọi nội dung trong câu trả lời đều truy nguyên được về bản ghi nguồn.

2.6 Mô hình ngôn ngữ lớn, Tác tử thông minh và RAG

2.6.1 Mô hình ngôn ngữ lớn (LLM) và Structured Output

Mô hình ngôn ngữ lớn (LLMs) học các mẫu biểu diễn và sinh ngôn ngữ; trong quá trình sinh văn bản tự hồi quy, mô hình ước lượng xác suất của token tiếp theo dựa trên chuỗi token đã có:

$$P(x_1, \dots, x_T) = \prod_{t=1}^{T} P\left(x_t \mid x_{<t}\right)$$

Structured Output là cách yêu cầu LLM tạo kết quả theo một cấu trúc dữ liệu xác định (chẳng hạn JSON), trong đó mô hình phải cung cấp các trường dữ liệu phù hợp với yêu cầu của hệ thống thay vì sinh văn bản tự do. Structured Output giúp giảm sự mơ hồ khi trao đổi dữ liệu, hỗ trợ xác thực kiểu dữ liệu, và tạo điều kiện ghi log nhất quán.

2.6.2 LLM Agent và các cơ chế điều phối hành động

2.6.2.1 Khái niệm LLM Agent

- LLM Agent là mô hình kiến trúc lấy LLM làm trung tâm điều phối (Central Controller). Thay vì chỉ sinh văn bản thụ động, Agent có khả năng tiếp nhận mục tiêu từ người dùng, diễn giải trạng thái môi trường, lập kế hoạch, lựa chọn hành động và tương tác với các công cụ bên ngoài để hoàn thành các tác vụ phức tạp.
- Quá trình vận hành khái quát của một Agent qua các mốc thời gian t được biểu diễn bằng chuỗi chuyển trạng thái:

  $$o_t \rightarrow p_t \rightarrow a_t \rightarrow o_{t+1}$$

  Trong đó:
  - o_t: Quan sát (Observation) hoặc thông tin nhận được tại thời điểm t.
  - p_t: Kế hoạch (Plan) hoặc bước suy luận được định hình.
  - a_t: Hành động (Action) được thực thi (ví dụ: gọi một API/Tool cụ thể).
  - o_{t+1}: Kết quả phản hồi hoặc quan sát mới thu được từ môi trường sau khi thực hiện hành độnga_t.

2.6.2.2 Các chiến lược điều phối và lập kế hoạch (Planning Strategies)

Trong thực tế phát triển các hệ thống Agent, có nhiều chiến lược lập kế hoạch và điều phối luồng thực thi khác nhau tùy thuộc vào bài toán:

- Vòng lặp tương tác từng bước (Step-by-step Interleaved Planning - ReAct):
  - Cơ chế: Agent thực hiện luồng suy luận luân phiên (Thought - Action - Observation). Tại mỗi bước, LLM gọi một công cụ, chờ nhận kết quả

phản hồi o_{t+1}, sau đó mới tiếp tục suy luận để quyết định hành động kế tiếp.

- Đặc điểm: Có tính linh hoạt cao trong môi trường động, tuy nhiên nhược điểm là tốn nhiều vòng gọi LLM (API calls), làm tăng đáng kể độ trễ và chi phí vận hành token.

- Lập kế hoạch trước khi thực thi (Plan-then-Execute / Plan-first):
  - Cơ chế: Thay vì tương tác và suy luận nhỏ lẻ từng bước, Agent phân tích toàn bộ yêu cầu của người dùng ngay từ đầu và sinh ra một bản kế hoạch tổng thể hoàn chỉnh (thường ở dạng cấu trúc dữ liệu JSON) chứa danh sách toàn bộ các công cụ cần gọi, thứ tự thực thi và quy tắc truyền dữ liệu. Trình điều phối sau đó sẽ tự động thực thi chuỗi công cụ này theo đúng kế hoạch.
  - Đặc điểm: Đây là phương pháp tối ưu cho các hệ thống ứng dụng thực tế nhờ khả năng kiểm soát chặt chẽ luồng thực thi, giảm tối đa số lượt gọi LLM, tiết kiệm chi phí và hạ độ trễ phản hồi. Dữ liệu đầu ra của bước trước có thể được tham chiếu trực tiếp sang tham số của bước sau (ví dụ sử dụng cú pháp biến tham chiếu `$N.path`).
- Quy trình cố định kết hợp Tác tử (Fixed Workflow / Hybrid Planning):
  - Cơ chế: Kế hoạch thực thi được định nghĩa cứng sẵn dưới dạng một Pipeline cố định. LLM chỉ đóng vai trò hỗ trợ ở một số công đoạn nhất định (như trích xuất tham số hoặc diễn đạt câu trả lời), giúp hệ thống hoạt động vô cùng ổn định và đạt tốc độ tối đa.

2.6.2.3 Các thành phần cốt lõi của một LLM Agent

Dù sử dụng chiến lược điều phối nào, một hệ thống Agentic tổng quát thường được cấu thành từ 4 thành phần nền tảng:

- Tool Calling (Thao tác công cụ): Cơ chế cho phép LLM đề xuất và truyền tham số vào các hàm/API bên ngoài (truy vấn CSDL, tìm kiếm FTS, gọi mô

hình gợi ý). Hệ thống thực thi sẽ kiểm tra tính hợp lệ của tham số (schema validation) trước khi chạy để đảm bảo an toàn.

- Planning (Lập kế hoạch): Khả năng phân rã mục tiêu lớn của người dùng thành các tác vụ nhỏ có thể thực thi. Hệ thống có thể áp dụng chiến lược Plan-first để chốt sơ đồ thực thi ngay từ đầu hoặc Dynamic Planning để điều chỉnh linh hoạt.
- Reflection (Tự phản tư & Tự sửa lỗi): Đóng vai trò là cơ chế phục hồi khi kế hoạch ban đầu gặp sự cố. Ví dụ trong cơ chế Plan-reflect, nếu bản kế hoạch đầu tiên chạy bị lỗi tham số hoặc truy xuất ra kết quả rỗng, Agent sẽ nhận báo cáo lỗi, xem xét lại nguyên nhân và sinh ra một bản kế hoạch điều chỉnh mới mà không làm gián đoạn toàn bộ quy trình.
- Memory (Bộ nhớ): Thành phần lưu giữ bối cảnh dài hạn (như hồ sơ sở thích cá nhân, lịch sử mua sắm) hoặc ngắn hạn (các biến trung gian trong phiên làm việc) giúp Agent đưa ra các quyết định cá nhân hóa chính xác.

2.6.3 Grounded Response Generation & Giảm thiểu ảo giác

Grounding là quá trình gắn nội dung sinh ra với thông tin hoặc trạng thái có thể kiểm tra. Một pipeline RAG tổng quát có thể biểu diễn:

$$q \rightarrow \text{Retrieve}(q) \rightarrow C_q \rightarrow \text{Generate}(q, C_q)$$

Quá trình này bao gồm truy xuất tài liệu liên quan (C_q), tổ chức ngữ cảnh và sinh câu trả lời. Đối với hệ thống gợi ý, bằng chứng có thể là thuộc tính sản phẩm hoặc giá trị ghi nhận. Hallucination (ảo giác) là hiện tượng mô hình tạo nội dung không chính xác hoặc thiếu bằng chứng. Các biện pháp giảm thiểu bao gồm truy xuất từ nguồn xác định, giữ liên kết bản ghi nguồn, kiểm tra ràng buộc bằng quy tắc xác minh và đánh giá tính nhất quán giữa nội dung trả lời và dữ liệu truy xuất.

2.7 Phương pháp đánh giá hệ thống

2.7.1 Các chỉ số đánh giá gợi ý

2.7.1.1 Recall@K

Recall@K đo lường mức độ bao phủ, tức là tỷ lệ các sản phẩm mục tiêu liên quan được hệ thống tìm thấy và đưa vào K vị trí đầu của danh sách đề xuất. Gọi R_u là tập sản phẩm liên quan và L_u^K là danh sách đề xuất:

$$\text{Recall@K}_u = \frac{|L_u^K \cap R_u|}{|R_u|}$$

2.7.1.2 Precision@K

Precision@K đo lường mức độ tập trung, tức là tỷ lệ các kết quả nằm trong K vị trí đầu thực sự liên quan đến nhu cầu của người dùng:

$$\text{Precision@K}_u = \frac{|L_u^K \cap R_u|}{|L_u^K|}$$

2.7.1.3 Normalized Discounted Cumulative Gain (NDCG@K)

NDCG@K đánh giá chất lượng xếp hạng có xét đến vị trí kết quả, với mức phạt nặng hơn nếu sản phẩm liên quan bị đẩy xuống vị trí thấp. Discounted Cumulative Gain tại vị trí K được tính bằng:

$$\text{DCG@K} = \sum_{i=1}^{K} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}$$

Và NDCG@K được chuẩn hóa theo danh sách lý tưởng (IDCG@K):

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

2.7.2 Đánh giá Agent và Hiệu năng vận hành

Việc đánh giá Agentic RS không thể chỉ gói gọn trong các metric gợi ý truyền thống mà đòi hỏi các chỉ số phản ánh quá trình thực thi đa bước:

- Chỉ số hoàn thành (Task Success Rate): Được định nghĩa là tỷ lệ yêu cầu thỏa mãn hoàn toàn các tiêu chí thành công đã được định trước, phân biệt

rạch ròi với chất lượng xếp hạng (vì một Agent có thể chạy xong tác vụ nhưng Recall vẫn bằng 0).

$$\text{SuccessRate} = \frac{N_{\text{success}}}{N_{\text{total}}}$$

- Chất lượng thực thi công cụ (Tool-use & Plan Validity):
  - Tool-use correctness (khả năng cung cấp tham số đúng schema)
  - Plan validity (tỷ lệ kế hoạch JSON hợp lệ)
  - Constraint satisfaction (mức độ đáp ứng ràng buộc cứng)
  - Consistency (tính nhất quán giữa các bước).
- Hiệu suất và độ trễ (Execution Latency): Đánh giá số bước thực thi, số vòng reflection và đo lường độ trễ đầu cuối:

$$T_{e2e} = t_{\text{finish}} - t_{\text{start}}$$

- Chi phí vận hành (LLM Cost): Chi phí sử dụng LLM được tính toán dựa trên khối lượng token tiêu thụ tại các bước lập kế hoạch và sinh câu trả lời:

$$C_{LLM} = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}}$$

Việc đánh giá toàn diện đòi hỏi phải đo lường sự đánh đổi (trade-off) giữa chất lượng đầu ra, tỷ lệ thành công của tác vụ, độ tin cậy và nguồn tài nguyên/token sử dụng.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT VÀ THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

3.1.1 Nguyên tắc thiết kế kiến trúc

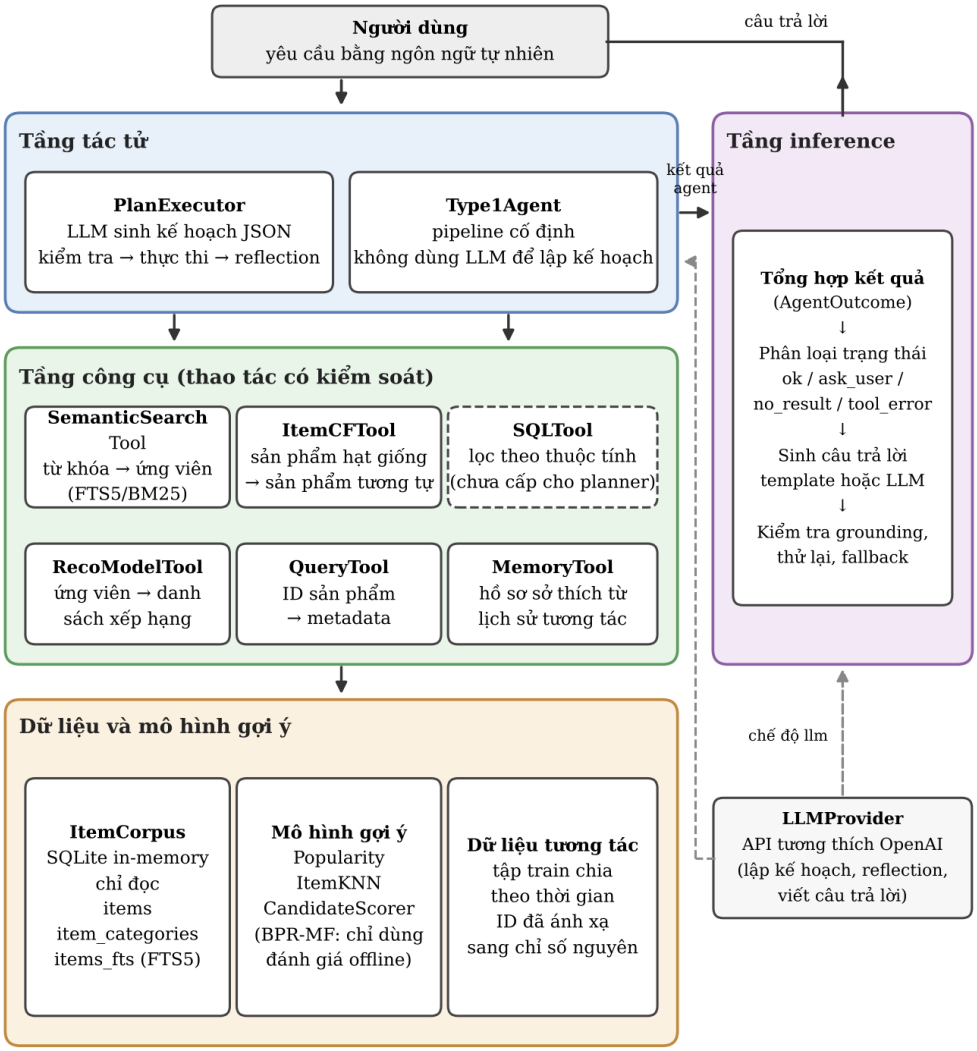
Kiến trúc dựa trên sáu nguyên tắc, mỗi nguyên tắc đều xuất phát từ một rủi ro cụ thể của hệ gợi ý dùng LLM:

- LLM quyết định "làm gì", công cụ quyết định "làm thế nào". LLM chỉ thấy tên, mô tả và lược đồ vào – ra của từng công cụ, không thấy mã nguồn hay dữ liệu thô. Cách phân vai này giới hạn không gian hành động của LLM vào những thao tác đã được kiểm tra.
- Tách ba chức năng truy xuất, xếp hạng và tra cứu metadata. Truy xuất trả lời "sản phẩm nào có thể liên quan", xếp hạng trả lời "sản phẩm nào đứng trên", tra cứu trả lời "thông tin của các ID đã biết là gì". Khi mỗi công cụ chỉ làm một việc, nguồn gốc của thứ tự sản phẩm luôn truy vết được và lỗi dễ khoanh vùng.
- Mọi thông tin sản phẩm đều đến từ dữ liệu. Tên, giá, danh mục lấy từ kho sản phẩm; thứ tự lấy từ mô hình gợi ý; LLM không được bổ sung sản phẩm hay thuộc tính.
- Tham số nằm trong cấu hình, hợp đồng nằm trong mã. Các giá trị cần chỉnh (ngưỡng lọc, số bước tối đa, số vòng reflection…) được đặt trong tệp cấu hình; tên cột, khóa bộ lọc và lược đồ công cụ là hợp đồng nên được cố định trong mã.
- Phụ thuộc một chiều giữa các tầng. Tầng thấp (công cụ) không biết đến tầng cao (tác tử, inference), nhờ đó có thể thay tác tử hoặc cách diễn đạt mà không động đến công cụ.

– Lỗi được ghi nhận thay vì che giấu. Công cụ trả về trạng thái thành công/thất bại, các trường hợp đặc biệt (người dùng chưa có lịch sử, ứng viên không có điểm) được báo cáo rõ để phân tích về sau.

3.1.2 Các tầng chức năng

Hình 3.1 trình bày kiến trúc tổng thể và Bảng 3.2 tóm tắt nhiệm vụ, cách thực hiện và lý do của từng tầng.



Hình 3.1. Kiến trúc tổng thể của hệ thống

Tầng	Nhiệm vụ	Cách thực hiện	Lý do
Dữ liệu	Biến review thô thành ba tập train/validation/test theo thời gian, đã ánh xạ ID	Pipeline ba giai đoạn, lưu Parquet, kiểm tra rò rỉ	Cung cấp dữ liệu nhất quán cho mô hình và đánh giá
Kho sản phẩm	Lưu metadata và hỗ trợ tìm kiếm văn bản, lọc thuộc tính	ItemCorpus: SQLite in-memory, chỉ đọc, chỉ mục FTS5	Nguồn thông tin kiểm chứng được, không phụ thuộc LLM
Mô hình gợi ý	Tạo tín hiệu xếp hạng từ hành vi	Popularity, ItemKNN (cho agent), BPR-MF (đánh giá offline)	Tách thứ tự gợi ý khỏi LLM
Công cụ	Chuẩn hóa thao tác truy xuất, xếp hạng, tra cứu, hồ sơ người dùng	Lớp cơ sở Tool với lược đồ, kiểm tra tham số, ghi log	Không gian hành động có kiểm soát cho tác tử
Tác tử	Điều phối việc gọi công cụ	Type1Agent (cố định), PlanExecutor (kế hoạch do LLM sinh)	So sánh điều phối cố định và linh hoạt
Inference	Biến kết quả tác tử thành câu trả lời	Template hoặc LLM, kiểm tra grounding, fallback	Tách chất lượng diễn đạt khỏi chất lượng xếp hạng

Tầng	Nhiệm vụ	Cách thực hiện	Lý do
Đánh giá và demo	Đo chất lượng, so sánh cấu hình, minh họa	Bộ đánh giá offline, đánh giá tác tử, ablation, giao diện Gradio	Kiểm chứng theo từng tầng

Bảng 3.1. Nhiệm vụ, cách thực hiện và lý do của từng tầng

3.1.3 Quy tắc phụ thuộc giữa các tầng

Các gói mã nguồn được tổ chức theo một chiều phụ thuộc cố định: tầng công cụ chỉ dựa vào dữ liệu và cấu hình; tầng tác tử dựa vào công cụ, bộ nhớ và lớp truy cập LLM; tầng inference dựa vào tác tử và lớp truy cập LLM; tầng đánh giá và tầng demo được phép dùng các tầng bên dưới nhưng không được phụ thuộc lẫn nhau. Quy tắc này được kiểm tra tự động trong bộ kiểm thử (mục 3.8).

Lý do của quy tắc là tránh vòng phụ thuộc và giữ cho từng tầng thay thế được độc lập. Ví dụ, vì tầng tác tử không được phụ thuộc vào đánh giá, chức năng trích xuất "người dùng thực sự nhìn thấy sản phẩm nào" được đặt ở tầng tác tử để cả inference, đánh giá và demo cùng dùng một định nghĩa duy nhất, tránh việc mỗi nơi tính một kiểu.

3.1.4 Hai luồng xử lý chính

- Luồng offline (không dùng LLM):
    - Review thô được xử lý thành dữ liệu đã chia tập; các mô hình gợi ý được khởi tạo từ tập train
    - Mô hình được đánh giá trên validation hoặc test.

      Luồng này đo chất lượng của mô hình gợi ý trong điều kiện xác định, không bao gồm chi phí lập kế hoạch bằng LLM.
- Luồng tương tác qua tác tử (có LLM).
    - Yêu cầu của người dùng được chuyển tới một trong hai tác tử.
    - Với Type1Agent, hệ thống chạy chuỗi công cụ cố định.

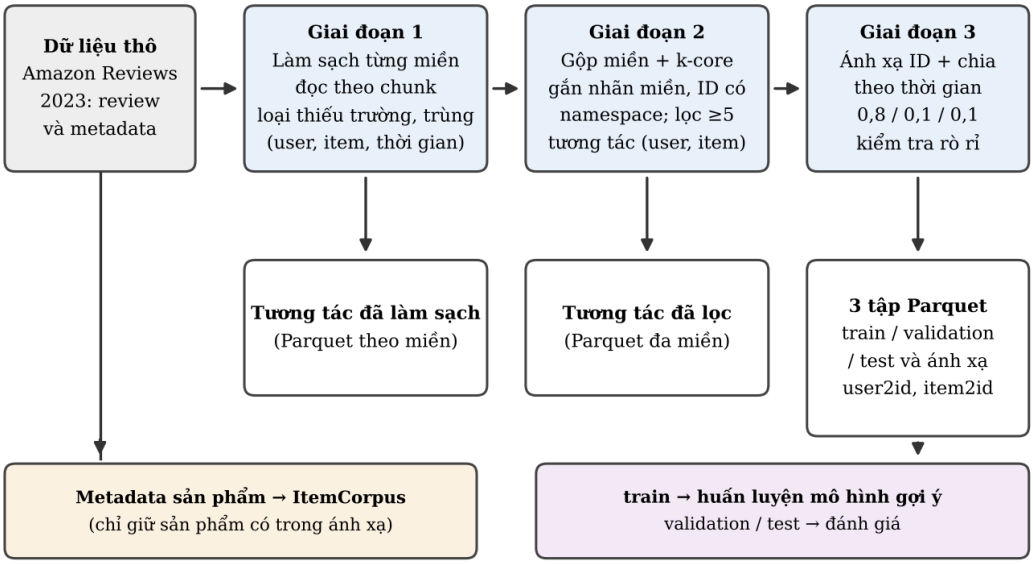
- Với PlanExecutor, LLM sinh kế hoạch có cấu trúc; hệ thống kiểm tra kế hoạch, thực thi từng bước, giải quyết tham chiếu giữa các bước và tổng hợp kết quả.
- Nếu có lỗi hoặc không có sản phẩm nào và cơ chế reflection được bật, kế hoạch được sửa và chạy lại trong giới hạn cho phép.
- Kết quả của hai tác tử cùng được đưa về một dạng chung (AgentOutcome).
- Sau đó tầng inference tạo câu trả lời từ đúng các sản phẩm đã được chọn.

– Reflection chỉ chạy khi cấu hình và điều kiện cho phép, không phải mọi yêu cầu đều kích hoạt. Tương tự, tầng inference chỉ diễn đạt lại kết quả đã có nên việc bổ sung inference không làm thay đổi chất lượng xếp hạng.

3.2 Dữ liệu và kho tri thức sản phẩm

3.2.1 Pipeline xử lý dữ liệu tương tác

Dữ liệu tương tác là nguồn tín hiệu chính cho các mô hình gợi ý cộng tác. Pipeline gồm ba giai đoạn độc lập, mỗi giai đoạn lưu kết quả trung gian dưới dạng Parquet để có thể chạy lại riêng từng bước (Hình 3.2, Bảng 3.3).



Hình 3.2. Pipeline xử lý dữ liệu ba giai đoạn

Giai đoạn	Công việc thực hiện trong giai đoạn	Ý nghĩa
1. Làm sạch từng miền	Đọc review theo từng khối (chunk) để không nạp toàn bộ tệp vào bộ nhớ Loại bản ghi thiếu trường bắt buộc (người dùng, sản phẩm, điểm đánh giá, thời gian) Loại bản ghi trùng theo bộ ba (người dùng, sản phẩm, thời gian), dùng bảng SQLite tạm để phát hiện trùng xuyên các khối	Tệp review lớn; trùng lặp làm lệch tín hiệu và có thể tạo cặp (người dùng, sản phẩm) xuất hiện ở nhiều tập

Giai đoạn	Công việc thực hiện trong giai đoạn	Ý nghĩa
2. Gộp miền và lọc k-core	Gắn nhãn miền, đặt ID sản phẩm có tiền tố miền Gộp các miền Lọc lặp (tối đa 20 vòng) để mỗi người dùng và mỗi sản phẩm có ít nhất 5 tương tác	Lọc sau khi gộp cho phép đếm tương tác xuyên miền; ngưỡng tối thiểu giúp mô hình cộng tác có đủ tín hiệu
3. Ánh xạ ID và chia tập	Gán chỉ số nguyên cho người dùng và sản phẩm Tính hai mốc thời gian theo phân vị toàn cục và cắt thành train/validation/test với tỷ lệ 0,8/0,1/0,1 Chạy các kiểm tra rò rỉ	Chia theo thời gian mô phỏng việc dự đoán tương lai từ quá khứ; kiểm tra rò rỉ bảo đảm kết quả đánh giá không bị thổi phồng

Bảng 3.2. Ba giai đoạn của pipeline dữ liệu

- Ba kiểm tra rò rỉ được thực hiện tự động:
    - Thời điểm lớn nhất của tập train không vượt thời điểm nhỏ nhất của validation, và tương tự giữa validation với test
    - Không bản ghi nào xuất hiện ở hai tập
    - Khi dựng hồ sơ người dùng cho một mốc thời gian, không tương tác nào có thời gian sau mốc đó.
- Ngoài ra, hệ thống có công cụ chẩn đoán phép chia, báo cáo tỷ lệ sản phẩm và người dùng ở validation/test đã xuất hiện trong train, tỷ lệ cặp lặp, và tỷ lệ bản ghi có thể đạt được (reachable) – tức sản phẩm đã có trong train và cặp người dùng – sản phẩm chưa nằm trong lịch sử. Chỉ số này cần thiết để diễn giải Recall thấp: nếu một phần mục tiêu không thể đạt được theo phép chia thì Recall thấp chưa chắc do mô hình yếu.

3.2.2 Định danh và ánh xạ

Mỗi sản phẩm có định danh dạng "miền::mã gốc" (ví dụ Video_Games::B00…), nhằm tránh trùng mã giữa các miền khi gộp dữ liệu; mã gốc vẫn được giữ trong một cột riêng và miền được lưu thành cột, không suy ra bằng cách tách chuỗi ID. Ngược lại, ID người dùng không gắn tiền tố vì đây là định danh toàn cục của Amazon, giữ nguyên để nối được hành vi của cùng một người ở nhiều miền.

Ở giai đoạn 3, ID được ánh xạ sang chỉ số nguyên liên tục để dựng ma trận tương tác. Ánh xạ được xây dựng trên toàn bộ dữ liệu, kể cả validation và test, để mọi sản phẩm đều có chỉ số; gặp ID chưa có trong ánh xạ, hệ thống báo lỗi thay vì âm thầm bỏ qua. Việc giữ song song ID gốc và chỉ số nguyên là điều kiện để điểm số tính trên chỉ số nội bộ luôn được dịch đúng về sản phẩm khi hiển thị cho người dùng.

3.2.3 ItemCorpus

- ItemCorpus là kho metadata dùng chung cho các công cụ, được dựng từ metadata của các miền và chỉ giữ những sản phẩm có trong ánh xạ ID; nếu độ phủ thấp hơn ngưỡng tối thiểu thì quá trình dựng báo lỗi. Kho được hiện thực bằng SQLite chạy trong bộ nhớ với ba thành phần:

  - Bảng sản phẩm: ID, miền, ID gốc, tiêu đề, cửa hàng, giá, điểm đánh giá trung bình, số lượt đánh giá, danh mục chính. Giá được làm sạch (bỏ ký hiệu tiền tệ và dấu phân cách); giá dạng khoảng hoặc thiếu được đặt là rỗng.
  - Bảng danh mục: quan hệ nhiều – nhiều giữa sản phẩm và danh mục, vì một sản phẩm có thể thuộc nhiều danh mục.
  - Chỉ mục toàn văn FTS5 trên tiêu đề, cửa hàng, danh mục chính và danh mục, dùng bộ tách từ porter/unicode61, kèm các chỉ mục B-tree cho ID, miền, giá và cặp (sản phẩm, danh mục).
- Kho được khóa ở chế độ chỉ đọc bằng hai lớp: tham số chỉ đọc của SQLite và một bộ ủy quyền chỉ cho phép câu lệnh SELECT. Mỗi truy vấn có giới hạn thời gian (mặc định 5 giây) và giới hạn số dòng trả về để tránh một truy vấn nặng làm treo cả lượt chạy.
- Lý do chọn SQLite trong bộ nhớ là kho cần được dựng nhanh, không đòi hỏi dịch vụ cơ sở dữ liệu riêng, cho kết quả tất định và đủ nhẹ cho nguyên mẫu; FTS5 có sẵn trong SQLite nên tìm kiếm văn bản không cần hạ tầng bổ sung. Đổi lại, kho bị giới hạn bởi dung lượng bộ nhớ và phải dựng lại mỗi lần chạy.
- Hai đặc điểm của dữ liệu ảnh hưởng đến thiết kế công cụ:
  - Nhiều sản phẩm thiếu giá (khoảng 55% ở miền Video_Games), do đó mọi bộ lọc giá mặc định loại sản phẩm không có giá, trừ khi người gọi yêu cầu giữ lại.
  - Điểm đánh giá trung bình và số lượt đánh giá là số liệu tổng hợp đến cuối bộ dữ liệu chứ không tại thời điểm chia tập, nên không được xem là đặc trưng nhất quán về thời gian khi phân tích.

3.2.4 Phân biệt kho sản phẩm và dữ liệu tương tác

ItemCorpus cung cấp thuộc tính mô tả sản phẩm; dữ liệu tương tác cung cấp tín hiệu hành vi cho mô hình gợi ý. Hai nguồn được kết hợp có chủ đích ở những bước xác định, chẳng hạn MemoryTool nối lịch sử tương tác với danh mục của sản

phẩm để tạo hồ sơ sở thích. Metadata không phải bằng chứng về sở thích cá nhân, và lịch sử tương tác không phải nguồn đầy đủ cho thuộc tính sản phẩm; việc kết hợp phải tuân theo mốc thời gian để thông tin của validation/test không rò rỉ vào quá trình tạo gợi ý.

3.3 Mô hình gợi ý và bộ chấm điểm ứng viên

3.3.1 Các mô hình gợi ý

Hệ thống cài đặt bốn mô hình ở mức baseline, đại diện cho các mức độ phức tạp khác nhau:

- Random: sắp xếp ngẫu nhiên có hạt giống, làm mốc dưới.
- Popularity: xếp theo tần suất sản phẩm trong tập huấn luyện. Mô hình này cũng làm phương án dự phòng cho người dùng chưa có lịch sử (cold-start).
- ItemKNN: tính độ tương đồng cosine giữa các sản phẩm từ ma trận đồng xuất hiện, mỗi sản phẩm giữ k láng giềng gần nhất (k = 20 theo cấu hình). Điểm của một ứng viên đối với người dùng là tổng độ tương đồng giữa ứng viên và các sản phẩm trong lịch sử của người dùng. Ma trận tương đồng được tính theo từng khối để không làm tràn bộ nhớ.
- BPR-MF: phân rã ma trận học theo hàm mất mát xếp hạng cặp, cài đặt bằng NumPy với lấy mẫu phủ định và cập nhật theo lô (learning rate 0,001; 20 epoch theo cấu hình).

Việc truyền ma trận tương đồng của ItemKNN cho cả công cụ truy xuất (ItemCFTool) và bộ chấm điểm giúp cùng một tri thức cộng tác phục vụ hai giai đoạn: tìm ứng viên và xếp hạng ứng viên.

3.3.2 Bộ chấm điểm ứng viên

CandidateScorer là giao diện chuẩn giữa RecoModelTool và mô hình gợi ý. Một bộ chấm điểm cần cho biết một người dùng có được mô hình biết đến hay không, những sản phẩm người đó đã tương tác, và điểm của từng ứng viên trong danh sách đầu vào.

Bộ chấm điểm hiện có (BaselineScorer) bọc các mô hình Popularity và ItemKNN: với người dùng có lịch sử, điểm là tổng độ tương đồng cộng một thành phần popularity rất nhỏ để phá hòa; với người dùng không có lịch sử, điểm chỉ dựa vào popularity. Bộ chấm điểm kiểm tra độ dài, hình dạng và tính hữu hạn của điểm trước khi sử dụng.

Lý do dùng giao diện là để RecoModelTool không phụ thuộc vào một mô hình cụ thể: nếu cần thay bằng mô hình khác (chẳng hạn BPR-MF hoặc mô hình tuần tự), chỉ cần viết thêm một bộ chấm điểm tuân theo giao diện mà không sửa công cụ hay tác tử.

3.4 Hệ thống công cụ

3.4.1 Hợp đồng chung của công cụ

Mỗi công cụ là một lớp con của lớp cơ sở Tool và phải khai báo bốn thuộc tính: tên, mô tả, lược đồ đầu vào và lược đồ đầu ra. Đây chính là phần duy nhất LLM nhìn thấy khi lập kế hoạch. Lớp cơ sở thực thi các quy ước sau:

- Công cụ bị từ chối ngay khi định nghĩa nếu thiếu một trong bốn thuộc tính hoặc dùng tên tham số bị cấm, nhằm bảo đảm mọi công cụ đều tự mô tả được.
- Khi được gọi, lớp cơ sở đo thời gian, ghi nhật ký (đầu vào rút gọn, tóm tắt đầu ra, độ trễ) và trả về một kết quả chuẩn gồm trạng thái thành công, dữ liệu và thông báo lỗi.
- Lỗi do đầu vào không hợp lệ (bộ lọc lạ, SQL không an toàn, miền không tồn tại…) được chuyển thành kết quả thất bại có thông báo rõ ràng để LLM có thể sửa kế hoạch; lỗi bất ngờ khác được ghi log rồi ném lại, không bị che giấu.
- Các hàm kiểm tra đầu vào dùng chung (danh sách, số nguyên, giới hạn top-k) cho thông báo lỗi thống nhất; ID ứng viên có thể được truyền dưới dạng danh sách chuỗi hoặc trực tiếp là đầu ra của công cụ khác.

Lý do của hợp đồng này là LLM chỉ có thể dùng đúng công cụ nếu hiểu được chúng từ mô tả và lược đồ; đồng thời một lỗi đầu vào cần được biểu diễn như dữ liệu để cơ chế reflection xử lý được.

3.4.2 Phân nhóm và tổng quan các công cụ

Sáu công cụ được chia thành bốn nhóm theo chức năng (Bảng 3.4). Các giới hạn thời gian chạy dùng chung (tối đa 1000 ứng viên, 100 dòng cho mỗi truy vấn metadata, 5 giây cho truy vấn SQL) được đặt trong cấu hình.

Công cụ	Nhóm	Đầu vào chính	Đầu ra chính	Cấp cho planner
SemanticSearchTool	Truy xuất	Truy vấn văn bản, bộ lọc, giới hạn	Ứng viên kèm điểm BM25, tổng số khớp	Có
ItemCFTool	Truy xuất	Danh sách sản phẩm hạt giống, top-k, miền	Ứng viên kèm độ tương đồng, sản phẩm hạt giống không nhận diện được	Có
SQLTool	Truy xuất	Bộ lọc, thứ tự sắp xếp, giới hạn	Ứng viên, tổng số khớp	Chưa
RecoModelTool	Xếp hạng	Người dùng, ứng viên, top-k, cờ loại sản phẩm đã xem	Danh sách xếp hạng, cờ cold-start, sản phẩm không có điểm	Có
QueryTool	Tra cứu	Danh sách ID hoặc một câu SELECT	Metadata, ID không tồn tại	Có

Công cụ	Nhóm	Đầu vào chính	Đầu ra chính	Cấp cho planner
MemoryTool	Bộ nhớ	Người dùng, số danh mục hàng đầu	Số tương tác, danh mục hàng đầu, khoảng giá	Chỉ ở cấu hình có memory

Bảng 3.3. Tổng quan các công cụ

3.4.3 SemanticSearchTool

- SemanticSearchTool là công cụ tìm kiếm sản phẩm theo nội dung văn bản của truy vấn. Cài đặt hiện tại dựa trên đối sánh từ khóa và xếp hạng BM25 trên chỉ mục văn bản của kho sản phẩm, chưa sử dụng embedding.
- Chức năng: nhận một truy vấn dạng từ khóa cùng các bộ lọc metadata (miền sản phẩm, danh mục, khoảng giá) và trả về danh sách sản phẩm ứng viên. Đây là công cụ duy nhất có thể khởi tạo tập ứng viên khi người dùng không có sản phẩm hạt giống và ít lịch sử. Điều này cần thiết vì khoảng 72% người dùng trong dữ liệu chỉ có một tương tác, nên với nhóm này nguồn thông tin chính là nội dung yêu cầu.
- Cách thực hiện:
    1. Truy vấn được tách thành các từ và loại bỏ từ dừng, kể cả các từ chỉ mức giá như “cheap”, “best”, vì mức giá được biểu diễn bằng bộ lọc chứ không bằng từ khóa.
    2. Mỗi từ được đặt trong dấu nháy và nối với nhau bằng toán tử OR, nhờ đó văn bản của người dùng không thể chèn cú pháp truy vấn của FTS5.
    3. Truy vấn chạy trên chỉ mục FTS5, kết quả xếp theo điểm BM25; các sản phẩm đồng điểm được phân định theo số lượt đánh giá, rồi theo mã sản phẩm.
    4. Công cụ đếm thêm tổng số sản phẩm khớp. Nếu phép đếm quá thời gian thì vẫn trả ứng viên và đánh dấu tổng số là không xác định.

- Lược đồ đầu vào của công cụ liệt kê các miền sản phẩm có trong kho, để planner chỉ chọn miền có thật.
- Lý do lựa chọn: đối sánh từ khóa không cần hạ tầng bổ sung, chạy ngay trong cơ sở dữ liệu của kho sản phẩm và cho kết quả tất định, thuận lợi cho việc kiểm thử và so sánh giữa các cấu hình.
- Giới hạn: do chỉ đối sánh từ vựng, công cụ có thể bỏ sót sản phẩm dùng từ đồng nghĩa hoặc cách diễn đạt khác. Dữ liệu bằng tiếng Anh nên yêu cầu tiếng Việt phải được planner chuyển thành từ khóa tiếng Anh trước khi gọi. Danh mục trong bộ lọc phải khớp chính xác. Truy xuất bằng embedding là hướng mở rộng, chỉ nên bổ sung khi đo được rằng candidate recall của truy xuất từ vựng là điểm nghẽn.

3.4.4 ItemCFTool

- Chức năng: Từ một hoặc nhiều sản phẩm hạt giống, tìm các sản phẩm tương tự theo lọc cộng tác item-based. Công cụ phù hợp khi yêu cầu có sản phẩm tham chiếu cụ thể, ví dụ "game giống Elden Ring".
- Cách thực hiện: Công cụ loại trùng seed, ánh xạ ID sang chỉ số, lấy các hàng tương ứng của ma trận tương đồng ItemKNN, cộng điểm theo cột, loại chính các seed, sắp xếp giảm dần rồi dịch ngược về ID và cắt top-k. Các hàng được xử lý ở dạng thưa nên không bao giờ biến ma trận thành dày. Seed không có trong ma trận được liệt kê trong danh sách "không nhận diện được" thay vì làm công cụ thất bại. Mặc định công cụ trả ứng viên xuyên miền; có thể lọc theo một miền.
- Giới hạn: Vì mỗi sản phẩm chỉ giữ k láng giềng gần nhất khi huấn luyện, và bộ lọc miền được áp dụng sau khi gộp, kết quả có thể ít hơn top-k yêu cầu. Công cụ không có bộ lọc giá hay danh mục, nên các ràng buộc đó phải được xử lý bằng công cụ khác. Tổng độ tương đồng của nhiều seed có thể lớn hơn 1 và không phải xác suất.

3.4.5 SQLTool

- Chức năng: Lọc sản phẩm theo điều kiện có cấu trúc: miền, khoảng giá, điểm đánh giá tối thiểu, số lượt đánh giá tối thiểu, danh mục (khớp một trong hoặc khớp tất cả), cửa hàng, từ trong tiêu đề, danh sách ID cần loại.
- Cách thực hiện. Bộ lọc do LLM đưa ra được biên dịch thành mệnh đề WHERE có tham số; khóa bộ lọc ngoài danh sách cho phép bị từ chối, cột sắp xếp bị giới hạn trong danh sách định sẵn, giá trị người dùng không bao giờ được nối trực tiếp vào câu lệnh. Do đó LLM không có đường nào để chèn SQL tùy ý. Kết quả được sắp xếp tất định (giá trị rỗng xếp cuối, rồi theo ID).
- Vị trí trong hệ thống. SQLTool đã được cài đặt và kiểm thử nhưng chưa nằm trong bộ công cụ của planner. Lý do là SemanticSearchTool đã dùng chung bộ biên dịch bộ lọc này và hỗ trợ cả văn bản lẫn thuộc tính, nên giữ không gian hành động của planner nhỏ gọn sẽ làm giảm khả năng chọn sai công cụ. Nếu cần lọc thuần cấu trúc, công cụ có thể được cấp thêm cho planner kèm cập nhật hướng dẫn trong prompt.

3.4.6 RecoModelTool

- Chức năng. Xếp hạng một tập ứng viên cho một người dùng bằng mô hình gợi ý. Đây là điểm nối giữa các công cụ truy xuất và tín hiệu hành vi: bước truy xuất cho biết sản phẩm nào có thể liên quan, bước này quyết định thứ tự cuối.
- Cách thực hiện. Công cụ chuẩn hóa và loại trùng ID ứng viên, ánh xạ người dùng, xác định có phải cold-start hay không, tách ứng viên thành nhóm chấm được điểm và nhóm không có trong ma trận, tùy chọn loại những sản phẩm người dùng đã tương tác, gọi bộ chấm điểm, kiểm tra điểm hợp lệ rồi sắp xếp ổn định. Đầu ra gồm danh sách xếp hạng, tên mô hình, cờ cold-start, danh sách sản phẩm bị loại vì đã xem và danh sách sản phẩm không có điểm.
- Lý do. Việc xếp hạng do mô hình thực hiện, không phải LLM, để thứ tự gợi ý có thể truy vết và đánh giá bằng các chỉ số xếp hạng chuẩn. Việc báo cáo cold-start, sản phẩm đã loại và sản phẩm không có điểm thay vì ẩn chúng giúp phân biệt “danh sách rỗng vì không có ứng viên” với “danh sách rỗng vì mô hình

không biết người dùng". Công cụ chỉ xếp hạng danh sách đầu vào: nếu sản phẩm mục tiêu không nằm trong ứng viên thì không thể đưa lên cao.

3.4.7 QueryTool

- Chức năng. Lấy metadata của các sản phẩm đã biết ID (tên, giá, đánh giá, cửa hàng…) để dùng cho câu trả lời cuối. Công cụ nhận hoặc một danh sách ID hoặc một câu SELECT chỉ đọc, không nhận đồng thời cả hai.
- Cách thực hiện. Với danh sách ID, công cụ loại trùng, truy vấn một lần bằng điều kiện IN, trả kết quả theo đúng thứ tự đầu vào và liệt kê ID không tồn tại. Số dòng trả về bị chặn (100) vì kết quả có thể đi thẳng vào prompt.
- Lý do. QueryTool tách riêng việc "biết sản phẩm là gì" khỏi việc "sắp xếp sản phẩm", nên việc lấy metadata không bao giờ làm thay đổi thứ tự gợi ý và câu trả lời cuối có thể được kiểm tra chỉ nhắc đến các ID có trong kết quả.

3.4.8 MemoryTool

- Chức năng. Tổng hợp hồ sơ sở thích của một người dùng từ lịch sử tương tác: số tương tác, các danh mục hay xuất hiện nhất và khoảng giá thường mua.
- Cách thực hiện. Công cụ đọc lịch sử người dùng, tính danh mục và khoảng giá bằng truy vấn SQL tổng hợp trên kho sản phẩm (nên không bị giới hạn số dòng của truy vấn thông thường), danh mục được đếm theo số tương tác. Tham số mốc thời gian cho phép chỉ dùng lịch sử đến một thời điểm, kết hợp kiểm tra rò rỉ. Người dùng lạ hoặc chưa có tương tác không gây lỗi mà trả về hồ sơ rỗng được đánh dấu "chưa biết", nhất quán với cách RecoModelTool xử lý cold-start. Đầu ra có thể được dùng làm tham số cho bước sau, chẳng hạn danh mục hàng đầu trở thành bộ lọc danh mục của bước tìm kiếm.
- Lý do và giới hạn. MemoryTool chỉ đọc vì bộ dữ liệu Amazon Reviews không có hội thoại để trích xuất sở thích; do đó "memory" ở đây là hồ sơ suy ra từ lịch sử, không phải bộ nhớ hội thoại dài hạn có ghi, cập nhật hay quên.

3.4.9 Kết nối công cụ và phân loại lỗi

Các công cụ được nối với nhau bằng tham chiếu đến kết quả của bước trước, viết dưới dạng $N.đường_dẫn, trong đó N là số thứ tự bước; dấu * lấy một trường từ mọi phần tử của danh sách. Ví dụ, $1.candidates là tập ứng viên của bước 1 và $2.ranked.*.item_id là danh sách ID đã xếp hạng ở bước 2. Chuỗi tham chiếu phải chiếm toàn bộ giá trị tham số. Cơ chế này cho phép chuyển ứng viên từ bước truy xuất sang bước xếp hạng rồi sang bước lấy metadata mà LLM không cần chép lại dữ liệu, qua đó giảm lỗi sao chép và giảm số token.

Khi phân tích quỹ đạo thực thi, các lỗi được phân loại theo nguyên nhân vì mỗi loại cần cách khắc phục khác nhau (Bảng 3.5).

Tình huống	Cách hệ thống xử lý	Ý nghĩa khi phân tích
Kế hoạch không hợp lệ (JSON hỏng)	Thử lại kèm thông báo lỗi, tối đa một lần	Lỗi của LLM ở bước lập kế hoạch
Công cụ lạ hoặc tham số ngoài lược đồ	Không gọi công cụ; báo lỗi kèm gợi ý (ví dụ tham số giá phải nằm trong bộ lọc)	Lỗi hiểu lược đồ công cụ
ID do LLM bịa	Không gọi công cụ; ghi nhận ID bịa; không kích hoạt reflection	Dấu hiệu ảo giác, cần theo dõi riêng
Tham chiếu hỏng hoặc sai kiểu	Chỉ bước đó thất bại, các bước còn lại vẫn chạy	Lỗi nối công cụ
Công cụ trả lỗi hoặc hết thời gian	Ghi trạng thái; reflection có thể sửa (trừ lỗi hết thời gian SQL)	Lỗi vận hành hoặc truy vấn quá nặng
Công cụ thành công nhưng rỗng	Có thể kích hoạt reflection	Vấn đề truy xuất hoặc bộ lọc quá chặt

Tình huống	Cách hệ thống xử lý	Ý nghĩa khi phân tích
Thành công nhưng không liên quan mục tiêu	Không phát hiện được ở thời điểm chạy; phát hiện qua chỉ số đánh giá	Vấn đề chất lượng truy xuất hoặc xếp hạng

Bảng 3.4. Phân loại các tình huống lỗi khi thực thi

3.5 Tầng tác tử

3.5.1 Lớp truy cập mô hình ngôn ngữ

Phần còn lại của hệ thống chỉ biết đến giao diện LLMProvider, gồm hai thao tác: sinh phản hồi từ danh sách tin nhắn (có tham số nhiệt độ, số token tối đa và chế độ đầu ra JSON) và kiểm tra kết nối. Bản cài đặt hiện tại là provider tương thích API kiểu OpenAI, gọi qua HTTP nên dùng được với OpenAI hoặc các máy chủ cục bộ như vLLM, Ollama, LM Studio. Thông tin kết nối (nhà cung cấp, mô hình, địa chỉ, khóa) nằm trong cấu hình; địa chỉ và khóa được lấy từ biến môi trường để không lưu bí mật trong tệp cấu hình. Mỗi phản hồi kèm số token vào/ra và độ trễ để phục vụ đánh giá chi phí. Mọi lỗi mạng, hết thời gian hay JSON hỏng đều được chuyển thành một loại lỗi thống nhất của lớp này.

Nhiệt độ mặc định được đặt bằng 0 để giảm biến thiên giữa các lần chạy. Việc tách giao diện khỏi bản cài đặt cho phép đổi mô hình hoặc nhà cung cấp mà không sửa tác tử, và cho phép thay LLM bằng bản giả lập khi kiểm thử.

3.5.2 Bộ công cụ theo từng loại tác tử

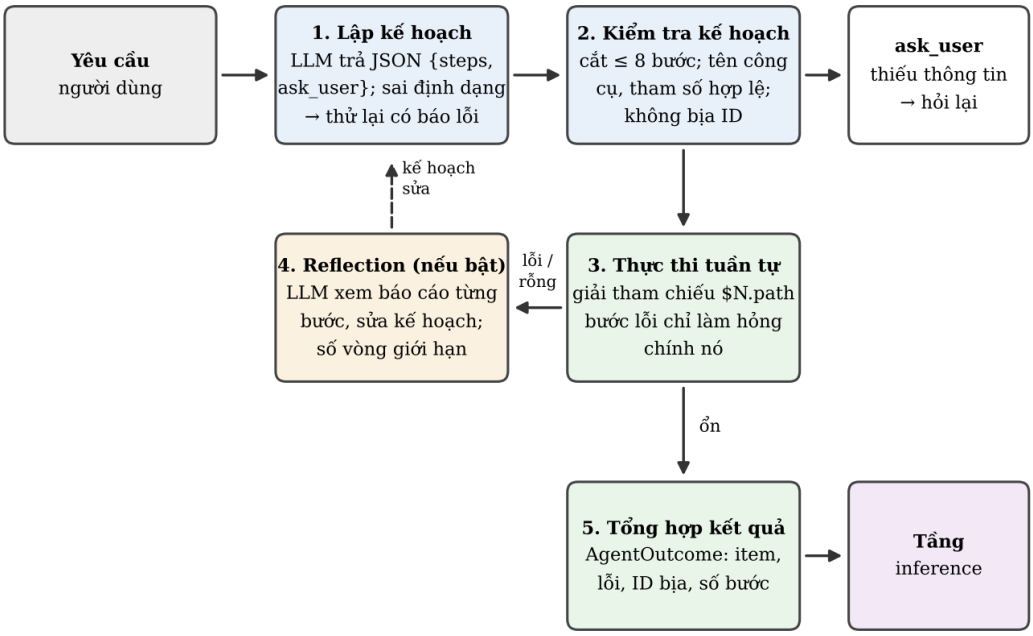
Danh sách công cụ mà mỗi tác tử được dùng được khai báo tập trung tại một nơi. Planner mặc định có bốn công cụ: SemanticSearchTool, ItemCFTool, RecoModelTool và QueryTool. Cấu hình planner có memory bổ sung MemoryTool. SQLTool không thuộc cấu hình nào. Việc khai báo tập trung bảo đảm rằng khi so sánh hai cấu hình, chênh lệch về công cụ là có chủ đích và đo được, và mô tả công cụ trong prompt luôn khớp với công cụ thực sự khả dụng.

3.5.3 Type1Agent: pipeline cố định

- Type1Agent thực hiện chuỗi xác định: ItemCFTool → RecoModelTool → QueryTool.
- Cách thực hiện: ItemCFTool lấy số ứng viên bằng giá trị lớn hơn giữa (top-k nhân hệ số 5) và 50, rộng hơn top-k cần trả về để bước xếp hạng có chỗ lựa chọn; RecoModelTool xếp hạng các ứng viên cho người dùng; QueryTool gắn metadata cho các sản phẩm được chọn. Nếu một bước thất bại, tác tử trả về lỗi kèm tên giai đoạn (truy xuất, xếp hạng hoặc lấy metadata).
- Vì sao có tác tử này: không cần LLM lập kế hoạch nên chi phí thấp, hành vi tất định và dễ phân tích lỗi theo từng giai đoạn; nhờ đó nó là mốc đối chứng để tách riêng phần đóng góp (và chi phí) của việc lập kế hoạch bằng LLM. Đổi lại, pipeline chỉ áp dụng cho yêu cầu có sản phẩm hạt giống và phụ thuộc vào khả năng tạo ứng viên của ItemCF, nên gặp khó với người dùng cold-start hoặc yêu cầu mô tả bằng văn bản.

3.5.4 PlanExecutor: lập kế hoạch bằng LLM

- PlanExecutor chuyển yêu cầu thành một kế hoạch có cấu trúc rồi thực thi. Quy trình (Hình 3.3) gồm các bước sau.



Hình 3.3. Quy trình lập kế hoạch, kiểm tra và thực thi của PlanExecutor

- Dựng prompt. Với mỗi công cụ, hệ thống đưa vào prompt tên, mô tả, lược đồ đầu vào và lược đồ đầu ra (để LLM biết trường nào có thể dùng trong tham chiếu). Prompt hệ thống nêu các quy tắc: chỉ dùng công cụ và tham số có sẵn, không bịa ID, cách viết tham chiếu, luôn xếp hạng bằng RecoModelTool rồi lấy metadata bằng QueryTool, và hỏi lại người dùng khi thiếu thông tin.
- Sinh kế hoạch. LLM trả về một đối tượng JSON gồm danh sách bước (mỗi bước có tên công cụ và tham số) và một trường câu hỏi làm rõ. Phản hồi sai định dạng được thử lại một lần kèm thông báo lỗi; phần bao quanh kiểu khối mã (nếu có) được tự động bóc ra. Kế hoạch dài hơn 8 bước bị cắt và được đánh dấu.
- Kiểm tra trước khi chạy. Công cụ lạ bị từ chối; các định danh người dùng, sản phẩm được ghi trực tiếp trong tham số mà không xuất hiện trong câu yêu cầu bị coi là ID bịa và không được thực thi; tham số không có trong lược đồ bị báo lỗi kèm gợi ý.

  - Thực thi tuần tự. Từng bước được chạy theo thứ tự, tham chiếu được giải quyết từ kết quả thật của bước trước. Bước lỗi chỉ làm hỏng chính nó và bước phụ thuộc vào nó; các bước độc lập vẫn chạy.
  - Tổng hợp. Kết quả được ghi cùng kế hoạch, câu hỏi làm rõ, mức sử dụng token, độ trễ, số lần thử và cờ cắt kế hoạch, rồi chuyển sang tầng inference.
- Nếu thiếu thông tin quan trọng, LLM được hướng dẫn trả về kế hoạch rỗng cùng câu hỏi làm rõ; hệ thống xem đây là kết quả hợp lệ thay vì buộc tác tử đoán.
- Lý do cho lập kế hoạch một lần (plan-first): Thiết kế này dùng một lần gọi LLM để sinh toàn bộ kế hoạch rồi mới thực thi, thay vì vòng lặp quan sát – hành động từng bước. Ưu điểm là ít lần gọi LLM hơn, kế hoạch được kiểm tra toàn bộ trước khi chạy (chặn ID bịa và tham số sai ngay từ đầu), và quỹ đạo thực thi được ghi lại thành một đối tượng có cấu trúc để đánh giá. Nhược điểm là tác tử không thể thay đổi hướng giữa chừng dựa trên kết quả trung gian; cơ chế reflection là biện pháp giảm nhẹ hạn chế này, nhưng ở phạm vi hẹp.
- Lý do cho tính linh hoạt kéo theo rủi ro: Planner có thể thay đổi thứ tự và tổ hợp công cụ theo từng yêu cầu: yêu cầu có sản phẩm hạt giống ưu tiên ItemCFTool, yêu cầu mô tả thể loại bắt đầu bằng SemanticSearchTool, yêu cầu cần cá nhân hóa có thể dùng hồ sơ từ MemoryTool. Đổi lại, LLM có thể chọn sai công cụ, truyền tham số sai, thiếu bước xếp hạng hoặc dùng kết quả truy xuất không đúng. Vì vậy hiệu quả của planner phải được đánh giá cả ở chất lượng sản phẩm lẫn hành vi thực thi.

3.5.5 Reflection và cơ chế thử lại

Reflection ở đây là cơ chế sửa kế hoạch trong phạm vi một yêu cầu, không phải tự cải thiện dài hạn. Cơ chế được kích hoạt khi kế hoạch chạy xong nhưng có bước lỗi, hoặc không tạo ra sản phẩm nào (và đó không phải trường hợp hỏi lại người

dùng). Khi đó hệ thống gửi cho LLM một báo cáo từng bước (công cụ, trạng thái, nội dung lỗi được cắt còn khoảng 300 ký tự), nhận kế hoạch đã sửa và chạy lại. Số vòng tối đa là một tham số cấu hình (mặc định 1; đặt 0 để tắt).

Hai trường hợp không được reflection vì kế hoạch mới không giúp ích: kế hoạch dùng ID bịa (cần xử lý như một lỗi ảo giác, không nên "sửa cho khớp") và truy vấn SQL chạm giới hạn thời gian. Nếu phản hồi reflection không dùng được hoặc nhà cung cấp LLM báo lỗi, phản hồi đó bị bỏ qua và giữ kết quả trước. Giới hạn số vòng và các điều kiện loại trừ nhằm kiểm soát chi phí và tránh vòng lặp vô hạn. Để phục vụ đánh giá, hệ thống ghi lại số lần reflection thực sự diễn ra, kết quả của lần chạy đầu tiên và kế hoạch của từng lượt.

3.5.6 Memory và cá nhân hóa

Ở cấu hình planner có memory, MemoryTool được thêm vào danh sách công cụ và prompt hướng dẫn cách dùng. Planner có thể lấy hồ sơ người dùng trước rồi truyền danh mục hàng đầu hoặc khoảng giá vào bộ lọc của bước tìm kiếm (tham chiếu như $1.top_categories). Cơ chế này nhằm giúp lựa chọn công cụ và xây dựng truy vấn phù hợp hơn với người dùng có lịch sử. Vì memory là hồ sơ suy ra từ lịch sử (mục 3.5.8), nó có ích với yêu cầu cá nhân hóa nhưng không thay thế được thông tin mà người dùng nêu trực tiếp trong yêu cầu.

3.5.7 Phạm vi hội thoại nhiều lượt [Phần này nhóm chưa hoàn thiện]

Hệ thống hiện xử lý từng yêu cầu như một lượt độc lập: tác tử không giữ trạng thái giữa các lượt và chưa có định danh hội thoại. Việc có memory hoặc gọi LLM nhiều lần trong một yêu cầu không đồng nghĩa với hỗ trợ hội thoại nhiều lượt. Để mở rộng, kiến trúc cần bổ sung:

- Một định danh hội thoại và kho lưu trạng thái theo phiên
- Cơ chế truyền ràng buộc và sản phẩm đã gợi ý của lượt trước vào bước lập kế hoạch lượt sau
- Phân biệt ràng buộc hiện thời với sở thích được rút ra từ lượt trước. Đây là hướng phát triển được nêu ở chương 6, chưa thuộc phần đã cài đặt.

3.6 Tầng inference và tạo câu trả lời

3.6.1 Vai trò và đầu vào

Tầng inference chỉ làm nhiệm vụ trình bày: nhận kết quả mà tác tử đã chọn và viết câu trả lời cho người dùng. Tầng này không chọn lại sản phẩm, không thêm, bớt hay đổi thứ tự; nó cũng không phải vòng lặp quan sát – quyết định tiếp theo của tác tử.

Kết quả của tác tử được chuyển thành một đối tượng chung (AgentOutcome) gồm văn bản yêu cầu, danh sách sản phẩm, câu hỏi làm rõ (nếu có), các lỗi, các ID bịa, cờ cold-start và số bước. Danh sách sản phẩm được lấy theo thứ tự ưu tiên: danh sách xếp hạng của RecoModelTool, rồi kết quả của công cụ truy xuất, rồi QueryTool, để phản ánh đúng điều người dùng sẽ nhìn thấy. Sản phẩm được loại trùng và cắt theo giới hạn hiển thị (mặc định 10); metadata lấy từ đầu ra của công cụ chứ không do LLM bổ sung. Đầu ra của tầng là một đối tượng Answer gồm trạng thái, nội dung, danh sách sản phẩm, cờ "có bám dữ liệu", nguồn sinh (template hay LLM), lý do fallback và mức sử dụng token.

3.6.2 Phân loại trạng thái

Mỗi kết quả được gán một trong bốn trạng thái theo các luật xét tuần tự, luật đầu tiên khớp sẽ được chọn (Bảng 3.6).

Trạng thái	Điều kiện	Cách trình bày
ask_user	Planner trả câu hỏi làm rõ và không có bước nào được thực thi	Hiển thị câu hỏi của planner
tool_error	Có ID bịa hoặc lỗi lập kế hoạch; hoặc có lỗi công cụ mà không có sản phẩm	Thông báo lỗi cố định
ok	Có sản phẩm (nếu một số bước lỗi, ghi chú kết quả chỉ một phần)	Danh sách sản phẩm kèm diễn giải

Trạng thái	Điều kiện	Cách trình bày
no_result	Không có lỗi nhưng cũng không có sản phẩm	Thông báo không tìm thấy kết quả

Bảng 3.5. Các trạng thái kết quả của tầng inference

3.6.3 Hai chế độ sinh câu trả lời

- Template. Tạo câu trả lời bằng mẫu cố định: văn bản chuẩn cho trạng thái no_result và tool_error, câu hỏi của planner cho ask_user, và danh sách đánh số kèm giá, điểm đánh giá, cửa hàng (nếu có) cho trạng thái ok. Chế độ này tất định, không tốn lượt gọi LLM và luôn sẵn sàng làm phương án dự phòng.
- LLM. Chỉ dùng cho trạng thái ok. LLM nhận danh sách sản phẩm (tiêu đề được cắt còn 120 ký tự) và được yêu cầu trả JSON gồm câu trả lời và danh sách ID đã nhắc đến. Kết quả phải qua kiểm tra grounding; nếu không đạt, hệ thống thử lại tối đa một lần kèm lý do, và nếu vẫn không đạt hoặc nhà cung cấp lỗi thì chuyển sang template và ghi lý do fallback. Tầng này không bao giờ ném lỗi ra ngoài: mọi thất bại đều dẫn tới một câu trả lời hợp lệ.

3.6.4 Grounding và fallback

Cơ chế grounding kiểm tra sự nhất quán giữa câu trả lời và kết quả tác tử gồm: phản hồi là JSON hợp lệ; mọi ID được nhắc đến có trong dữ liệu; câu trả lời đủ sản phẩm và đúng thứ tự; và, khi được bật, giá và điểm đánh giá nêu trong văn bản không lệch quá dung sai (0,5 USD đối với giá). Mục đích là giảm khả năng câu trả lời nhắc đến sản phẩm không có trong kết quả hoặc đưa ra thông tin không được hỗ trợ.
Kiểm tra grounding không chứng minh được mọi phát biểu tự nhiên đều đúng về ngữ nghĩa; nó chỉ bao phủ các thuộc tính kiểm tra được bằng quy tắc (ID, thứ tự, giá, điểm). Việc tách riêng tầng inference còn cho phép đánh giá độc lập: cố định danh sách sản phẩm rồi so sánh hai chế độ về độ bám dữ liệu, tỷ lệ fallback, độ trễ và số token, và bảo đảm mọi khác biệt về câu chữ không bị diễn giải thành cải thiện chất lượng xếp hạng

3.7 Cấu hình, kiểm thử và quy ước kỹ thuật

3.7.1 Cấu hình.

Mọi giá trị chỉnh được nằm trong các tệp YAML chia thành bốn nhóm: đường dẫn, dữ liệu, đánh giá và tác tử (gồm giới hạn công cụ, tham số tác tử, LLM, đánh giá tác tử, inference, demo và ablation). Một mô-đun cấu hình đọc tệp, thay biến môi trường dạng ${TÊN} hoặc ${TÊN:-mặc định}, và là nguồn duy nhất cho đường dẫn dữ liệu; không đoạn mã nào được ghi cứng đường dẫn thư mục dữ liệu hay thí nghiệm. Các nhóm tham số quan trọng được kiểm tra hợp lệ khi nạp (ví dụ số vòng reflection không âm), nên cấu hình sai sẽ báo lỗi sớm thay vì gây hành vi khó hiểu giữa chừng. Khóa LLM chỉ được đọc từ biến môi trường và có kiểm thử bảo đảm tệp cấu hình không chứa bí mật.

3.7.2 Quy ước cấu trúc mã.

Mỗi mô-đun công khai chỉ chứa một lớp hoặc hàm công khai và trùng tên tệp; tên công khai là duy nhất trong toàn gói; mô-đun bắt đầu bằng dấu gạch dưới chỉ dùng nội bộ trong thư mục chứa nó; mọi lệnh nhập phải trỏ tới mô-đun có thật. Các quy ước này được biến thành kiểm thử tự động cùng với kiểm tra hướng phụ thuộc giữa các tầng (mục 3.2.3). Lý do là mã được nhiều thành phần (đánh giá, demo, tác tử) cùng sử dụng, nên cần quy ước ép buộc để tìm đúng thành phần nhanh và ngăn phụ thuộc vòng.

3.7.3 Kiểm thử.

Bộ kiểm thử pytest được tổ chức theo từng nhóm thành phần: cấu hình, dữ liệu (kể cả kiểm tra rò rỉ cả hai chiều: đạt khi hợp lệ và báo lỗi khi vi phạm), công cụ (với một "thế giới nhỏ" có giá trị mong đợi tính tay), tác tử (kể cả reflection, ID bịa, tham chiếu), inference, đánh giá và demo. Việc dùng LLM giả lập theo kịch bản cho phép kiểm tra hành vi của PlanExecutor mà không cần dịch vụ LLM thật.

Mỗi lời gọi công cụ được ghi lại với đầu vào rút gọn, tóm tắt đầu ra và độ trễ; mỗi lượt chạy của tác tử lưu kế hoạch, kết quả từng bước, mức sử dụng token. Đây là nền tảng để phân tích lỗi theo quỹ đạo thay vì chỉ nhìn tỷ lệ thành công.

3.8 Hệ thống đánh giá trong kiến trúc

3.8.1 Bốn bộ đánh giá

Kiến trúc có bốn bộ đánh giá, mỗi bộ trả lời một câu hỏi khác nhau; giao thức thực nghiệm chi tiết được trình bày ở chương 4.

Bộ đánh giá	Câu hỏi trả lời	Cách thực hiện	Cần LLM
Mô hình gợi ý offline	Mô hình nào xếp hạng tốt hơn trên phép chia dữ liệu?	Chấm điểm toàn bộ danh mục theo lô; các chỉ số top-K (K = 5, 10, 20), chia theo nhóm người dùng cold (0 tương tác), sparse (1–4), warm (từ 5)	Không
Candidate recall	Bước truy xuất có đưa sản phẩm mục tiêu vào tập ứng viên không?	Với mỗi nguồn ứng viên (popularity, ItemCF, tìm kiếm văn bản, hợp các nguồn) và mỗi ngân sách (50, 100, 200), đếm sản phẩm held-out xuất hiện	Không
Đánh giá tác tử	Tầng tác tử có đóng góp gì so với mô hình thuần, ở đâu?	Dựng yêu cầu từ dữ liệu giữ lại, gửi cùng một tập yêu cầu cho từng tác tử, chấm quỹ đạo	Có (trừ baseline và Type1Agent)

Bộ đánh giá	Câu hỏi trả lời	Cách thực hiện	Cần LLM
Ablation	Cấu hình A có hơn B không, có ý nghĩa thống kê không?	Ghép cặp theo mã yêu cầu, khoảng tin cậy bootstrap, kiểm định McNemar cho chỉ số nhị phân	Không

Bảng 3.6. Bốn bộ đánh giá trong hệ thống

Giao thức fit/target được dùng thống nhất: khi đánh giá trên validation, mô hình huấn luyện trên train; khi đánh giá trên test, mô hình huấn luyện trên train và validation. Mỗi lượt chạy được lưu thành một thư mục thí nghiệm đánh số gồm cấu hình, chỉ số, các tham số phần cứng, hạt giống và mã phiên bản, nhờ đó so sánh và tái lập được.

Bộ đánh giá tác tử dựng năm loại yêu cầu: tìm sản phẩm tương tự (từ sản phẩm gần nhất trong lịch sử), cá nhân hóa (chỉ từ lịch sử), ràng buộc (danh mục và giá lấy từ sản phẩm giữ lại), mô tả văn bản cho người dùng cold-start (danh mục của sản phẩm giữ lại) và mơ hồ (câu hỏi chung chung, trong đó hỏi lại là hành vi đúng). Bốn tác tử được so sánh: baseline (ItemKNN thuần, không LLM), Type1Agent, planner và planner có memory. Các chỉ số xếp hạng của agent dùng chung hàm tính với bộ đánh giá offline để số liệu so sánh được với nhau.

3.8.2 Tách ba nhóm chất lượng

Ba nhóm chất lượng được theo dõi độc lập, vì mỗi nhóm có nguyên nhân lỗi và cách khắc phục khác nhau (Bảng 3.8).

Nhóm	Câu hỏi cần trả lời	Ví dụ chỉ số
Truy xuất và xếp hạng	Sản phẩm mục tiêu có được đưa vào tập ứng viên và xếp đủ cao không?	Candidate Recall@K, Recall@K, NDCG@K

Nhóm	Câu hỏi cần trả lời	Ví dụ chỉ số
Độ tin cậy của tác tử	Kế hoạch có hợp lệ, công cụ có chạy thành công, yêu cầu có được đáp ứng không?	Tỷ lệ kế hoạch hợp lệ, tỷ lệ công cụ thành công, số bước, task success
Chất lượng câu trả lời cuối	Câu trả lời có dựa đúng vào sản phẩm và thông tin đã truy xuất không?	Grounding, fallback, trạng thái câu trả lời, độ trễ và token

Bảng 3.7. Ba nhóm chất lượng được theo dõi độc lập

Việc tách này đặc biệt quan trọng khi phân tích lỗi: một tác tử có thể hoàn thành mọi bước công cụ nhưng không đưa sản phẩm mục tiêu lên đầu danh sách; ngược lại danh sách tốt vẫn có thể được diễn đạt chưa tốt. Cách đọc có hệ thống là đi từ tầng thấp lên: nếu candidate recall thấp thì ưu tiên cải thiện truy xuất; nếu candidate recall tốt nhưng Recall@K thấp thì kiểm tra bộ chấm điểm, việc loại sản phẩm đã xem, ánh xạ ID và khả năng đạt được của mục tiêu; nếu task success cao nhưng Recall@10 bằng 0 thì task success không đại diện cho chất lượng gợi ý và hai chỉ số phải được báo cáo song song.

3.8.3 Truy vấn ORACLE và truy vấn thực tế

Do không có nhật ký truy vấn thật, yêu cầu đánh giá được sinh từ dữ liệu giữ lại. Một số loại (ràng buộc, mô tả văn bản cho người dùng cold-start) lấy danh mục hoặc giá từ chính sản phẩm mục tiêu nên được đánh dấu là ORACLE: chúng hữu ích để kiểm tra một thành phần nhưng cho cận trên, không đại diện cho truy vấn tự nhiên. Tương tự, trong đánh giá candidate recall, truy vấn dựng từ lịch sử là trung thực nhưng rỗng với người dùng cold-start, còn truy vấn dựng từ danh mục của sản phẩm mục tiêu là cận trên. Kiến trúc đánh giá do đó luôn gắn nhãn loại truy vấn để kết quả ORACLE không bị báo cáo như hiệu năng thực tế.

3.9 Ví dụ xử lý một yêu cầu

Xét yêu cầu: “Tôi là người dùng u1, thích Elden Ring, gợi ý game tương tự dưới 60 đô.” Quá trình xử lý với PlanExecutor diễn ra như sau.

1. LLM nhận yêu cầu cùng mô tả và lược đồ các công cụ, rồi sinh kế hoạch ba bước: (a) tìm kiếm bằng SemanticSearchTool với từ khóa tiếng Anh như “open world action rpg”, bộ lọc miền Video_Games và giá tối đa 60; (b) xếp hạng tập ứng viên của bước (a) cho người dùng u1 bằng RecoModelTool, truyền ứng viên thông qua tham chiếu $1.candidates; (c) lấy metadata của các ID đã xếp hạng bằng QueryTool thông qua tham chiếu $2.ranked.*.item_id.
2. Hệ thống kiểm tra kế hoạch: các công cụ đều tồn tại; u1 xuất hiện trong câu yêu cầu nên không bị xem là ID bịa; tham số giá nằm đúng bên trong bộ lọc. Sau đó ba bước được chạy tuần tự.
3. Kết quả ba bước được tổng hợp thành AgentOutcome; tầng inference phân loại trạng thái là ok rồi sinh câu trả lời (bằng template hoặc LLM kèm kiểm tra grounding) chỉ gồm các sản phẩm đã được xếp hạng.

Ví dụ này minh họa hai quyết định thiết kế. Thứ nhất, ràng buộc giá phải được đặt ngay ở bước truy xuất và nằm trong tham số bộ lọc; chuỗi “ItemCFTool rồi lọc giá” không thực hiện được vì ItemCFTool không có bộ lọc giá và SQLTool vừa không nhận danh sách ứng viên vừa không nằm trong bộ công cụ của planner. Thứ hai, nếu yêu cầu thiếu thông tin (ví dụ không nêu người dùng hoặc sản phẩm tham chiếu), planner trả về câu hỏi làm rõ và tầng inference hiển thị câu hỏi đó thay vì đoán.

3.10 Quyết định thiết kế và giới hạn kiến trúc

3.10.1 Các quyết định thiết kế chính

Bảng 3.9 tổng hợp những quyết định có ảnh hưởng lớn nhất đến kiến trúc, phương án thay thế và đánh đổi đi kèm.

Quyết định	Phương án thay thế	Lý do chọn	Đánh đổi
LLM lập kế hoạch, công cụ thực thi, mô hình xếp hạng	LLM tự liệt kê sản phẩm	Tránh ảo giác về ID và thuộc tính; thứ tự gợi ý kiểm chứng được	Thêm tầng, thêm chi phí điều phối
Lập kế hoạch một lần kèm reflection giới hạn	Vòng lặp quan sát – hành động	Ít lần gọi LLM; kiểm tra kế hoạch trước khi chạy; quỹ đạo dễ ghi và đánh giá	Không điều chỉnh hướng dựa trên kết quả trung gian
Truy xuất từ vựng FTS5/BM25	Truy xuất bằng embedding	Không cần hạ tầng bổ sung, tất định, nhẹ	Bỏ sót đồng nghĩa; cần từ khóa tiếng Anh
SQLite in-memory chỉ đọc	Máy chủ cơ sở dữ liệu hoặc vector DB	Dễ triển khai, dễ tái lập, an toàn trước thao tác ghi	Giới hạn bởi bộ nhớ; dựng lại mỗi lần chạy
Bộ lọc tham số hóa, khóa cho phép	Cho LLM viết SQL tự do	Chống chèn SQL; kiểm tra được đầu vào	Kém linh hoạt hơn
Giữ Type1Agent bên cạnh planner	Chỉ dùng planner	Có mốc đối chứng để tách chi phí và đóng góp của lập kế hoạch bằng LLM	Chỉ phủ yêu cầu có sản phẩm hạt giống

Quyết định	Phương án thay thế	Lý do chọn	Đánh đổi
Inference chỉ trình bày	Cho LLM chọn lại sản phẩm	Tách chất lượng diễn đạt khỏi chất lượng xếp hạng	LLM không sửa được danh sách kém
Chia dữ liệu theo thời gian toàn cục	Chia ngẫu nhiên hoặc leave-one-out	Tránh dùng tương lai để dự đoán quá khứ	Validation/test có thể chứa người dùng, sản phẩm chưa thấy ở train
Memory chỉ đọc, suy từ lịch sử	Bộ nhớ hội thoại có ghi	Dữ liệu không có hội thoại để trích sở thích	Chưa cá nhân hóa theo phiên

Bảng 3.8. Các quyết định thiết kế chính và đánh đổi

3.10.2 Giới hạn kiến trúc hiện tại

Kiến trúc hiện tại có những giới hạn cần được ghi nhận khi diễn giải kết quả:

- Truy xuất văn bản là truy xuất từ vựng, nên chất lượng phụ thuộc vào từ khóa và danh mục chính xác; truy xuất bằng embedding chưa được cài đặt.
- ItemCFTool chỉ dùng các láng giềng đã giữ khi huấn luyện và không có bộ lọc thuộc tính; ứng viên có thể ít hơn số yêu cầu.
- Planner không quan sát kết quả trung gian để đổi hướng; reflection chỉ sửa kế hoạch khi có lỗi hoặc kết quả rỗng.
- Memory là hồ sơ suy ra từ lịch sử; chưa có bộ nhớ hội thoại và chưa hỗ trợ nhiều lượt.
- BPR-MF mới được dùng trong đánh giá offline; chưa là bộ chấm điểm của tác tử.
- Điểm đánh giá trung bình và số lượt đánh giá trong metadata là số liệu tổng hợp đến cuối bộ dữ liệu, không nhất quán theo thời gian với phép chia.

- Giao diện demo (Gradio) chưa nối với tầng inference; câu trả lời hiển thị ở demo hiện do hàm mẫu tất định tạo ra.
- Độ trễ từng công cụ và từng tầng chưa được đo bằng một bộ benchmark thống nhất; các nhận định về hiệu năng vì vậy chỉ mang tính định hướng cho đến khi có phép đo.

CHƯƠNG 4. THỰC NGHIỆM VÀ THIẾT LẬP ĐÁNH GIÁ

4.1 Mục tiêu và nguyên tắc thực nghiệm

4.1.1 Mục tiêu đánh giá

Thực nghiệm được thiết kế để trả lời ba câu hỏi nối tiếp nhau, từ tầng dữ liệu đến tầng tác tử. Cách chia này phản ánh đúng cấu trúc của hệ thống: chất lượng danh sách cuối cùng chỉ có thể tốt khi tập ứng viên có chứa sản phẩm đúng, và tác tử chỉ có giá trị khi nó làm tốt hơn một mô hình gợi ý thuần túy ở những tình huống cụ thể.

- Chất lượng xếp hạng của mô hình gợi ý. Các mô hình cơ sở (Random, Popularity, ItemKNN, BPR-MF) được so sánh trên cùng một phép chia dữ liệu theo thời gian bằng các chỉ số top-K. Kết quả cho biết tín hiệu hành vi thuần túy mạnh đến đâu, và là mốc để đánh giá phần đóng góp của tầng tác tử.
- Độ bao phủ của tập ứng viên. Bước truy xuất quyết định sản phẩm nào đến được bộ xếp hạng; nếu sản phẩm mục tiêu không nằm trong tập ứng viên thì không bộ xếp hạng nào cứu được. Vì vậy candidate recall được đo riêng, trước bước xếp hạng, cho từng nguồn ứng viên.
- Hành vi và giá trị gia tăng của tác tử. Với cùng một tập yêu cầu, các loại tác tử được so sánh về khả năng lập kế hoạch hợp lệ, chọn đúng công cụ, đáp ứng ràng buộc, hỏi lại khi yêu cầu mơ hồ, cũng như chất lượng danh sách, độ trễ và chi phí.

4.1.2 Nguyên tắc thiết kế thực nghiệm

Bốn nguyên tắc sau chi phối mọi cấu hình đánh giá, nhằm bảo đảm các con số so sánh được với nhau và không bị diễn giải quá mức.

- Cùng dữ liệu, cùng định nghĩa mục tiêu. Các mô hình và tác tử được so sánh trên cùng phép chia, cùng quy tắc loại sản phẩm đã tương tác, cùng giá trị K và cùng cách xác định sản phẩm liên quan.
- Chỉ đổi một yếu tố mỗi lần. Khi kiểm chứng một thành phần (công cụ, Memory, Reflection, chế độ inference), các yếu tố còn lại được giữ cố định;

nếu đổi đồng thời nhiều yếu tố thì chênh lệch không quy được cho riêng thành phần nào.

- Tách chất lượng gợi ý khỏi khả năng hoàn thành tác vụ. Một tác tử có thể chạy trọn kế hoạch mà vẫn không đưa sản phẩm mục tiêu vào top-K, nên hai nhóm chỉ số luôn được báo cáo riêng.
- Giữ lại mọi trường hợp lỗi. Lượt chạy lỗi được tính là điểm không thay vì bị loại khỏi mẫu, để tỷ lệ thành công không bị thổi phồng bởi chỉ những yêu cầu chạy trót lọt.

4.2 Dữ liệu và môi trường thực nghiệm

4.2.1 Tập dữ liệu và lý do chọn

Dữ liệu là Amazon Reviews 2023, gồm hai phần: các lượt đánh giá (tạo thành tương tác người dùng – sản phẩm) và metadata sản phẩm (tiêu đề, giá, danh mục, cửa hàng, điểm đánh giá). Ba miền được sử dụng: Video_Games là miền chính, Toys_and_Games và Electronics là miền bổ sung. Việc gộp nhiều miền nhằm hai mục đích: tăng mật độ tương tác cho người dùng có hoạt động ở nhiều miền, và kiểm tra khả năng truy xuất xuyên miền của hệ thống. Mã người dùng của Amazon là toàn cục nên giữ nguyên khi gộp, còn mã sản phẩm được gắn tiền tố miền để tránh trùng nhau giữa các miền.

Một đặc điểm của dữ liệu ảnh hưởng trực tiếp đến thiết kế đánh giá: phần lớn người dùng có rất ít tương tác (khoảng 72% chỉ có một tương tác), và nhiều sản phẩm thiếu thông tin giá. Vì thế người dùng được chia thành ba nhóm theo độ dài lịch sử và mọi chỉ số đều được báo cáo theo nhóm, thay vì chỉ một số trung bình dễ bị nhóm đông chi phối.

Nhóm	Số tương tác trong lịch sử	Ý nghĩa đối với đánh giá
cold	0	Không có tín hiệu hành vi; chỉ dựa được vào điều người dùng nói hoặc độ phổ biến

Nhóm	Số tương tác trong lịch sử	Ý nghĩa đối với đánh giá
sparse	1 – 4	Tín hiệu ít, mô hình cộng tác dễ nhiễu; phép thử cho việc kết hợp văn bản và hành vi
warm	từ 5	Đủ lịch sử để mô hình cộng tác và Memory phát huy

Bảng 4.1. Phân nhóm người dùng theo độ dài lịch sử

4.2.2 Tiền xử lý và phân chia dữ liệu

Quy trình tiền xử lý gồm ba giai đoạn, mỗi giai đoạn lưu kết quả trung gian để có thể chạy lại riêng từng bước. Bảng 4.3 nêu việc làm, tham số và lý do của từng giai đoạn.

Giai đoạn	Việc thực hiện và tham số	Lý do
Làm sạch theo miền	Loại bản ghi thiếu trường bắt buộc; loại trùng theo bộ ba (người dùng, sản phẩm, thời điểm); đọc theo khối 250.000 bản ghi	Tệp đánh giá rất lớn nên không nạp vào bộ nhớ một lần; loại trùng xuyên khối để không bỏ sót bản ghi lặp
Gộp miền và lọc k-core	Gắn nhãn miền, gộp các miền, giữ người dùng và sản phẩm có tối thiểu 5 tương tác	Lọc sau khi gộp để đếm tương tác xuyên miền; ngưỡng 5 loại bớt thực thể quá thưa để mô hình học được
Ánh xạ và chia theo thời gian	Ánh xạ mã sang chỉ số nguyên; chia toàn cục theo thời gian 0,8 / 0,1 / 0,1 thành train, validation, test; chạy ba phép kiểm tra rò rỉ	Chia theo thời gian mô phỏng đúng bài toán dự đoán tương lai từ quá khứ; kiểm tra rò rỉ chặn lỗi trộn dữ liệu giữa các tập

Bảng 4.2. Các giai đoạn tiền xử lý dữ liệu

Phép chia theo thời gian là toàn cục (cùng hai mốc thời điểm cho mọi người dùng), không dùng chia ngẫu nhiên hay giữ lại một tương tác cuối của mỗi người

dùng, vì hai cách đó cho phép mô hình "nhìn thấy tương lai" so với một số người dùng khác. Ba phép kiểm tra rò rỉ gồm: thứ tự thời gian giữa các tập, không có bản ghi trùng giữa các tập, và không có tương tác nào xảy ra sau thời điểm hồ sơ người dùng được chụp (dùng cho Memory).

Cần nêu rõ giới hạn của các phép kiểm tra này: chúng chỉ phát hiện lỗi của chính bước chia. Chúng không phủ ba nguồn rò rỉ tiềm ẩn khác: bộ lọc k-core chạy trên toàn bộ dòng thời gian trước khi chia; ánh xạ mã dựng trên toàn bộ dữ liệu; và điểm đánh giá trung bình, số lượt đánh giá của sản phẩm trong metadata được tổng hợp đến cuối bộ dữ liệu chứ không đến thời điểm chia. Các điểm này được ghi nhận để thận trọng khi diễn giải kết quả ở chương 5.

4.2.3 Kho thông tin sản phẩm

Metadata được nạp vào một kho chỉ đọc, gồm bảng sản phẩm (tiêu đề, cửa hàng, giá, điểm đánh giá, số lượt đánh giá, danh mục chính), bảng danh mục (một sản phẩm có nhiều danh mục) và một chỉ mục tìm kiếm toàn văn. Kho này là nguồn sự thật duy nhất mà các công cụ truy xuất và tra cứu cùng đọc, nên mọi thông tin sản phẩm xuất hiện trong câu trả lời đều có thể đối chiếu lại. Giá được làm sạch về dạng số; sản phẩm không có giá mặc định bị loại khỏi mọi bộ lọc theo giá, trừ khi yêu cầu nói rõ cho phép.

4.2.4 Môi trường và cấu hình chạy

Hệ thống chạy bằng Python trên máy cá nhân. Mô hình ngôn ngữ được gọi qua một giao diện chung tương thích với API OpenAI, nên có thể thay nhà cung cấp mà không đổi logic nghiệp vụ. Các tham số điều khiển tác tử được giữ cố định trong mọi lượt chạy chính: nhiệt độ sinh bằng 0 để kết quả ổn định, kế hoạch tối đa 8 bước, được thử lại 1 lần khi kế hoạch JSON không hợp lệ, và giới hạn 2048 token cho mỗi lần sinh kế hoạch.

Cấu hình chi tiết của máy (CPU, GPU, bộ nhớ, phiên bản thư viện) chưa được tổng hợp thành một bảng thống nhất cho từng lượt chạy; mỗi thư mục thí nghiệm có ghi nhận thông tin nền tảng và phiên bản mã nguồn. Vì vậy các so sánh về thời gian

chạy trong chương 5 chỉ có giá trị tương đối, không dùng để kết luận về hiệu năng tuyệt đối.

4.3 Thiết lập đánh giá mô hình gợi ý

4.4 Các mô hình cơ sở

Bốn mô hình được chọn để đại diện cho bốn mức độ phức tạp, từ mốc tối thiểu đến mô hình học biểu diễn.

Mô hình	Cách hoạt động	Vai trò trong so sánh
Random	Sắp xếp ngẫu nhiên có hạt giống cố định	Mốc dưới: mọi mô hình phải vượt qua
Popularity	Xếp theo số lần sản phẩm được tương tác trong dữ liệu huấn luyện	Mốc đơn giản nhưng khó thắng ở dữ liệu thưa; cũng là phương án dự phòng cho người dùng cold
ItemKNN	Độ tương đồng cosin giữa các sản phẩm từ đồng xuất hiện; giữ 20 láng giềng gần nhất mỗi sản phẩm	Đại diện lọc cộng tác dựa trên sản phẩm; đồng thời là nguồn ứng viên ItemCF của tác tử
BPR-MF	Phân rã ma trận học bằng hàm mất mát xếp hạng theo cặp, tốc độ học 0,001, 20 epoch	Đại diện mô hình học biểu diễn từ phản hồi ngầm

Bảng 4.3 Các mô hình gợi ý cơ sở

Các tham số trên là giá trị cấu hình đang dùng, không đồng nghĩa đã tối ưu siêu tham số. Quy trình tinh chỉnh (nếu chạy) chỉ dùng tập validation để chọn tham số theo NDCG@10; sau đó mô hình được huấn luyện lại trên train và validation rồi đo trên test đúng một lần, để tập test không tham gia vào việc chọn mô hình.

4.4.1 Giao thức huấn luyện và đánh giá

Giao thức fit/target được áp dụng thống nhất: khi đánh giá trên validation, mô hình huấn luyện trên train và dự đoán các tương tác trong validation; khi đánh giá trên test, mô hình huấn luyện trên train cùng validation và dự đoán các tương tác trong test. Với mỗi người dùng, các sản phẩm đã xuất hiện trong dữ liệu huấn luyện bị loại khỏi danh sách đề xuất, vì gợi ý lại sản phẩm đã mua không có ý nghĩa; tập sản phẩm liên quan là các sản phẩm người dùng thực sự tương tác trong giai đoạn đích.

Hệ quả cần lưu ý: nếu một cặp người dùng – sản phẩm lặp lại giữa giai đoạn huấn luyện và giai đoạn đích thì cặp đó vẫn được tính là liên quan nhưng không thể xuất hiện trong danh sách (do bị loại), làm giá trị tuyệt đối của Recall và NDCG thấp hơn thực tế. Mức ảnh hưởng này chưa được đo riêng.

4.4.2 Đánh giá độ bao phủ ứng viên

Với mỗi người dùng được lấy mẫu, mỗi nguồn ứng viên trả về một số lượng sản phẩm cố định (ngân sách 50, 100 và 200); sản phẩm người dùng đã có bị loại; kết quả là tỷ lệ sản phẩm giữ lại (held-out) xuất hiện trong tập ứng viên. Bốn nguồn được đo: Popularity, ItemCF (từ các sản phẩm gần đây trong lịch sử), tìm kiếm văn bản và phép hợp các nguồn (hợp toàn bộ và hợp luân phiên).

Truy vấn của nguồn tìm kiếm văn bản phải được mô phỏng vì không có nhật ký truy vấn thật. Hai cách được dùng và phải phân biệt rõ: dựng từ tiêu đề các sản phẩm gần nhất trong lịch sử (trung thực, nhưng rỗng với người dùng cold) và dựng từ danh mục của chính sản phẩm giữ lại (đây là cận trên, không báo cáo như kết quả thực tế). Một ràng buộc cần ghi nhớ khi đọc kết quả: ItemKNN chỉ giữ 20 láng giềng mỗi sản phẩm nên với số ít sản phẩm hạt giống, ngân sách 200 gần như không thể đạt được.

4.5 Thiết lập đánh giá tác tử

4.5.1 Bốn tác tử được so sánh

Bốn cấu hình được chạy trên cùng tập yêu cầu để cô lập từng lớp đóng góp.

Cấu hình	Cách hoạt động	Câu hỏi mà cấu hình trả lời
baseline	Chỉ dùng ItemKNN thuần, không LLM	Mô hình gợi ý đơn lẻ làm được đến đâu
Type1Agent	Luồng cố định: ItemCF, xếp hạng, tra cứu metadata; không LLM; chỉ áp dụng cho yêu cầu cần sản phẩm hạt giống	Phần đóng góp của việc ghép công cụ khi chưa có LLM
planner	LLM lập kế hoạch với bốn công cụ: tìm kiếm văn bản, ItemCF, xếp hạng, tra cứu	Phần đóng góp của việc LLM chọn và ghép công cụ
planner có Memory	Như planner, thêm công cụ Memory để đọc hồ sơ sở thích từ lịch sử	Phần đóng góp của thông tin cá nhân hóa

Bảng 4.4. Các cấu hình tác tử trong đánh giá

4.5.2 Xây dựng bộ yêu cầu đánh giá

Do không có nhật ký truy vấn thật, bộ yêu cầu được sinh tự động từ dữ liệu giữ lại, với hạt giống ngẫu nhiên cố định để mọi tác tử nhận đúng cùng một tập yêu cầu. Mỗi yêu cầu có loại, nhóm người dùng, câu hỏi bằng ngôn ngữ tự nhiên (theo mẫu tiếng Việt hoặc tiếng Anh), mục tiêu kỳ vọng và, khi cần, ràng buộc giá hoặc danh mục. Năm loại yêu cầu được dùng (Bảng 4.7).

Loại	Cách dựng	Điều kiểm tra	Mức tin cậy
similar	Từ sản phẩm gần nhất trong lịch sử	Truy xuất theo sản phẩm tương tự	Trung thực
personalized	Chỉ từ lịch sử người dùng	Tác dụng của Memory và cá nhân hóa	Trung thực

Loại	Cách dựng	Điều kiểm tra	Mức tin cậy
constrained	Danh mục và giá lấy từ sản phẩm giữ lại	Đáp ứng ràng buộc cấu trúc	ORACLE (cận trên)
cold_text	Danh mục của sản phẩm giữ lại làm mô tả	Tìm kiếm văn bản cho người dùng cold	ORACLE (cận trên)
ambiguous	Câu hỏi chung chung	Hỏi lại thay vì bịa	Thành công là hỏi lại

Bảng 4.5. Các loại yêu cầu đánh giá

Hai loại ORACLE mượn thông tin từ đáp án nên hữu ích để kiểm tra một thành phần riêng lẻ nhưng cho kết quả lạc quan hơn truy vấn tự nhiên; chúng luôn được đánh dấu và không dùng để kết luận về hiệu năng thực tế. Loại ambiguous có tiêu chí ngược: tác tử thành công khi đặt câu hỏi làm rõ, và thất bại khi tự bịa mã sản phẩm hoặc đoán.

4.5.3 Tiêu chí thành công và chấm điểm

Mỗi lượt chạy được chấm theo quỹ đạo đầy đủ chứ không chỉ câu trả lời cuối. Chấm điểm xét các nội dung sau, theo thứ tự từ cơ chế đến kết quả:

- Chọn đúng công cụ: nhóm công cụ mà kế hoạch đi qua có khớp với kế hoạch tham chiếu của loại yêu cầu đó không.
- Tuân thủ ràng buộc: các sản phẩm trong kết quả có thỏa giá và danh mục được yêu cầu không.
- Không bịa mã: mọi mã người dùng hay sản phẩm trong kế hoạch có xuất hiện trong câu yêu cầu hoặc kết quả công cụ không.
- Hỏi lại đúng lúc: với yêu cầu mơ hồ, tác tử có đặt câu hỏi làm rõ không.
- Chất lượng xếp hạng: các chỉ số top-K của danh sách mà người dùng thực sự thấy.

Danh sách "người dùng thực sự thấy" được xác định theo thứ tự ưu tiên: kết quả đã xếp hạng của công cụ xếp hạng, nếu không có thì kết quả của công cụ truy xuất, cuối cùng là kết quả tra cứu metadata. Nhờ vậy việc đo chất lượng không phụ thuộc cách tác tử trình bày. Với loại personalized, tác tử không có công cụ đọc lịch sử không bị chấm điểm thành công, vì yêu cầu đó vốn không thể thực hiện được với công cụ của nó; đây là điểm tránh trừng phạt tác tử vì giới hạn thiết kế.

4.5.4 Ghi nhận quỹ đạo và chỉ số tác tử

Mỗi yêu cầu lưu lại kế hoạch, kết quả từng bước, trạng thái cuối, số bước, số lần gọi công cụ, mức dùng token, độ trễ và thông tin Reflection. Các chỉ số tác tử được chia thành ba nhóm độc lập (Bảng 4.8), khớp với ba nhóm chất lượng đã nêu ở chương 3.

Nhóm	Chỉ số	Cách đọc
Độ tin cậy	Tỷ lệ kế hoạch hợp lệ, tỷ lệ công cụ thành công, số bước, task success	Tác tử có chạy ổn định không
Chất lượng gợi ý	Recall@K, NDCG@K, Hit Rate@K	Danh sách có đúng và xếp tốt không; báo cáo tách khỏi task success
Chi phí	Độ trễ, số token, số lần gọi LLM	Cái giá phải trả cho chất lượng

Bảng 4.6. Nhóm chỉ số đánh giá tác tử

Khi phân tích lỗi, mỗi trường hợp thất bại được xếp vào một trong các nhóm: kế hoạch không hợp lệ, tham số hoặc tham chiếu sai, lỗi công cụ, không tìm thấy ứng viên, kết quả không đáp ứng ràng buộc, và đầu ra không đạt kiểm tra grounding. Phân loại này cho phép chỉ ra phải sửa tầng nào thay vì chỉ biết tỷ lệ thành công thấp.

4.5.5 Đánh giá tầng inference

Tầng inference được đánh giá độc lập bằng cách cố định danh sách sản phẩm rồi so sánh hai chế độ sinh câu trả lời (mẫu và LLM). Phép đo gồm: tỷ lệ đạt kiểm tra grounding (JSON hợp lệ, đúng mã, đủ sản phẩm, đúng thứ tự, giá và điểm trong dung sai), tỷ lệ phải chuyển sang mẫu dự phòng, phân bố bốn trạng thái kết quả, độ trễ và số token. Vì tầng này không chọn hay xếp hạng lại sản phẩm, khác biệt giữa hai chế độ chỉ phản ánh cách diễn đạt và chi phí, không phản ánh chất lượng gợi ý.

4.6 Thiết kế so sánh và ablation

4.6.1 So sánh với mô hình cơ sở

Mọi so sánh với baseline dùng cùng phép chia, cùng bộ yêu cầu, cùng K và cùng định nghĩa mục tiêu. Mô hình offline được báo cáo bằng các chỉ số xếp hạng và candidate recall; tác tử được báo cáo thêm trạng thái tác vụ, quỹ đạo, độ trễ và chi phí. Không dùng một chỉ số đơn lẻ để kết luận toàn hệ thống tốt hơn: một cấu hình có thể thắng ở chỉ số này nhưng thua ở chỉ số khác hoặc ở một nhóm người dùng khác.

4.6.2 Ablation theo thành phần

Ablation thay đổi đúng một yếu tố rồi so sánh kết quả ghép cặp theo mã yêu cầu, tức là mỗi yêu cầu được chạy ở cả hai cấu hình và so sánh trực tiếp.

Yếu tố	Hai nhóm so sánh	Điều kiện giữ cố định
Công cụ	Có và không có từng công cụ trong bộ công cụ của planner	Bộ yêu cầu, mô hình LLM, tham số đánh giá
Memory	planner so với planner có Memory	Bộ yêu cầu và chế độ inference; tách riêng người dùng có đủ lịch sử
Reflection	Số vòng reflection bằng 0 và bằng 1	Chế độ inference, mô hình LLM, bộ yêu cầu

Yếu tố	Hai nhóm so sánh	Điều kiện giữ cố định
Chế độ inference	Mẫu so với LLM	Cùng danh sách sản phẩm đầu vào

Bảng 4.7. Các thí nghiệm ablation

Phân tích thống kê dùng khoảng tin cậy bootstrap cho hiệu số giữa hai nhóm ghép cặp và kiểm định McNemar cho các chỉ số nhị phân như task success. Một khác biệt chỉ được xem là có ý nghĩa khi khoảng tin cậy không chứa 0 hoặc kiểm định đạt ngưỡng, và vẫn phải đối chiếu với số lượng yêu cầu và số lượt chạy. Một lượt chạy cho mỗi nhóm chưa đủ để ước lượng độ biến thiên do LLM, nên các kết luận từ ablation được ghi nhận là sơ bộ cho đến khi lặp lại nhiều lần.

4.6.3 Đánh giá nhiều lượt hội thoại

Đánh giá nhiều lượt nhằm kiểm tra khả năng giữ ngữ cảnh qua các lượt, tận dụng sở thích đã ghi nhận và điều chỉnh kết quả khi người dùng bổ sung hoặc đổi yêu cầu. Mỗi kịch bản cần có lịch sử hội thoại, mục tiêu đầu ra và tiêu chí chấm điểm rõ ràng. Bộ kịch bản và quy trình chạy cho nội dung này chưa hoàn thiện, nên mục này được giữ như một thiết kế đánh giá sẽ thực hiện tiếp và chưa có kết quả đi kèm.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH

Chương này trình bày kết quả của các lượt đánh giá đã thực hiện theo thiết kế ở chương 4. Do các lượt chạy khác nhau về phạm vi, số yêu cầu và cấu hình, bảng đầu tiên liệt kê từng lượt chạy cùng điều kiện của nó; mọi con số phía sau đều gắn với một lượt chạy cụ thể và không được gộp thành một bảng xếp hạng chung.

Lượt chạy	Loại đánh giá	Phạm vi	Ghi chú
exp_003	Mô hình gợi ý offline	Test, miền Video_Games; 95.899 người dùng	Popularity
phase4 (test)	Mô hình gợi ý offline	Test, đa miền; 926.131 người dùng được đánh giá	Popularity
phase4 (validation)	Mô hình gợi ý offline	Validation, đa miền; 1.129.419 người dùng được đánh giá	Popularity
exp_018	Candidate recall	Validation; 1.000 người dùng mỗi phân đoạn cold, sparse, warm	Bốn nguồn ứng viên, ngân sách 50, 100, 200
exp_004 – exp_006	Đánh giá tác tử	Validation, đa miền; 86 yêu cầu mỗi lượt	Cấu hình khác nhau giữa các lượt
exp_007	Đánh giá tác tử	Validation, đa miền; 146 yêu cầu	Inference bằng LLM
exp_008	Đánh giá tác tử	Validation, đa miền; 146 yêu cầu	Inference bằng mẫu; danh sách tác tử khác exp_007

Bảng 5.1. Các lượt chạy được dùng trong chương này

5.1 Kết quả đánh giá hệ thống gợi ý

5.1.1 Kết quả của các mô hình baseline

Các lượt đánh giá baseline hiện có cho thấy chất lượng gợi ý trên tập dữ liệu lớn còn thấp theo các chỉ số top-K (Bảng 5.2). Trên tập test của miền Video_Games, Popularity đạt Recall@10 bằng 0,006182 và NDCG@10 bằng 0,004409. Trên cấu hình đa miền, cũng với Popularity, giá trị tương ứng trên test là 0,002194 và 0,000975.

Lượt chạy	Phạm vi	Recall@10	NDCG@10	Hit Rate@10
exp_003	Test, Video_Games; 95.899 người dùng	0,006182	0,004409	0,018603
phase4 (test)	Test, đa miền; 926.131 người dùng	0,002194	0,000975	0,005475
phase4 (validation)	Validation, đa miền; 1.129.419 người dùng	0,007147	0,003622	0,015009

Bảng 5.2. Kết quả Popularity theo lượt chạy và phạm vi

Các hàng thuộc phạm vi và cấu hình khác nhau nên không dùng được để kết luận mô hình nào tốt hơn. Điều có thể rút ra là: trong cùng cấu hình đa miền, Popularity trên test thấp hơn đáng kể so với validation (Recall@10 từ 0,007147 xuống 0,002194), một dấu hiệu cho thấy khả năng khái quát theo thời gian cần được xem xét. Việc xếp hạng Random, Popularity, ItemKNN và BPR-MF chỉ có ý nghĩa khi cả bốn được chạy trên cùng phép chia, cùng quy tắc loại sản phẩm đã tương tác và cùng định nghĩa mục tiêu, theo giao thức ở chương 4.

5.1.2 Phân tích Recall@K, Precision@K và NDCG@K

Các chỉ số top-K cho thấy những khía cạnh khác nhau. Recall@K đo tỷ lệ mục tiêu được tìm thấy; Precision@K đo độ sạch của danh sách; NDCG@K tính đến vị trí; Hit Rate@K và MRR@K cho biết có tìm thấy mục tiêu hay không và ở thứ hạng nào. Giá trị tuyệt đối của mọi chỉ số đều thấp, và sự chênh lệch giữa Hit Rate@10

(khoảng 0,5% đến 1,9%) với Recall@10 cho thấy phần lớn người dùng không có mục tiêu nào lọt vào top-10.

Recall thấp có nhiều nguyên nhân có thể: tập ứng viên không chứa mục tiêu, giới hạn của dữ liệu thưa và phép chia theo thời gian, hoặc chất lượng bước xếp hạng. Báo cáo chẩn đoán phép chia trên dữ liệu thật chưa được thực hiện, nên chưa đủ cơ sở để quy nguyên nhân cho riêng mô hình xếp hạng.

5.1.3 Phân tích khả năng đưa sản phẩm mục tiêu vào tập ứng viên

Lượt đánh giá candidate recall (exp_018) đo độ bao phủ của từng nguồn ứng viên trước bước xếp hạng, trên validation với 1.000 người dùng mỗi phân đoạn; phân đoạn cold có 4.147 tương tác đích. Bảng 5.3 trình bày kết quả; "Union-RR" là hợp luân phiên các nguồn, "Union-all" là hợp toàn bộ.

Phân đoạn	Nguồn ứng viên	Recall@50	Recall@100	Recall@200
cold	Popularity	0,0153	0,0207	0,0345
cold	ItemCF	0,0000	0,0000	0,0000
cold	Semantic	0,0000	0,0000	0,0000
cold	Union-RR	0,0153	0,0207	0,0345
cold	Union-all	0,0153	0,0207	0,0345
sparse	Popularity	0,0166	0,0215	0,0355
sparse	ItemCF	0,0092	0,0102	0,0102
sparse	Semantic	0,0098	0,0116	0,0154
sparse	Union-RR	0,0155	0,0229	0,0324
sparse	Union-all	0,0286	0,0353	0,0520
warm	Popularity	0,0128	0,0179	0,0283
warm	ItemCF	0,0028	0,0090	0,0090
warm	Semantic	0,0053	0,0067	0,0106

Phân đoạn	Nguồn ứng viên	Recall@50	Recall@100	Recall@200
warm	Union-RR	0,0105	0,0179	0,0235
warm	Union-all	0,0191	0,0292	0,0414

Bảng 5.3. Candidate Recall theo phân đoạn, nguồn ứng viên và ngân sách

Ở hai phân đoạn sparse và warm, Union-all đạt Recall cao nhất tại mọi ngân sách. Với k = 200, Union-all đạt 0,0520 ở sparse so với 0,0355 của Popularity, và 0,0414 ở warm so với 0,0283. Điều này cho thấy kết hợp các nguồn có tiềm năng tăng độ bao phủ khi người dùng đã có một ít lịch sử.

Ở phân đoạn cold, ItemCF và Semantic có Recall bằng 0 tại cả ba ngân sách, nên phép hợp không cải thiện so với Popularity. Kết quả này cần đọc cùng cách dựng truy vấn: một nguyên nhân có thể là với người dùng cold, truy vấn xây từ lịch sử là rỗng nên Semantic và ItemCF không có đầu vào để làm việc (cần kiểm tra thêm). Vì vậy không thể suy ra rằng tìm kiếm văn bản nói chung vô ích, mà chỉ rằng trong cấu hình thử nghiệm này nó chưa đóng góp mục tiêu nào ở phân đoạn cold.

5.1.4 Phân tích các trường hợp gợi ý thất bại

Một nhóm lỗi quan trọng là sản phẩm mục tiêu không được đưa vào tập ứng viên: khi candidate recall bằng 0 thì bước xếp hạng phía sau không thể khôi phục mục tiêu. Nhóm lỗi thứ hai là khoảng cách giữa hoàn thành tác vụ và chất lượng gợi ý, khi một yêu cầu được tính là hoàn thành nhưng danh sách đề xuất vẫn không chứa sản phẩm mục tiêu.

Chưa có đủ dữ liệu chẩn đoán để phân biệt giữa bốn khả năng: mục tiêu không thể tiếp cận theo cách chia dữ liệu, mục tiêu đã xuất hiện trong lịch sử người dùng, lỗi ở bước truy xuất và lỗi ở bước xếp hạng. Đây là các giả thuyết cần kiểm tra thêm chứ chưa phải kết luận về nguyên nhân.

5.2 Kết quả đánh giá tác tử

5.2.1 Kết quả thực hiện các kịch bản đánh giá

Các lượt đánh giá tác tử được thực hiện trên cấu hình dữ liệu đa miền, tập validation. Các lượt exp_004 đến exp_006 có 86 yêu cầu mỗi lượt; exp_007 và exp_008 có 146 yêu cầu. Vì phạm vi và cấu hình khác nhau, các nhóm lượt chạy không được gộp thành một xu hướng duy nhất. Bảng 5.4 tổng hợp những kết quả đã ghi nhận được, theo từng loại yêu cầu.

Loại yêu cầu	Cấu hình	Task success	Recall@10	Ghi chú
similar	baseline, Type1Agent	1,00	0,0000	Hoàn thành tác vụ nhưng không đưa mục tiêu vào top-10
cold_text (ORACLE)	planner	n/a	0,0417	So sánh Memory, 146 yêu cầu
cold_text (ORACLE)	planner có Memory	n/a	0,1917	NDCG@10 tăng 0,0579
constrained (ORACLE)	planner và planner có Memory	n/a	0,1750	Không thay đổi khi thêm Memory
cold_text (ORACLE), n = 20	Reflection 0 vòng	1,00	0,0667	So sánh ghép cặp
cold_text (ORACLE), n = 20	Reflection 1 vòng	0,85	0,0417	So sánh ghép cặp

Bảng 5.4. Một số kết quả đánh giá tác tử theo loại yêu cầu

Quan sát nổi bật là ở loại similar, baseline và Type1Agent đều có task success bằng 1,0 nhưng Recall@10 bằng 0. Điều này khẳng định hai đại lượng là độc lập:

hoàn thành tác vụ không có nghĩa là danh sách đúng, nên task success không thể dùng thay cho Recall@K hay NDCG@K.

Hai loại cold_text và constrained được dựng từ sản phẩm giữ lại để lấy mục tiêu hoặc ràng buộc, nên là cận trên (ORACLE) chứ không phải hiệu năng trên truy vấn tự nhiên. Với loại ambiguous, tiêu chí thành công là hỏi lại để làm rõ thay vì đưa đề xuất ngay.

5.2.2 Phân tích quỹ đạo thực thi (Execution Trajectory)

Mỗi yêu cầu có quỹ đạo thực thi đầy đủ, cho phép kiểm tra kế hoạch do LLM sinh ra, các bước được thực hiện, đầu ra của từng công cụ và trạng thái cuối. Tuy nhiên các số liệu tổng hợp hiện có chưa đủ để khẳng định một kiểu quỹ đạo duy nhất chiếm ưu thế trong mọi loại yêu cầu.

Planner sinh trọn kế hoạch dạng JSON trước khi thực thi tuần tự. Cơ chế Reflection có thể lập lại kế hoạch khi một bước lỗi hoặc không trả về sản phẩm, trừ các trường hợp bị chặn như mã bịa hoặc vượt ngân sách thời gian truy vấn. Cấu trúc này giúp phân tích lỗi theo từng bước, nhưng chưa tương đương vòng lặp ReAct, trong đó LLM quyết định lại sau mỗi kết quả công cụ.

5.2.3 Phân tích khả năng lựa chọn và sử dụng công cụ

Bốn công cụ trong bộ công cụ mặc định của planner đảm nhiệm bốn vai trò khác nhau: tìm kiếm văn bản tạo ứng viên theo từ khóa; ItemCF tạo ứng viên theo sản phẩm tương tự; công cụ xếp hạng chấm điểm ứng viên; công cụ tra cứu lấy metadata. Công cụ Memory chỉ tham gia khi được bật. Vì vậy cần đánh giá cả việc chọn công cụ lẫn đầu ra của từng công cụ, không chỉ xem danh sách cuối có rỗng hay không.

Kết quả ở phân đoạn cold của exp_018 (ItemCF và Semantic đều không tìm được mục tiêu nào) cho thấy cần kiểm tra đầu vào truy xuất, sự có mặt của sản phẩm hạt giống hoặc lịch sử, và cách dựng truy vấn trước khi tinh chỉnh phần lập kế hoạch. Với các yêu cầu tương tự, quỹ đạo cần đối chiếu với candidate recall để biết lỗi phát sinh ở truy xuất hay ở các bước sau.

5.2.4 Phân tích các trường hợp thực thi thất bại

Các trường hợp cần phân tích riêng gồm: kế hoạch không hợp lệ, tham số công cụ sai lược đồ, tham chiếu kết quả bước trước không hợp lệ, lỗi công cụ, không tìm thấy ứng viên, đầu ra không đạt grounding, và trường hợp được ghi nhận thành công nhưng Recall bằng 0. Các nhóm này không nên gộp thành một loại lỗi chung vì mỗi loại cần hướng khắc phục khác nhau.

Chênh lệch giữa task success và Recall@10 trong các kết quả hiện có là bằng chứng rõ ràng cho việc phải tách hành vi thực thi khỏi chất lượng đề xuất. Nguyên nhân chi tiết của từng lỗi phải được xác nhận bằng quỹ đạo và yêu cầu tương ứng, không suy diễn từ một chỉ số tổng hợp.

5.3 Kết quả thực nghiệm so sánh và Ablation

5.3.1 So sánh với baseline

Mô hình offline và tác tử được đánh giá ở hai tầng khác nhau: mô hình offline đo chất lượng xếp hạng trên phép chia dữ liệu; tác tử đo việc thực thi yêu cầu, dùng công cụ và tạo đầu ra. Các lượt chạy khác nhau về phạm vi, số yêu cầu, chế độ inference hoặc cấu hình không thể ghép thành một bảng xếp hạng chung.

Về độ bao phủ ứng viên, Union-all cải thiện so với Popularity ở sparse và warm trong exp_018. Tuy nhiên mức Recall@200 cao nhất vẫn chỉ là 0,0520 ở sparse và 0,0414 ở warm, nên độ bao phủ ứng viên vẫn là hạn chế đáng kể của cấu hình hiện tại.

5.3.2 Phân tích đóng góp của từng thành phần

Kết quả candidate recall gợi ý rằng kết hợp nguồn hữu ích hơn từng nguồn riêng lẻ ở các phân đoạn có lịch sử. Nhưng phép hợp không giúp gì ở cold, vì cả ItemCF và Semantic đều bằng 0; thêm nhiều nguồn không làm tăng độ bao phủ nếu các nguồn đó không sinh được ứng viên phù hợp.

Với tầng inference, chế độ LLM diễn đạt các sản phẩm tác tử đã chọn, còn chế độ mẫu tạo câu trả lời theo khuôn. Do tầng này không chọn lại hay xếp hạng lại sản

phẩm, chất lượng gợi ý phải được tách khỏi grounding, tỷ lệ fallback, độ trễ và số token của câu trả lời cuối.

5.3.3 Phân tích ảnh hưởng của Memory và Reflection

Memory. So sánh planner với planner có Memory trên 146 yêu cầu cho thấy ở cold_text, Recall@10 tăng từ 0,0417 lên 0,1917 và NDCG@10 tăng 0,0579; ở constrained, Recall@10 không đổi ở 0,1750. Đây là tín hiệu tích cực cục bộ, nhưng cold_text và constrained đều là loại ORACLE, hai loại trung thực là personalized và similar vẫn có Recall@10 bằng 0, và mỗi nhóm chỉ có một lượt chạy. Vì vậy chưa thể kết luận Memory cải thiện toàn hệ thống.

Reflection. So sánh số vòng reflection bằng 0 và bằng 1 dùng 146 yêu cầu ghép cặp, nhưng mỗi nhóm chỉ có một lượt chạy. Ở cold_text với n = 20, nhóm không reflection có task success 1,00 và Recall@10 bằng 0,0667; nhóm một vòng có task success 0,85 và Recall@10 bằng 0,0417. Kết quả này không chứng minh Reflection luôn làm giảm chất lượng, nhưng cũng không ủng hộ việc cho rằng Reflection đang cải thiện kết quả.

Hơn nữa, exp_007 dùng inference LLM còn exp_008 dùng mẫu và có danh sách tác tử khác, nên hai lượt này không phải một ablation Reflection thuần. Muốn kết luận cần giữ cố định chế độ inference, bộ yêu cầu, mô hình, lời nhắc và dữ liệu, rồi chạy lặp để ước lượng độ biến thiên do LLM.

5.4 Đánh giá hiệu năng và chi phí thực thi

- Thời gian thực thi: Các lượt chạy inference có ghi độ trễ và mức dùng LLM khi nhà cung cấp trả về. Tuy nhiên chưa có bộ đo thời gian chạy thống nhất để báo cáo trung vị hay P95 theo từng công cụ hoặc toàn bộ quy trình. Độ trễ phụ thuộc mô hình, điểm cuối, mạng, giới hạn thời gian và số lần thử lại, nên không đưa ra kết luận tối ưu hiệu năng khi chưa đo trong cùng điều kiện.
- Số bước thực thi và số lần gọi công cụ: Quỹ đạo và chỉ số của tác tử có ghi số bước, số lần gọi công cụ, tính hợp lệ của kế hoạch và hoạt động Reflection. Đây là cơ sở để phân tích chi phí thực thi, nhưng cần tổng hợp theo loại yêu

cầu và cấu hình. Số liệu hiện có chưa đồng nhất để đưa ra một giá trị trung bình đại diện cho toàn hệ thống.

- Token sử dụng và chi phí LLM: Tầng LLM ghi nhận số token và độ trễ cho các lần gọi mà nhà cung cấp hỗ trợ; chi phí có thể suy ra khi biết mô hình, số token vào ra và đơn giá. Chưa có bảng chi phí chuẩn hóa giữa các cấu hình, nên không đưa ra con số chi phí tổng quát.
- Đánh đổi giữa chất lượng kết quả và hiệu năng: Cần đánh giá đồng thời chất lượng đề xuất, khả năng hoàn thành tác vụ và chi phí vận hành. Reflection có thể tạo thêm lời gọi LLM; inference bằng LLM tốn hơn inference bằng mẫu, trong khi chất lượng xếp hạng không nhất thiết đổi khi đổi cách diễn đạt. Vì chưa có phép đo thời gian chạy đồng nhất, các nhận định này mô tả đặc điểm kiến trúc và hướng đánh giá, chưa phải kết luận định lượng về mức đánh đổi tối ưu.

5.5 Thảo luận kết quả

5.5.1 Những kết quả đạt được

- Có các lượt đánh giá baseline trên validation và test cùng các chỉ số xếp hạng cơ bản.
- Có một bộ đánh giá candidate recall với nhiều nguồn ứng viên, ngân sách top-K và phân đoạn người dùng.
- Trong exp_018, Union-all có Recall cao hơn Popularity ở các phân đoạn sparse và warm tại mọi ngân sách được báo cáo.
- Có các lượt đánh giá tác tử cùng so sánh sơ bộ về Memory, Reflection và chế độ inference.
- Kết quả chỉ ra rõ rằng task success không thay thế được chỉ số về độ liên quan của danh sách sản phẩm.

5.5.2 Những vấn đề chưa giải quyết được

- Recall của các nguồn ứng viên còn thấp; ở phân đoạn cold, ItemCF và Semantic không đưa được mục tiêu nào vào tập ứng viên.
- Chưa có chẩn đoán phép chia trên dữ liệu thật để xác định mức mục tiêu có thể tiếp cận và nguyên nhân Recall thấp.
- Ablation Memory và Reflection chưa đủ số lượt chạy để ước lượng độ biến thiên hoặc kết luận ổn định.
- So sánh giữa exp_007 và exp_008 bị nhiễu bởi khác biệt chế độ inference và danh sách tác tử.
- Chưa có bằng chứng đầy đủ cho đánh giá nhiều lượt hội thoại.
- Chưa có đo thời gian chạy thống nhất để kết luận về độ trễ, P95 và chi phí tổng thể.

5.5.3 Các yếu tố ảnh hưởng đến kết quả

Kết quả có thể chịu ảnh hưởng của khả năng tiếp cận mục tiêu trong phép chia theo thời gian, lịch sử tương tác của người dùng, cách dựng truy vấn, ngân sách ứng viên, cấu hình công cụ, chế độ inference và biến thiên từ LLM. Số yêu cầu ở một số phân đoạn còn nhỏ, và các loại ORACLE không đại diện cho truy vấn tự nhiên. Vì vậy mọi so sánh cần nêu rõ số mẫu, cấu hình, phép chia và giới hạn diễn giải

CHƯƠNG 6. CẢI THIỆN VÀ HƯỚNG PHÁT TRIỂN

Báo cáo này được nộp ở giữa kỳ, khi hệ thống đã có đủ các thành phần chính (dữ liệu, kho sản phẩm, công cụ, mô hình gợi ý, tác tử, tầng inference và bộ đánh giá) nhưng bằng chứng thực nghiệm mới ở mức ban đầu.

Vì vậy chương này chia làm hai phần: phần thứ nhất là những cải thiện cần hoàn thành từ nay đến hạn nộp đồ án (21/11/2026); phần thứ hai là các hướng phát triển nằm ngoài phạm vi đồ án, dành cho giai đoạn sau.

6.1 Cải thiện cần thực hiện đến cuối kỳ

6.1.1 Các vấn đề cần giải quyết

Từ kết quả ở chương 5, sáu vấn đề được nhận diện. Bảng 6.1 nêu từng vấn đề, bằng chứng và cải thiện tương ứng; các cải thiện được mô tả chi tiết ở các mục sau.

Vấn đề	Bằng chứng hiện có	Cải thiện đề xuất
Độ bao phủ ứng viên thấp, đặc biệt ở người dùng cold	Recall@200 cao nhất chỉ 0,0520 (sparse) và 0,0414 (warm); ItemCF và Semantic bằng 0 ở cold	Thiết kế lại cách tạo truy vấn và nguồn dự phòng cho cold; so sánh các chiến lược hợp nhất trong cùng điều kiện
Nguyên nhân Recall thấp chưa được xác định	Chưa có báo cáo chẩn đoán phép chia trên dữ liệu thật	Chẩn đoán khả năng tiếp cận mục tiêu, mục tiêu đã có trong lịch sử, tác động của việc loại sản phẩm đã mua
Baseline chưa được so sánh đầy đủ	Mới có kết quả Popularity, thuộc các phạm vi khác nhau	Chạy Random, Popularity, ItemKNN, BPR-MF trên cùng phép chia và cùng K
Ablation chưa đủ chặt chẽ	Mỗi nhóm một lượt chạy; chế độ inference và danh sách tác tử khác nhau	Giữ cố định mọi yếu tố trừ thành phần kiểm chứng; chạy lặp nhiều lần

Vấn đề	Bằng chứng hiện có	Cải thiện đề xuất
Tầng inference chưa nối với giao diện demo	Giao diện demo vẫn dùng câu trả lời mẫu	Hoàn thiện bộ chuyển đổi và kiểm thử luồng đầu-cuối
Chưa đo thời gian và chi phí	Chưa có số trung vị, P95 hay chi phí theo cấu hình	Đo thống nhất độ trễ, số bước, số lần gọi LLM và token

Bảng 6.1. Các vấn đề hiện tại và cải thiện tương ứng

6.1.2 Cải thiện chất lượng và độ bao phủ của tập ứng viên

Đây là hạng mục ưu tiên cao nhất, vì chất lượng danh sách cuối bị chặn trên bởi tập ứng viên: không bộ xếp hạng nào cứu được mục tiêu đã vắng mặt. Công việc gồm bốn việc.

- Chẩn đoán trước, tối ưu sau. Với mỗi mục tiêu giữ lại, ghi nhận mục tiêu có tiếp cận được theo phép chia hay không (sản phẩm đã có mặt trong dữ liệu huấn luyện chưa), đã xuất hiện trong lịch sử người dùng chưa, và nằm ở vị trí nào trong từng nguồn ứng viên. Kết quả cho biết nên sửa truy xuất hay sửa xếp hạng, thay vì đoán.
- Xử lý người dùng cold. Với người dùng không có lịch sử, nguồn ItemCF không có sản phẩm hạt giống và nguồn tìm kiếm văn bản không có truy vấn. Cải thiện là cho phép tác tử lấy thông tin từ chính câu yêu cầu (danh mục, từ khóa, khoảng giá) để dựng truy vấn, và dùng Popularity theo danh mục thay cho Popularity toàn cục làm nguồn dự phòng.
- So sánh chiến lược hợp nhất trong cùng điều kiện. Popularity, ItemCF, tìm kiếm văn bản, Union-RR và Union-all được đánh giá trên cùng tập người dùng, cùng mục tiêu và cùng ngân sách, theo từng phân đoạn. Chỉ chọn chiến lược cuối cùng sau khi có kết quả cố định và tái lập được.

- Rà soát giới hạn của ItemCF. Vì mỗi sản phẩm chỉ giữ một số ít láng giềng gần nhất, ngân sách ứng viên lớn gần như không đạt được; cần thử tăng số láng giềng và đo tác động tới Recall lẫn chi phí.

Mục tiêu không chỉ là tăng một chỉ số tổng thể mà còn giảm chênh lệch giữa các phân đoạn: nếu một nguồn không hữu ích với một nhóm người dùng thì hệ thống cần có nguồn dự phòng thay vì giả định mọi nguồn đều đóng góp như nhau.

6.1.3 Hoàn thiện đánh giá mô hình gợi ý và tác tử

Cải thiện này nhằm biến các kết quả sơ bộ thành kết luận có thể bảo vệ được. Ba việc chính:

- Benchmark đầy đủ cho mô hình gợi ý. Chạy bốn mô hình cơ sở trên cùng phép chia, báo cáo Recall, Precision, Hit Rate, NDCG, MRR ở K = 5, 10, 20 và theo ba phân đoạn người dùng; đây là mốc để đánh giá phần đóng góp của tầng tác tử.
- Chuẩn hóa đánh giá tác tử. Cố định bộ yêu cầu và hạt giống, cùng cấu hình công cụ; so sánh bốn cấu hình (baseline, Type1Agent, planner, planner có Memory) trên cùng tập yêu cầu. Loại yêu cầu ORACLE luôn được đánh dấu riêng; số lượng yêu cầu mỗi loại được nâng lên để các phân đoạn nhỏ không còn dựa trên vài chục mẫu.
- Ablation có kiểm soát và chạy lặp. Mỗi lần chỉ đổi một yếu tố (Memory, Reflection hoặc chế độ inference), giữ cố định phần còn lại, chạy lặp ít nhất vài lần để ước lượng độ biến thiên do LLM, và báo cáo khoảng tin cậy cùng kiểm định theo thiết kế ở chương 4.

Nếu không đủ thời gian lặp đủ số lần, kết luận về Memory và Reflection sẽ tiếp tục được trình bày là sơ bộ và không khẳng định mức cải thiện tổng quát.

6.1.4 Củng cố độ tin cậy của công cụ và tác tử

Các cải thiện về mặt kỹ thuật tập trung vào những điểm đã quan sát được khi đọc quỹ đạo thực thi và kiểm thử:

- Chặn lỗi trước khi thực thi. Giữ các chốt kiểm tra kế hoạch (công cụ có tồn tại, tham số đúng lược đồ, tham chiếu đến kết quả bước trước hợp lệ, không vượt số bước tối đa, không dùng mã bịa) và bổ sung kiểm thử cho các trường hợp JSON hỏng, công cụ trả kết quả rỗng, thiếu metadata.
- Hồ sơ sở thích của Memory. Công cụ Memory hiện tổng hợp danh mục và khoảng giá chỉ trên một phần lịch sử khi người dùng có nhiều sản phẩm, nên hồ sơ có thể lệch. Cần dùng toàn bộ lịch sử hoặc ưu tiên các tương tác gần nhất, đồng thời giữ cơ chế chặn tương tác sau thời điểm đánh giá để không rò rỉ.
- Quyết định về công cụ truy vấn có cấu trúc. Công cụ lọc theo thuộc tính (SQLTool) đã cài đặt nhưng chưa nằm trong bộ công cụ của planner. Cần quyết định có đưa vào hay không; nếu đưa vào thì cập nhật lời nhắc và bổ sung kiểm thử, nếu không thì ghi rõ trong báo cáo như một lựa chọn thiết kế.
- Đồng bộ đầu ra của công cụ với lời nhắc. Mô tả đầu ra của từng công cụ hiện chưa được đưa vào lời nhắc của planner; bổ sung giúp LLM tham chiếu đúng trường dữ liệu của bước trước.

6.1.5 Hoàn thiện tầng inference và giao diện demo

Tầng inference đã có hai chế độ (mẫu và LLM) cùng cơ chế kiểm tra grounding, nhưng bộ chuyển đổi nối kết quả của tác tử với giao diện demo chưa hoàn chỉnh nên demo vẫn dùng câu trả lời mẫu. Công việc gồm hoàn thiện bộ chuyển đổi này, kiểm thử bốn trạng thái kết quả (ok, ask_user, no_result, tool_error) bằng các tình huống có đầu ra xác định, so sánh hai chế độ trên cùng danh sách sản phẩm đầu vào, và chạy thử luồng từ yêu cầu của người dùng đến danh sách kèm phần giải thích. Chỉ khi các bước đó đạt thì demo mới được mô tả là hoạt động đầu-cuối.

6.1.6 Đo thời gian và chi phí

Việc tối ưu bắt đầu bằng đo lường thống nhất: ghi tổng thời gian và thời gian từng giai đoạn (lập kế hoạch, thực thi từng công cụ, sinh câu trả lời); số lần gọi LLM, số token vào/ra theo từng cấu hình; báo cáo trung vị và P95 khi đủ số lần chạy. Sau

đó mới xét biện pháp giảm chi phí như bớt bước dư, giới hạn lượng metadata đưa vào lời nhắc, hoặc tái sử dụng kết quả công cụ khi an toàn. Mọi biện pháp tối ưu chỉ được đưa vào cấu hình mặc định sau khi có phép đo trước và sau để biết mức cải thiện đến từ thay đổi kỹ thuật chứ không phải từ khác biệt điều kiện chạy.

6.1.7 Đánh giá nhiều lượt hội thoại ở phạm vi nhỏ

Đánh giá nhiều lượt là hạng mục có rủi ro cao nhất vì cần bộ kịch bản có trạng thái và đầu ra kỳ vọng. Trong khuôn khổ đồ án, phạm vi được thu hẹp: một số kịch bản ngắn (khoảng 2 đến 3 lượt) trong đó người dùng bổ sung hoặc đổi một ràng buộc như giá hoặc danh mục. Tiêu chí là tác tử cập nhật đúng ràng buộc mới và không dùng ràng buộc đã lỗi thời; toàn bộ hội thoại, trạng thái sau mỗi lượt và lời gọi công cụ được ghi lại để tái lập. Nếu không hoàn thành kịp, nội dung này được chuyển sang phần hướng phát triển và không báo cáo kết quả chưa có.

6.1.8 Kế hoạch thực hiện đến hạn nộp

Bốn tuần còn lại được chia thành bốn giai đoạn, mỗi giai đoạn có tiêu chí hoàn thành đo được (Bảng 6.2). Mốc thời gian là dự kiến và có thể điều chỉnh theo tiến độ thực tế.

Giai đoạn	Công việc chính	Tiêu chí hoàn thành
Tuần 1 (24/10 – 30/10)	Chẩn đoán phép chia; chốt tập đánh giá, hạt giống và quy tắc tính chỉ số; benchmark đầy đủ bốn mô hình cơ sở	Có báo cáo chẩn đoán và bảng chỉ số trên cùng phép chia, theo phân đoạn
Tuần 2 (31/10 – 6/11)	Cải thiện nguồn ứng viên cho cold; so sánh các chiến lược hợp nhất; sửa các điểm yếu của Memory và công cụ	Có bảng candidate recall mới so với exp_018; kiểm thử các sửa đổi đạt

Giai đoạn	Công việc chính	Tiêu chí hoàn thành
Tuần 3 (7/11 – 13/11)	Đánh giá tác tử trên bộ yêu cầu mở rộng; ablation Memory, Reflection, inference chạy lặp; đo thời gian và chi phí; hoàn thiện bộ chuyển đổi inference	Có số liệu có độ biến thiên cho từng ablation; demo chạy đầu-cuối
Tuần 4 (14/11 – 21/11)	Đánh giá nhiều lượt ở phạm vi nhỏ (nếu kịp); tổng hợp, rà soát bảng biểu và kết luận; hoàn thiện báo cáo	Mọi kết luận định lượng truy ngược được về lượt chạy và điều kiện so sánh

Bảng 6.2. Kế hoạch theo tuần đến hạn nộp

Nếu thiếu thời gian, thứ tự ưu tiên giữ nguyên như sau: (1) chẩn đoán phép chia và benchmark cùng điều kiện, (2) cải thiện nguồn ứng viên, (3) ablation có kiểm soát, (4) tích hợp inference và demo, (5) đo thời gian và chi phí, (6) đánh giá nhiều lượt. Các hạng mục cuối có thể được trình bày là hạn chế thay vì báo cáo kết quả chưa đủ chắc.

6.2 Hướng phát triển sau đồ án

Các hướng dưới đây vượt quá thời gian của đồ án nhưng xuất phát trực tiếp từ những hạn chế đã quan sát. Chúng là định hướng nghiên cứu, không phải cam kết sẽ hoàn thành trong đồ án.

- Tác tử có vòng lặp quan sát – quyết định: Kiến trúc hiện tại lập kế hoạch trọn vẹn một lần rồi thực thi (Reflection chỉ lập lại khi có lỗi), nên LLM không điều chỉnh bước kế tiếp dựa trên kết quả vừa nhận. Hướng phát triển là chuyển sang vòng lặp có trạng thái: LLM chọn một công cụ, quan sát kết quả, quyết định bước tiếp theo, rồi mới viết câu trả lời cuối. Cách này giúp xử lý trường hợp công cụ trả kết quả rỗng hoặc không đáp ứng ràng buộc, và là nền tảng

cho hội thoại nhiều lượt. Đổi lại, độ trễ và chi phí tăng, và cần cơ chế kiểm soát số bước để vòng lặp không trôi.

- Truy xuất bằng biểu diễn vector: Tìm kiếm văn bản hiện dựa trên khớp từ khóa (BM25), nên không bắt được các diễn đạt đồng nghĩa hay mô tả bằng tiếng Việt cho sản phẩm có tiêu đề tiếng Anh. Thêm truy xuất bằng biểu diễn vector (embedding), kết hợp với khớp từ khóa, có thể tăng độ bao phủ cho người dùng cold. Hướng này chỉ nên triển khai nếu phép đo candidate recall sau khi cải thiện ở phần trên cho thấy từ khóa vẫn là điểm nghẽn.
- Mô hình xếp hạng mạnh hơn cho tác tử: Bộ xếp hạng mà tác tử dùng hiện chỉ bọc các mô hình đơn giản; BPR-MF mới được dùng trong đánh giá offline, chưa là bộ chấm điểm cho tác tử. Hướng phát triển gồm đưa BPR-MF và các mô hình tuần tự (khai thác thứ tự thời gian của lịch sử mua) vào làm bộ chấm điểm, cùng bổ sung đặc trưng từ metadata như giá, danh mục, cửa hàng. Giao diện chấm điểm đã được thiết kế để thay thế bộ chấm điểm mà không phải sửa tác tử.
- Bộ nhớ và hội thoại nhiều lượt thực sự: Memory hiện chỉ đọc hồ sơ được tổng hợp từ lịch sử mua, vì dữ liệu Amazon không có hội thoại để trích sở thích. Khi có dữ liệu hội thoại hoặc nhật ký sử dụng thật, hệ thống có thể ghi, cập nhật và quên sở thích, xử lý mâu thuẫn giữa sở thích cũ và yêu cầu mới, và đánh giá tính nhất quán qua nhiều lượt. Hướng này cũng đòi hỏi suy xét về quyền riêng tư của thông tin được lưu.
- Đánh giá bằng truy vấn và người dùng thật: Yêu cầu đánh giá hiện được sinh từ dữ liệu giữ lại, và một số loại là cận trên (ORACLE). Bước tiếp theo là thu thập truy vấn tự nhiên thật hoặc khảo sát người dùng, kết hợp đánh giá bằng LLM có kiểm soát cho phần diễn giải. Việc này mới cho biết hệ thống hữu ích đến đâu với truy vấn thực, điều mà đánh giá offline không trả lời được.
- Dữ liệu, miền và ngôn ngữ: Metadata hiện chứa điểm đánh giá và số lượt đánh giá được tổng hợp đến cuối bộ dữ liệu, không phải tại thời điểm chia, nên là một nguồn rò rỉ tiềm ẩn. Hướng phát triển gồm tái dựng các thống kê đó đến

đúng thời điểm, mở rộng thêm miền sản phẩm, và hỗ trợ tốt hơn câu hỏi bằng tiếng Việt trên sản phẩm có metadata tiếng Anh.

– Triển khai và vận hành: Để đưa hệ thống thành dịch vụ, cần lớp phục vụ qua giao diện lập trình, bộ nhớ đệm cho kết quả công cụ, theo dõi độ trễ và chi phí theo thời gian thực, và cơ chế dự phòng khi nhà cung cấp LLM lỗi. Tầng inference đã luôn trả về một câu trả lời hợp lệ khi có lỗi, đây là nền tảng cho việc này.

Hướng	Động lực từ kết quả hiện tại	Điều kiện cần
Vòng lặp quan sát – quyết định	Kế hoạch tĩnh không thích ứng khi công cụ trả rỗng	Cơ chế giới hạn bước và đo chi phí
Truy xuất bằng vector	Recall của tìm kiếm từ khóa thấp ở cold	Kết quả candidate recall sau cải thiện
Bộ xếp hạng mạnh hơn	Chất lượng xếp hạng tuyệt đối còn thấp	Giao diện chấm điểm đã có; cần dữ liệu và tài nguyên huấn luyện
Bộ nhớ và nhiều lượt	Chưa có đánh giá nhiều lượt	Dữ liệu hội thoại hoặc nhật ký sử dụng
Đánh giá với truy vấn thật	Nhiều yêu cầu là ORACLE	Người dùng hoặc nhật ký truy vấn thật
Dữ liệu, miền, ngôn ngữ	Thống kê metadata chưa theo thời điểm	Dựng lại metadata theo thời gian
Triển khai dịch vụ	Chưa đo thời gian và chi phí	Hoàn thành đo lường ở phần trên

Bảng 6.3. Tổng hợp các hướng phát triển

KẾT LUẬN

Đồ án xây dựng một hệ thống gợi ý sản phẩm theo hướng Agentic RAG, trong đó mô hình ngôn ngữ lớn chỉ quyết định cần làm gì, còn việc truy xuất, tra cứu và xếp hạng do các công cụ có kiểm soát đảm nhiệm. Trên dữ liệu Amazon Reviews 2023 (ba miền Video_Games, Toys_and_Games, Electronics), đến thời điểm giữa kỳ nhóm đã hoàn thành quy trình xử lý dữ liệu và chia theo thời gian, kho thông tin sản phẩm, bộ công cụ cho tác tử, các mô hình gợi ý cơ sở, hai loại tác tử (lập kế hoạch bằng LLM và luồng cố định), tầng inference có kiểm tra grounding, cùng bộ đánh giá nhiều tầng từ mô hình gợi ý, tập ứng viên đến tác tử.

Các kết quả thực nghiệm hiện có mới ở mức ban đầu nhưng đã chỉ ra những điểm đáng chú ý. Việc kết hợp nhiều nguồn ứng viên giúp tăng độ bao phủ ở nhóm người dùng có lịch sử, song chưa giải quyết được người dùng chưa có lịch sử. Hoàn thành tác vụ không đồng nghĩa với gợi ý đúng: ở nhiều yêu cầu, tác tử đạt task success cao nhưng Recall@10 vẫn bằng 0, vì vậy hai nhóm chỉ số phải được đánh giá riêng. Memory cho tín hiệu cải thiện cục bộ và Reflection chưa cho thấy lợi ích rõ ràng, nhưng thiết kế thí nghiệm hiện tại chưa đủ để kết luận tổng quát.

Do bằng chứng còn hạn chế, đồ án chưa khẳng định được hệ thống đã tối ưu về chất lượng gợi ý, độ trễ hay chi phí. Các việc cần hoàn thành đến cuối kỳ gồm: chẩn đoán phép chia dữ liệu và chạy đầy đủ các mô hình cơ sở trên cùng điều kiện, cải thiện nguồn ứng viên cho người dùng cold, thực hiện các phép ablation có kiểm soát và chạy lặp, hoàn thiện kết nối tầng inference với giao diện demo, đo thời gian và chi phí, và đánh giá nhiều lượt hội thoại ở phạm vi nhỏ nếu kịp. Những nội dung chưa có kết quả xác nhận được xem là hạn chế, không được coi là chức năng đã hoàn thành.

Nhìn chung, ở giai đoạn giữa kỳ đồ án đã có nền tảng kỹ thuật để nghiên cứu một hệ thống gợi ý phối hợp nhiều công cụ thông qua tác tử, cùng cách đánh giá tách bạch độ bao phủ ứng viên, chất lượng xếp hạng và hành vi thực thi. Giai đoạn tiếp

theo ưu tiên chuẩn hóa thực nghiệm và hoàn thiện tích hợp đầu-cuối để có kết luận đáng tin cậy; các hướng mở rộng như tác tử có vòng lặp quan sát – quyết định, truy xuất bằng vector và đánh giá với người dùng thật được để lại cho giai đoạn phát triển sau đồ án.

TÀI LIỆU THAM KHẢO

Tiếng Anh

1. Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, *17*(6), 734–749.
2. Bruch, S., Gai, S., & Ingber, A. (2023). An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, *42*(1). https://doi.org/10.1145/3596512
3. Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for YouTube recommendations. *Proceedings of the 10th ACM Conference on Recommender Systems*, 191–198.
4. Cremonesi, P., Koren, Y., & Turrin, R. (2010). Performance of recommender algorithms on top-n recommendation tasks. *Proceedings of the 4th ACM Conference on Recommender Systems*, 39–46.
5. Deshpande, M., & Karypis, G. (2004). Item-based top-n recommendation algorithms. *ACM Transactions on Information Systems*, *22*(1), 143–177.
6. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.
7. Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., … Chen, E. (2024). *Understanding the planning of LLM agents: A survey*. Retrieved from https://arxiv.org/abs/2402.02716
8. Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, *20*(4), 422–446.
9. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., … Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), 1–38.
10. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., … Yih, W. -t. (2020). Dense passage retrieval for open-domain question answering.

Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP, 6769–6781.

11. Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 39–48.
12. Klimashevskaia, A., Jannach, D., Elahi, M., & Trattner, C. (2024). A survey on popularity bias in recommender systems. *User Modeling and User-Adapted Interaction*, *34*(5), 1777–1808.
13. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, *42*(8), 30–37.
14. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., … Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in neural information processing systems* (Vol. 33, pp. 9459–9474).
15. Lin, X., Deldjoo, Y., Dai, S., Bao, H., Ye, X., Nazary, F., … Chua, T.-S. (2026). *Autonomous information seeking: A roadmap for agentic recommender systems*. Retrieved from https://arxiv.org/abs/2607.04433
16. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
17. Panda, M., & Ray, S. (2022). Cold-start problem in recommender systems: A systematic review. *Knowledge-Based Systems*, *241*, 108213.
18. Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 1–22.
19. Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. *OpenAI Technical Report*.

20. Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence*, 452–461.
21. Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work*, 175–186.
22. Ricci, F., Rokach, L., & Shapira, B. (2022). *Recommender systems handbook* (3rd ed.). Springer.
23. Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4), 333–389.
24. Salton, G., & McGill, M. J. (1983). *Introduction to modern information retrieval*. McGraw-Hill.
25. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th International Conference on World Wide Web*, 285–295.
26. Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 253–260.
27. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., … Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in neural information processing systems* (Vol. 36, pp. 68539–68551).
28. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in neural information processing systems* (Vol. 36, pp. 8634–8652).

29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (Vol. 30, pp. 5998–6008).
30. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., … Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, *18*(6), 186345.
31. Wu, S., Sun, F., Zhang, W., Xie, X., & Cui, B. (2023). Graph neural networks in recommender systems: A survey. *ACM Computing Surveys*, *55*(5).
32. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR*.
33. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., … Stoica, I. (2023). Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in neural information processing systems* (Vol. 36, pp. 46595–46623).